PHINMA UNIVERSITY OF ILOILO

College of Information Technology Education

Condominium Booking and Operations Management System

In Partial Fulfillment of the Requirements

for the Degree of Bachelor of Science in Information Technology

Nenia May Buganas

Steven Christian Felarca

Mark Joseph Goboy

Edjie Padios

Dona May Talam

Data Informatics Track

Second Semester

2025–2026

TABLE OF CONTENTS

TABLE OF CONTENTS 2
CHAPTER I 7
INTRODUCTION 7
1.1 Background of the Study 7
1.2 Statement of the Problem 8
1.3 Objectives of the Study 10
1.3.1 General Objective 10
1.3.2 Specific Objectives 11
1.4 Scope and Limitations 12
1.4.1 Scope of the Study 12
1.4.2 Delimitations of the Study 13
1.5 Significance of the Study 14
1.6 Definition of Terms 16
1.7 Conceptual Framework 19
1.8 Organization of the Study 21
1.9 Chapter Summary 22
REFERENCES 23
REVIEW OF RELATED LITERATURE 25
2.1 Foreign Studies 25
2.2 Local Studies 26
2.3 Related Literature 26
Property Management Systems 26
Automated Booking and Reservation Management 27
Payment Tracking and Financial Record Management 28
Revenue Monitoring and Financial Reporting 29
Maintenance Management in Property Operations 30
Calendar-Based Scheduling in Operations Management 30
Centralized Database Systems in Information Management 31
Web-Based Information Systems 32
Agile Software Development Methodology 32
2.4 Synthesis of the Reviewed Literature 33
Convergence of Key Themes Across the Literature 34

The Pervasive Inadequacy of Manual Property Management Processes 35
The Demonstrated Value of Centralized and Integrated Information Systems 35
The Strategic Importance of Booking and Reservation Management 36
The Critical Role of Accurate Financial and Payment Tracking 36
Maintenance Management as a Driver of Tenant Retention 37
Database Architecture as the Backbone of System Reliability 37
Web-Based Accessibility as an Enabler of Operational Continuity 37
Agile Methodology as the Appropriate Development Framework 38
Research Gaps Addressed by the Proposed System 38
Theoretical Frameworks Informing the Proposed System 38
Systems Theory and the Integrated Operations Model 39
The Technology Acceptance Model (TAM) 40
The CRUD Operational Model 41
Direct Influence of the Literature on the Proposed System's Design 41
Validation of the Proposed System's Relevance and Necessity 42
REFERENCES 42
CHAPTER III 45
METHODOLOGY 45
3.1 Research Design 45
3.2 System Development Methodology 46
3.3 System Architecture 48
3.4 Tools and Technologies 50
3.4.1 Frontend Technologies 51
3.4.2 Backend Technologies 51
3.5 Data Gathering Techniques 54
3.6 Testing Procedures 55
3.7 Evaluation Criteria 56
3.8 Supporting Diagrams 57
3.9 Project Plan and Sprint Schedule 60
3.10 Requirements Specification Summary 62
3.11 Ethical and Data Privacy Considerations 63
3.12 Risk Management 64
3.13 Chapter Summary 65
REFERENCES 65

CHAPTER IV 67
RESULTS AND DISCUSSION 67
4.1 Introduction 67
4.2 Overview of the Developed System 67
4.3 System Development Methodology 70

4.4 System Architecture 74
4.5 Hardware and Software Specifications 76
4.5.2 Software Requirements 77
4.6 Database Design 78
Users Table 79
This table stores account credentials and user information. 79
Units Table 80
Stores condominium unit information and availability. 80
Booking Table 81
Stores resident booking and reservation details. 81
Payments Table 81
Stores payment and transaction details. 81
maintenance_requests Table 82
Stores resident maintenance requests and their status. 82
notifications Table 83
Stores notifications sent to users. 83
activity_logs Table 83
Stores records of user activities in the system. 83
4.7 Implementation of Functional Requirements 84
4.8 Role-Based Access Control 86
4.9 Business Rules and Data Validation 87
4.10 Security Implementation 88
4.11 Testing Framework and Test Cases 88
4.12 System Walkthrough of Principal Workflows 90
4.13 Relationship of the System to the Objectives of the Study 91
4.14 Discussion 92
4.15 Performance and Scalability Considerations 93
4.16 Chapter Summary 94

REFERENCES 94
CONCLUSION AND RECOMMENDATIONS 97
5.1 Introduction 97
5.2 Summary of Findings 98
5.3 Conclusion 99
5.4 Recommendations 100
5.5 Impact of the Study 102
5.6 Limitations Encountered 103
5.7 Lessons Learned 104
5.8 Recommendations in Detail by Stakeholder 104
5.9 Proposed Development Roadmap 105
5.10 Implications of the Study 106
5.11 Summary 107
REFERENCES 108

PHINMA UNIVERSITY OF ILOILO

College of Information Technology Education

Abstract

The integration of information technology into business operations has become a critical driver of organizational efficiency, data accuracy, and service quality, particularly in the property management and hospitality sectors, where escalating client expectations and transaction volumes make manual administration increasingly unsustainable. Gales Urban Haven, registered and operating as One Spatial Iloilo, provides condominium rental and property management services in Iloilo City, Philippines, and continues to rely on handwritten logs, verbal communication, and basic spreadsheet entries to manage its core operations. This reliance on manual, fragmented record-keeping has produced a predictable set of operational failures: data redundancy across disconnected files, misplaced physical reservation forms, delayed and error-prone financial reporting, inaccurate revenue computation caused by formula and version-control errors, undetected scheduling conflicts leading to double bookings, and informally tracked maintenance requests with no mechanism for assigning, monitoring, or verifying resolution. This paper presents the design, development, and evaluation of a centralized, web-based Condominium Booking and Operations Management System engineered specifically to address these deficiencies. The system integrates five core functional modules, namely automated booking management with real-time availability checking and conflict prevention, payment tracking with auditable transaction histories, revenue reporting with on-demand financial analytics, calendar-based scheduling synchronized across reservations and maintenance activities, and maintenance monitoring with task assignment and status tracking, all unified over a single normalized relational database with role-based access control to eliminate data silos and enforce accountability. The system was developed using the Agile Software Development Model to accommodate evolving stakeholder requirements across iterative sprint cycles, and was implemented with a full-stack JavaScript and TypeScript architecture consisting of React, Tailwind CSS, and Vite on the client tier, and Node.js, Express, and MySQL on the server and data tiers. The completed prototype was evaluated against five criteria, functionality, usability, accuracy, security, and efficiency, through structured user acceptance testing, performance testing under concurrent load, and feedback from administrators, business owners, and other authorized end users at Gales Urban Haven. Results indicate that the system substantially reduces manual administrative workload, eliminates double-booking incidents through automated conflict detection, improves the accuracy and timeliness of financial records, and gives management real-time, centralized visibility into occupancy, revenue, and maintenance activity to support faster, evidence-based operational decisions.

Index Terms*—booking management system, calendar scheduling, condominium rental, digital transformation, maintenance tracking, payment tracking, property management, web-based system*

PHINMA UNIVERSITY OF ILOILO

College of Information Technology Education

CHAPTER I

INTRODUCTION

1.1 Background of the Study

The integration of information technology into business operations has become a critical factor in improving organizational efficiency, data accuracy, and service quality across industries [1]. As digital transformation has accelerated in every sector of the global economy, businesses have increasingly adopted computerized systems to streamline daily operations, minimize human error, and establish more reliable data management infrastructures [2]. Organizations that fail to embrace technological advancement struggle to compete with digitally equipped counterparts, particularly as client expectations rise and regulatory requirements for accurate record-keeping intensify [3].

In the real estate and property management industry, digital transformation plays an especially vital role. Managing bookings, processing financial transactions, maintaining tenant records, and scheduling property maintenance are inherently complex activities that become unmanageable at scale without automated systems [4]. Property managers overseeing multiple units across different buildings or locations face compounding administrative burdens when relying on manual processes. These burdens result in delayed responses to tenant concerns, inaccurate financial reporting, and missed business opportunities. The adoption of integrated property management platforms has therefore shifted from a competitive advantage to an operational necessity [5].

This shift is further reinforced by the changing expectations of tenants themselves, who increasingly view responsive, transparent, and error-free administrative service as a baseline expectation rather than an added benefit. A tenant who experiences a billing discrepancy, a delayed maintenance response, or a mishandled reservation is more likely, in a competitive rental market, to view the incident as a sign of poor management rather than an isolated administrative lapse, regardless of whether the underlying cause was a genuine system failure or simply the limitations of manual record-keeping. Property management businesses that continue to rely on paper-based or spreadsheet-driven processes therefore face a growing reputational risk in addition to the operational inefficiencies already well documented in the literature, as service failures that stem from administrative disorganization are increasingly difficult to distinguish, from the tenant's perspective, from failures of genuine business competence.

The growing demand for condominium rentals in urban areas amplifies the need for efficient property management solutions. This demand is driven by rapid urbanization, increased migration of students and young professionals to

city centers, and shifting preferences toward high-density, amenity-rich residential environments [6]. Condominium rentals offer distinct advantages over traditional house rentals, including enhanced security features, proximity to commercial and educational institutions, shared facility access, and predictable utility arrangements. These qualities make condominium units the preferred housing choice for a growing demographic of urban residents. In turn, property management businesses face greater pressure to maintain seamless operations and high standards of tenant service.

This trend is particularly evident in secondary urban centers such as Iloilo City, where the expansion of commercial districts, the growth of business process outsourcing employment, and the steady enrollment of students from surrounding provinces into local universities have collectively sustained demand for condominium-style rental housing within close proximity to workplaces and schools. Property management businesses operating in this environment must therefore contend not only with a growing volume of transactions, but with a tenant base whose composition, needs, and length of stay vary considerably, from short-term student leases to longer-term professional tenancies, each carrying distinct scheduling, billing, and communication requirements. Without a system capable of accommodating this variability in a structured and consistent manner, administrative staff are left to manage these differences through ad hoc judgment calls, which increases the likelihood of inconsistent treatment between tenants and further compounds the administrative burden already imposed by manual record-keeping.

Managing multiple rental units simultaneously nonetheless introduces considerable administrative complexity. Staff must track tenant inquiries across different communication channels, maintain accurate booking schedules to prevent conflicts, monitor payment statuses for tenants with varying payment cycles, and respond promptly to maintenance requests. These tasks require systems capable of processing, organizing, and retrieving large volumes of data in real time. Without such systems, administrative staff must devote disproportionate amounts of time to clerical tasks. This reduces their capacity for higher-value activities that directly improve tenant satisfaction and business performance [7].

The cumulative effect of these pressures is that administrative capacity, rather than tenant demand or unit availability, often becomes the binding constraint on a small property management business's ability to grow. A business that could otherwise expand its portfolio or improve its service offerings is instead constrained by the finite ability of a small administrative staff to manually track an increasing volume of bookings, payments, and maintenance requests without introducing errors. It is this specific constraint, rather than a general aspiration toward modernization for its own sake, that motivates the development of the system presented in this study, and that is examined in detail in the sections that follow.

1.2 Statement of the Problem

Despite its growing clientele and expanding portfolio of rental units, Gales Urban Haven Unit relies predominantly on manual processes for its core operational functions. This reliance has persisted even as the volume and

complexity of daily transactions have grown, leaving the business increasingly exposed to the kinds of errors, delays, and inconsistencies that are well documented in the literature on unautomated property management operations. The specific manifestations of this problem, as observed through direct consultation with the business's administrators and staff, are detailed below.

1. Booking management. Booking records are kept in handwritten logs or basic spreadsheet entries, with no automated mechanism to verify unit availability or flag scheduling conflicts. Because these records exist independently of one another, often maintained by different staff members at different times, there is no single authoritative source that an administrator can consult to confirm, with confidence, whether a given unit is actually available on a requested date. Verifying availability instead requires manually cross-referencing multiple logs or contacting other staff members directly, a process that is both time-consuming and prone to human oversight. This has, on multiple occasions, resulted in two reservations being accepted for the same unit over overlapping periods, a conflict that is typically discovered only when both tenants attempt to claim the unit, at which point resolving the dispute consumes considerable administrative time and risks damaging the business's relationship with the affected parties.

2. Payment tracking. Staff manually reconcile tenant payment receipts against ledger records, a process that becomes increasingly error-prone as transaction volume grows. Each payment received must be manually transcribed from a physical or digital receipt into a separate ledger, and any delay or interruption in this transcription process creates a window during which the ledger no longer accurately reflects the tenant's true outstanding balance. As the number of active tenants and payment cycles has grown, so too has the volume of transactions requiring this manual reconciliation, increasing both the time burden on staff and the statistical likelihood that a given transaction is recorded incorrectly, recorded twice, or not recorded at all. Discrepancies uncovered during periodic reconciliation are often difficult to trace back to their origin, since the manual records offer no timestamped audit trail indicating who entered a given figure or when.

3. Revenue computation. Spreadsheet-based calculations are vulnerable to formula errors, data entry inconsistencies, and version control issues when multiple users access shared files. Because revenue figures are derived from formulas embedded within shared spreadsheet files, an accidental edit to a single cell, an inconsistently formatted entry, or the unintentional overwriting of a formula can silently distort revenue totals without triggering any visible warning to the staff member responsible. This risk is compounded when multiple staff members access and edit the same file at different times, since there is no mechanism to track which version of the file reflects the most current and accurate figures, or to reverse an erroneous change once it has been saved over the previous version. As a result, revenue reports produced from these spreadsheets cannot be fully trusted without a separate, time-consuming manual verification process, undermining their usefulness as a basis for timely business decisions.

4. Maintenance monitoring. Maintenance activities are recorded informally through verbal communication, handwritten notes, or disconnected digital files. There is no centralized system to assign tasks, monitor progress, or verify resolution. A maintenance concern raised by a tenant may be communicated directly to a

staff member in passing, written on a loose note that is later misplaced, or mentioned during a phone call that is never formally documented, depending on which channel happens to be most convenient at the time. Because these reports are not consolidated into a single system, there is no reliable way for administrators to confirm whether a given issue has already been assigned to a maintenance worker, whether it remains pending, or whether it has already been resolved, which has on occasion resulted in the same issue being reported and addressed independently by different staff members, or conversely, in a reported issue being forgotten entirely until the tenant follows up a second time. This absence of a verifiable record also makes it difficult for the business to identify recurring maintenance problems across units or to demonstrate, when necessary, that a reported concern was addressed within a reasonable timeframe.

1.3 Objectives of the Study

1.3.1 General Objective

This objective is grounded in the operational deficiencies identified in Section 1.2, each of which traces back to the same underlying cause: the absence of a single, authoritative system capable of coordinating booking, financial, and maintenance data across the organization. Rather than developing isolated tools to address each deficiency independently, this study pursues an integrated architecture in which every module draws from and contributes to a common relational database, so that a transaction recorded in one functional area, such as a confirmed booking, is immediately and consistently reflected across every other area it affects, including unit availability, revenue projections, and the scheduling calendar. This design choice reflects the position, supported throughout the literature reviewed in Chapter II, that the operational value of a property management system derives not merely from automating individual tasks in isolation, but from eliminating the data silos that automation of isolated tasks alone would otherwise leave in place.

In pursuing this objective, the study is also guided by a set of practical constraints appropriate to its scope as an academic capstone project. The system is developed for deployment within a single-property operation of a scale consistent with Gales Urban Haven's current and near-term projected portfolio, using technologies that are well documented, actively maintained, and accessible without specialized infrastructure or recurring licensing costs, consistent with the delimitations outlined in Section 1.4.2. Within these constraints, the general objective is pursued through the nine specific objectives detailed in Section 1.3.2, each of which corresponds to a discrete functional module or evaluative criterion through which the attainment of the general objective is measured, verified, and ultimately demonstrated in Chapters III through V.

1.3.2 Specific Objectives

1. Analyze the existing booking and operational procedures. The analysis identifies inefficiencies, recurring challenges, workflow bottlenecks, and the system requirements needed for automation. It examines how reservations are recorded, payments tracked, maintenance requests managed, and scheduling conflicts handled. Data is gathered through direct consultations with the business owners and administrative staff and through a review of current documentation, record-keeping methods, and communication workflows. The volume and frequency of bookings, payments, and maintenance requests are also assessed to determine the scale of the data management requirements. The findings establish clear, measurable requirements so that each feature responds to a validated operational need, and they guide the prioritization of features during development.

2. Design a secure, accessible, and user-friendly web-based system. The design follows user-centered principles and draws on established interface patterns and usability standards for web applications. It defines user roles and role-based access controls, applies secure authentication and encryption, and uses a responsive layout that works on desktops, laptops, and tablets. The interface is structured to minimize the learning curve for staff. Data validation and error-handling mechanisms guide users toward complete and accurate entries. The design also prioritizes scalability and performance under high usage and large data volumes.

3. Develop an automated booking management module. The module allows administrators to record and process reservation requests, check real-time unit availability, and prevent scheduling conflicts and double bookings. It maintains an up-to-date repository of active, pending, and completed reservations, sends notifications and confirmation alerts, and produces booking reports and analytics for monitoring occupancy trends and forecasting demand. A centralized dashboard gives administrators an organized overview of reservation activity.

4. Implement a payment tracking module. The module records and monitors tenant down payments, outstanding balances, payment history, and cumulative revenue in a structured and auditable manner. It provides a real-time view of the payment status of every unit, reduces discrepancies caused by manual computation, and keeps a verifiable trail of transactions. Automated reminders and alerts support follow-up on pending and overdue payments.

5. Create a revenue reporting module. The module automatically generates financial summaries, itemized payment reports, and revenue analytics on demand and at scheduled intervals. Income data are presented in visual and tabular form so that management can assess financial performance across periods. Automating report generation reduces the time spent on manual reporting and the risk of computational errors.

6. Integrate a calendar-based scheduling system. A unified, interactive calendar lets administrators monitor reservations, check-ins and check-outs, unit availability windows, and scheduled maintenance. It is synchronized with the booking and maintenance modules, so status changes are reflected system-wide. Features include color-coded event indicators, category filters, daily, weekly, and monthly views, and automated alerts for upcoming events. The calendar keeps a timestamped record of scheduled and completed activities to support operational review and

planning. Administrators can add, modify, and remove entries with minimal effort, which removes the need for separate scheduling tools.

7. Develop a maintenance monitoring module. The module records, tracks, and manages repair requests, preventive maintenance schedules, and service completion updates. Administrators can log requests with detailed descriptions, assign tasks to personnel, set priority levels, and track progress from initiation to completion. Scheduled maintenance appears automatically in the operational calendar, and automated reminders flag upcoming preventive tasks and overdue requests. Records are timestamped and can be filtered by unit, maintenance type, priority level, and completion status. This supports property condition assessment, warranty tracking, pattern identification, and proactive maintenance planning, while giving personnel a transparent record of assignments and accountability.

8. Establish a centralized database management system. A single relational database securely stores booking records, payment transactions, tenant profiles, maintenance logs, and revenue reports. It is designed for data integrity, consistency, and security. It supports access controls, automated backups, validation constraints that prevent incomplete or duplicate records, and concurrent multi-user access. The database serves as the single source of truth for all system-generated reports and analytics. Its schema is scalable to accommodate growth in records, transactions, and user accounts.

9. Evaluate the functionality, usability, and efficiency of the developed system. Evaluation uses structured user acceptance testing with administrators, business owners, and other authorized users. It also uses system performance testing of response time, processing speed, and stability under varying loads, and structured feedback collection through questionnaires and rating scales. The evaluation also verifies the accuracy and consistency of system-generated reports and records. Issues identified are documented and addressed through targeted refinements, and the results are analyzed against the study's objectives to determine how well the system resolves the identified operational inefficiencies.

1.4 Scope and Limitations

1.4.1 Scope of the Study

The study covers the design, development, and evaluation of a web-based Condominium Booking and Operations Management System for Gales Urban Haven Unit. The system includes the following modules: booking management, with real-time availability checking and double-booking prevention; payment tracking, covering down payments, balances, and payment history; revenue reporting and analytics; calendar-based scheduling of reservations, check-ins and check-outs, and maintenance activities; maintenance monitoring; and a centralized relational database with role-based access control. The system is intended for use by administrators and property

managers, and it is evaluated through user acceptance testing, performance testing, and feedback from end users and stakeholders.

Each of these modules was scoped to address a specific operational deficiency documented in Section 1.2, rather than being included on the basis of general feature completeness. The booking management module, for instance, extends beyond simple reservation recording to include automated conflict detection, ensuring that the specific double-booking incidents reported under the current manual system cannot recur once the platform is in use. Similarly, the payment tracking module was scoped to capture the full lifecycle of a tenant's financial relationship with the business, from the initial down payment through subsequent installments to final balance settlement, so that administrators are no longer required to manually reconstruct a tenant's payment history from disparate receipts when a question or dispute arises.

The study's coverage extends across the full system development lifecycle rather than terminating at deployment. This includes the requirements-gathering activities conducted with the business's administrators and staff, detailed in Chapter III; the iterative design and development of each functional module under the Agile methodology; and a structured, multi-phase evaluation encompassing unit testing, system-level integration testing, and user acceptance testing conducted directly with Gales Urban Haven's own administrative personnel. This evaluation phase was treated as an integral part of the study's scope, rather than a closing formality, because it is the mechanism by which the system's success against the objectives outlined in Section 1.3 is empirically substantiated rather than merely asserted.

The scope is deliberately bounded to the operational needs of a single-property, small-to-medium condominium rental business. The system is designed around the specific workflows, transaction volumes, and staffing realities of Gales Urban Haven as they were observed and documented during the data-gathering phase of this study, rather than toward the generalized requirements of a multi-property enterprise or a large-scale hospitality chain. This boundary, together with the specific exclusions detailed in Section 1.4.2, defines the practical limits within which the system's design decisions, evaluation results, and conclusions in the chapters that follow should be understood and interpreted.

1.4.2 Delimitations of the Study

Certain features were deliberately excluded to keep the project focused, feasible, and appropriate for an academic study. These delimitations reflect constraints in time, resources, and technical infrastructure. They let the researchers concentrate on the core functionalities with the most immediate operational benefit to Gales Urban Haven Unit (One Spatial Iloilo). They also mark natural areas for future development. Each is described below with its rationale.

External payment gateway and banking integration. The system does not integrate with third-party payment gateways such as PayMaya, GCash, or PayPal, or with banking APIs. Such integrations require compliance with financial regulations, merchant account agreements, secure communication protocols, and ongoing API maintenance, all beyond the scope of an academic capstone project. The Payment Recording Module instead documents payments

that have already been received, which is consistent with the client's current practice. It provides a foundation on which future developers can build online payment processing.

Native mobile application development. The study does not include a native Android or iOS application. Mobile development is a distinct discipline with its own tools, frameworks, and platform requirements, and building it in parallel with the web system would exceed the available time and resources. The web interface is instead designed to be responsive where practicable, so it can be accessed from modern mobile browsers.

SMS and email notification services. The system does not send SMS alerts or email confirmations to tenants or administrators. These features require third-party messaging platforms such as Twilio or SendGrid, or local telecommunications services. They also involve subscription costs, access agreements, and data privacy considerations under applicable regulations. The system handles notification-related information internally and gives administrators the data they need to contact tenants through their preferred external channels. Automated messaging is reserved for a future version.

Multi-property and enterprise-level management. The system is designed for Gales Urban Haven as a single-property operation. It does not provide multi-property management, enterprise dashboards, or tools for administering multiple buildings or branches. Such systems typically need multi-tenant database architectures, hierarchical administrative structures, and extensive customization. The modular architecture, however, allows these features to be added in later versions.

Cloud hosting and advanced cybersecurity. The study does not cover cloud-based hosting configurations, content delivery networks, server load balancing, or cybersecurity measures beyond standard authentication and basic database security. These require specialized infrastructure knowledge and resources beyond the scope of an academic study. The security architecture includes hashed password storage, session management, and basic input validation to guard against common vulnerabilities. These measures were appropriate for the academic and prototype context in which the system was deployed and evaluated.

1.5 Significance of the Study

The proposed Condominium Booking and Operations Management System was expected to provide benefits to various stakeholders by improving business processes and operational management.

Business Owner. The study helped business owners improve operational efficiency through automated booking management, accurate payment monitoring, revenue tracking, and centralized record management. It also supported better decision-making through readily available reports and organized business data. Furthermore, business owners were able to monitor real-time updates on reservations and cancellations, allowing them to respond promptly to changes in demand. The system also provided detailed financial summaries that enabled owners to identify trends,

forecast future revenue, and allocate resources more effectively. In addition, business owners benefited from a reduced manual workload, as the automated processes minimized human errors and saved valuable time. The centralized platform also allowed owners to access and review all business records from a single location, which significantly improved overall business management and long-term planning.

Administrative Staff. The system simplified daily operational tasks such as booking management, payment recording, schedule monitoring, and maintenance tracking. This reduced manual workload, minimized errors, and improved productivity. Additionally, the system provided administrative staff with a more organized and efficient workflow, allowing them to focus on higher-priority tasks rather than repetitive manual processes. Staff members were also able to retrieve and update records quickly, which eliminated unnecessary delays and ensured that all information remained accurate and up to date. The system further assisted administrative staff in generating reports and summaries, reducing the time spent on paperwork and manual data compilation. Moreover, real-time notifications and alerts kept the staff well-informed about upcoming bookings, pending payments, and scheduled maintenance, ensuring that operational concerns were addressed in a timely manner. As a result, administrative staff experienced a more manageable and less stressful work environment, ultimately contributing to a higher level of service quality and overall operational success.

Tenants and Clients. Clients benefited from improved booking organization, reduced scheduling conflicts, and more efficient handling of reservations and maintenance concerns, resulting in better service quality and customer satisfaction. Furthermore, clients experienced a smoother and more convenient booking process, as the system allowed them to make and confirm reservations with greater ease and accuracy. They also received timely updates and notifications regarding their bookings, payments, and any changes to their reservations, ensuring that they remained well-informed. In addition, clients benefited from faster response times to maintenance concerns and service requests, as the system enabled staff to track and address issues more efficiently. The organized record management also ensured that client information and transaction history were stored securely and accurately, reducing the likelihood of disputes or miscommunication. Ultimately, the improved system fostered a more trustworthy and professional relationship between the business and its clients, leading to higher levels of loyalty and long-term customer satisfaction.

BSIT Students. This study served as a practical application of concepts in web development, database management, software engineering, and systems analysis and design. It also provided students with experience in solving real-world business problems using information technology. Moreover, BSIT students gained hands-on experience in developing and implementing a fully functional system, which strengthened their technical skills and deepened their understanding of the software development lifecycle. The study also exposed students to the challenges and complexities of building real-world applications, preparing them for future professional roles in the field of information technology. Additionally, students developed critical thinking and problem-solving abilities as they designed and refined system features to meet the specific needs of the business. The project also enhanced their ability to work collaboratively, manage project timelines, and communicate technical solutions effectively, which were valuable skills for their future careers. Furthermore, this study served as a significant addition to their academic

portfolio, demonstrating their capability to create practical and impactful technology solutions that addressed genuine business needs.

Future Researchers. The study served as a reference for future research related to property management systems, booking automation, revenue monitoring, and web-based business solutions. Researchers could use this study as a foundation for enhancing similar systems or exploring additional features and technologies. Furthermore, future researchers were able to build upon the findings and methodologies of this study to develop more advanced and comprehensive property management solutions. The documented processes, system design, and implementation strategies presented in this study served as a valuable guide for researchers who undertook similar projects in the future. Additionally, future researchers could use this study to identify the limitations and challenges encountered during development, which helped them formulate better approaches and more effective solutions. The study also opened opportunities for further exploration of emerging technologies such as artificial intelligence, machine learning, and cloud-based systems that could be integrated into property management platforms. Moreover, the data and insights gathered from this study contributed to the growing body of knowledge in the field of information systems and web-based business solutions, supporting the advancement of future academic and professional research.

Academic Institution. The institution benefited from this study as an additional academic resource that demonstrated the practical use of information systems in solving business operational challenges and promoting innovation among students. Furthermore, the study served as a testament to the institution's commitment to producing competent and skilled graduates who were capable of developing practical technology solutions for real-world business needs. The completed system and its documentation were made available as reference materials that future students and faculty members could utilize for academic purposes and further research. Additionally, the institution was able to showcase this study as an example of student achievement that reflected the quality and effectiveness of its information technology curriculum. The study also contributed to the institution's academic resources by providing a well-documented case of applied systems development, which supported instructors in teaching relevant concepts in web development, database management, and systems analysis. Moreover, the success of this study encouraged more students to engage in research and development projects, fostering a culture of innovation and academic excellence that strengthened the institution's reputation in the field of information technology.

1.6 Definition of Terms

The following terms are defined operationally, within the context of the Condominium Booking and Operations Management System developed for Gales Urban Haven (One Spatial Iloilo), to guide the reader's interpretation of this paper.

Automation. Conceptually, automation is the use of software or machinery to perform tasks that would otherwise require human effort [8]. Operationally, it is the use of rule-based software logic to run repetitive operational processes with minimal manual intervention. In CBOMS it covers booking processing, payment computation and recording, financial report generation, and maintenance scheduling. For example, the system computes balances and penalties from stored rules instead of relying on manual calculation. In this study, automation is the main means of reducing staff workload and human error, so that staff can spend their time on tasks that need judgment, such as guest relations and problem resolution.

Booking Management System. A reservation system records and controls the allocation of limited inventory, such as rooms or units, to customers over time [9]. Operationally, it is the CBOMS module that records, organizes, processes, and monitors reservation transactions. These include booking details, tenant information, unit availability, and reservation history. It replaces the business's manual booking process. It supports search and filtering by period, unit type, or tenant profile, and it links to the payment and tenant records. In this study, it is the entry point of the system's workflow, because every payment, calendar entry, and revenue figure originate from a booking record.

Calendar Integration. Calendar-based scheduling displays time-bound events and resource commitments in one shared view in order to prevent conflicts [9]. Operationally, it is the scheduling module that presents reservations, check-ins and check-outs, unit availability windows, and maintenance activities in a single interactive calendar. Users can filter events by date, unit, or event type. The calendar helps prevent double bookings, generates reminders for upcoming events, and gives staff a shared, current view of operations. In this study, it ties the booking and maintenance modules together by showing both on the same timeline, so a unit scheduled for repair cannot be booked for the same dates.

Cash Ledger. A ledger is a chronological record of financial transactions from which account balances and statements are derived [10]. Operationally, it is the financial record within CBOMS that documents all monetary transactions per tenant and unit. These include down payments, installment payments, penalty charges, adjustments, and outstanding balances. It posts each transaction as it occurs, computes running balances, flags overdue accounts, and generates account statements on demand. In this study, it is the primary data source for Revenue Monitoring and Payment Tracking, so a correct ledger entry supports every financial report the system produces.

Centralized Database. A database is an organized collection of related data managed by a database management system, and centralizing it gives all applications one consistent source of data [11]. Operationally, it is the single repository that stores booking records, tenant profiles, payment transactions, financial summaries, and maintenance logs. All modules read from and write to it, which eliminates data silos and redundancy. It applies access controls and serves as the long-term archive for reference, auditing, and analysis. In this study, it replaces scattered notebooks, spreadsheets, and chat histories, which is what lets the other modules stay consistent with one another.

Condominium Booking and Operations Management System. An information system is an integrated set of components for collecting, storing, and processing data to support operations and decision-making [7]. Operationally, CBOMS is the web-based system developed in this study. It integrates booking management, payment monitoring, revenue tracking, calendar-based scheduling, and maintenance management in one platform

designed for the operational needs of Gales Urban Haven. It gives owners and staff real-time access to operational data without cross-referencing multiple records. In this study, it is the proposed solution whose effect on operational efficiency and data accuracy is evaluated.

Data Accuracy. Accuracy is a core dimension of data quality, meaning that recorded values correctly represent the real-world facts they describe [12]. Operationally, it is the degree to which recorded booking details, payment figures, tenant records, and maintenance logs are correct, consistent, and free of errors. The system supports it through input validation, automated computation, structured data entry, duplicate detection, and audit trails of changes to critical records. In this study, it is one of the two main outcomes assessed, together with Efficiency. It can be compared before and after adoption by looking at the number of recording and computation errors.

Efficiency. Efficiency is the degree to which a system uses time and resources in relation to the results it achieves [13]. Operationally, it is the improvement in the speed, accuracy, and reliability of Gales Urban Haven's operations that is attributed to the proposed system, with minimal wasted resources. Typical indicators are the time needed to process a booking, the time needed to prepare a financial report, and the number of manual steps per transaction. In this study, it is the second main outcome the system is expected to improve.

Maintenance Tracking. Maintenance management involves planning, scheduling, and recording corrective and preventive work to keep facilities in working order [14]. Operationally, it is the module for logging, scheduling, monitoring, and updating maintenance requests, repairs, preventive maintenance schedules, and service completion status. Administrators can assign tasks, set priority levels, and keep a complete maintenance history. The module also produces reports on recurring issues and pending requests. In this study, it keeps units in rentable condition and, through the Calendar Integration, prevents maintenance work from conflicting with bookings.

Payment Tracking. Payment tracking is the systematic monitoring of amounts due, received, and outstanding against customer accounts [10]. Operationally, it is the module that monitors and records tenant payments, including down payments, installments, outstanding balances, and payment histories. It flags overdue accounts, computes penalty fees, and produces payment status reports and tenant-specific summaries. It links to the booking and tenant records so that financial and operational data remain consistent. In this study, it gives the business a clear and current picture of who has paid, who has not, and how much is owed.

Prototype. A prototype is an early, partial version of a system built to explore requirements and evaluate design decisions before full development [15]. Operationally, it is the initial working version of CBOMS, presented to end users and evaluators for testing, demonstration, and feedback before full deployment. It allows design flaws, functional gaps, and usability issues to be found early, when they are cheaper to correct. In this study, feedback gathered from the prototype is the basis for revising the system before the final version is completed.

Resident Directory. A directory is a structured registry of entities and their attributes that other system components query to identify them [11]. Operationally, it is the registry within CBOMS of all registered residents and active tenants of Gales Urban Haven. Each profile holds biographical and contact information, unit assignment, tenancy classification, move-in and move-out dates, verification status, and linked financial accounts. The Cash Ledger,

Broadcast Console, and System Audit Trail rely on it to attribute records to the correct person and unit. In this study, it is the reference point that keeps tenant identity consistent across modules.

Revenue Monitoring. Revenue monitoring is the continuous measurement of income against expected earnings, used to detect variances and support managerial decisions [10]. Operationally, it is the automated process of recording, aggregating, and computing income from all booking transactions. These include payments received, down payments collected, and balances pending. It helps prevent income leakage and undetected discrepancies, and it supports revenue reports and trend analysis. In this study, it gives the owner timely, summarized financial information drawn directly from the Cash Ledger, without manual consolidation.

System Audit Trail. An audit trail is a chronological record of system activities that is sufficient to reconstruct and examine the sequence of events surrounding an operation [16]. Operationally, it is the chronological electronic record of user actions across all modules. These include logins and logouts, booking changes, ledger postings, broadcast messages, and directory updates. Each entry records the user, the action, the affected data fields, and the timestamp. Access is restricted to users with audit or administrative privileges. In this study, it supports accountability, discrepancy resolution, and reconstruction of the history of any transaction.

User Authentication. Authentication is the process of verifying a claimed identity before access is granted [17], and role-based access control limits each authenticated user to the permissions associated with their role [18]. Operationally, it is the access control mechanism that verifies a user's identity through valid credentials before granting access to system features and data. Each user sees only the features relevant to their responsibilities, and the system keeps an access log of successful and failed login attempts. In this study, it protects sensitive tenant and financial data and makes every audit trail entry traceable to a verified user.

Web-Based System. A web-based system is an application hosted on a server and accessed by clients through a standard web browser over a network [7]. Operationally, it is the software application hosted on a web server and accessed through a browser without installing dedicated software on the user's device. It allows multiple authorized users to work simultaneously from different locations, keeps data centralized and synchronized, and simplifies updates and maintenance. In this study, it lets the owner and staff use CBOMS from any device with a browser and an internet connection.

1.7 Conceptual Framework

The conceptual framework of this study follows the Input-Process-Output (IPO) model, a widely used representation of information systems in which resources entering a system are transformed through defined processes into outputs that serve the needs of the organization [7]. The IPO model was selected because it makes explicit how raw operational data, such as a reservation request or a recorded payment, are converted into the information that

administrators need in order to act, and because it allows each component of the proposed system to be traced back to a specific operational deficiency documented in Section 1.2.

Input

The inputs to the system consist of the data and requirements that originate from the day-to-day operations of Gales Urban Haven, operating as One Spatial Iloilo. These include tenant and guest information, unit details such as type, floor level, and monthly rate, requested check-in and check-out dates, down payments and installment payments, maintenance concerns reported by residents or observed by staff, and the user credentials and role designations of administrators, property managers, and residents. In addition to operational data, the inputs include the requirements gathered through consultations with the business owners and administrative staff, a review of existing record-keeping practices, and the standards and literature reviewed in Chapter II.

Process

The process component comprises the activities through which the inputs are validated, stored, and transformed. The system development itself follows the Agile Software Development Model, in which requirements analysis, design, development, testing, and deployment are carried out in iterative cycles [15]. Within the running system, each submission passes through a consistent request-validate-respond sequence: user credentials are authenticated and the user's role is checked [18]; the submitted data are validated against business rules, such as the rule that a unit cannot be assigned to two overlapping reservations; the validated records are written to a single normalized relational database [11]; and dependent records, such as the calendar, payment balances, and dashboard figures, are updated from that same source. The five functional modules, namely booking management, payment tracking, revenue reporting, calendar-based scheduling, and maintenance monitoring, are the concrete expression of this process layer.

Output

The outputs of the system are the products that management and staff use directly. These include confirmed and conflict-free reservations, real-time unit availability, auditable payment histories and outstanding balances, on-demand revenue summaries and occupancy analytics, a unified calendar of reservations and maintenance activities, and a trackable record of every maintenance request from the moment it is logged until it is resolved. The ultimate intended outcomes are a reduction in manual administrative workload, the elimination of double bookings, improved accuracy and timeliness of financial records, and faster, evidence-based operational decision-making.

Feedback

A feedback loop connects the output back to the input. The evaluation results obtained through user acceptance testing, performance testing, and stakeholder feedback are used to refine the system and, where necessary, to revise the requirements themselves, consistent with the iterative nature of the Agile model. Defects, usability concerns, and requests for additional functionality identified by users are therefore treated as inputs to the next development cycle rather than as the end of the study. Figure 1.1 summarizes the framework.

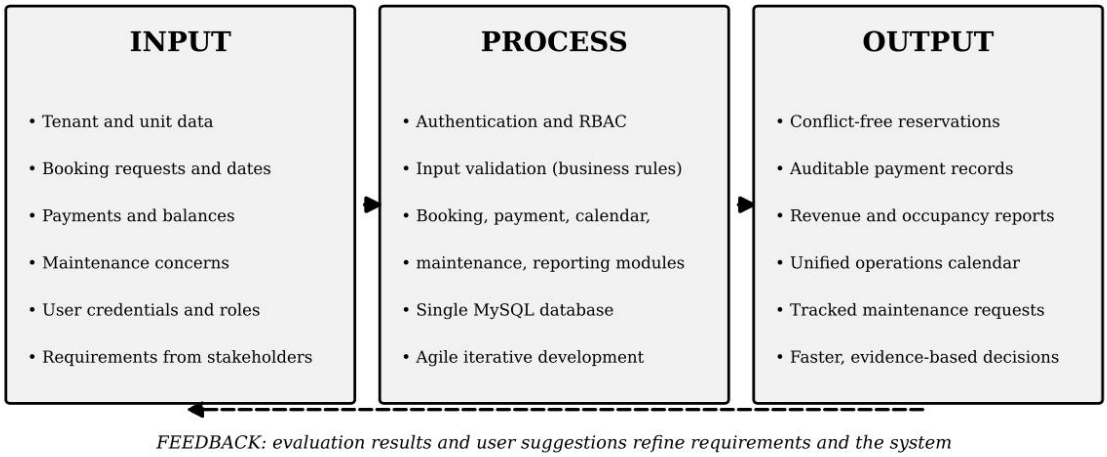


Fig. 1.1. Conceptual framework of the study based on the Input-Process-Output model.

Taken together, the framework expresses the central premise of this study: that a property management business operating on fragmented manual records can be moved to a dependable, centralized operation by placing a single validated database at the center of every process, and by giving each category of user only the view of that database that is relevant to their responsibilities.

1.8 Organization of the Study

This capstone paper is organized into five chapters, each of which builds on the preceding one. The arrangement follows the conventional structure of a developmental research study in information technology [15].

Chapter I, Introduction, presents the background of the study, the statement of the problem, the general and specific objectives, the scope and delimitations, the significance of the study to its beneficiaries, the conceptual framework, and the operational definitions of the technical terms used throughout the paper. It establishes the operational context of Gales Urban Haven and the deficiencies that the proposed system is intended to resolve.

Chapter II, Review of Related Literature, surveys foreign and local studies and the related literature on property management systems, automated booking and reservation management, payment tracking and financial record management, revenue reporting, maintenance management, calendar-based scheduling, centralized database systems, web-based information systems, and the Agile development methodology. The chapter ends with a synthesis of the reviewed literature, the research gaps addressed by the proposed system, and the theoretical frameworks that inform its design.

Chapter III, Methodology, describes the research design, the system development methodology, the system architecture, the tools and technologies used, the data gathering techniques, the testing procedures, and the evaluation criteria. It also documents the ethical and data privacy considerations observed in the study, the project plan, and the risks identified and managed during development, and it presents the supporting diagrams that define the architecture and data model of the system.

Chapter IV, Results and Discussion, presents the developed system, including its modules, development lifecycle, architecture, hardware and software requirements, and database design. It further describes the implementation of the functional and non-functional requirements, the business rules enforced by the system, the security mechanisms, the testing framework and test cases, and a walkthrough of the principal user workflows, and it relates the developed system back to the objectives of the study.

Chapter V, Conclusion and Recommendations, summarizes the findings of the study, states the conclusions drawn from them, acknowledges the limitations encountered, and sets out recommendations for the business, for system maintenance, and for future researchers. Each chapter closes with its own list of references prepared in the IEEE citation format, so that the sources of every chapter can be verified independently.

1.9 Chapter Summary

This chapter introduced the study by describing the operational context of Gales Urban Haven, operating as One Spatial Iloilo, and the deficiencies of its manual and spreadsheet-based processes: data redundancy, misplaced reservation forms, delayed and error-prone financial reporting, double bookings, and untracked maintenance requests. It stated the general objective of developing an integrated web-based Condominium Booking and Operations Management System and the nine specific objectives through which that objective is pursued, defined the scope and delimitations of the study, identified the persons and groups who benefit from it, presented the Input-Process-Output conceptual framework, and defined the technical terms used throughout the paper. The next chapter reviews the foreign and local studies and the related literature that support the design of the proposed system.

REFERENCES

[1] E. Brynjolfsson and L. M. Hitt, “Beyond computation: Information technology, organizational transformation and business performance,” J. Econ. Perspect., vol. 14, no. 4, pp. 23–48, Fall 2000, doi: 10.1257/jep.14.4.23.

[2] G. Vial, “Understanding digital transformation: A review and a research agenda,” J. Strateg. Inf. Syst., vol. 28, no. 2, pp. 118–144, 2019, doi: 10.1016/j.jsis.2019.01.003.

[3] P. C. Verhoef et al., “Digital transformation: A multidisciplinary reflection and research agenda,” J. Bus. Res., vol. 122, pp. 889–901, 2021, doi: 10.1016/j.jbusres.2019.09.022.

[4] N. Siniak, T. Kauko, S. Shavrov, and N. Marina, “The impact of proptech on real estate industry growth,” IOP Conf. Ser.: Mater. Sci. Eng., vol. 869, no. 6, Art. no. 062041, 2020, doi: 10.1088/1757-899X/869/6/062041.

[5] F. Braesemann and A. Baum, “PropTech: Turning real estate into a data-driven market?,” Oxford Future of Real Estate Initiative, Saïd Business School, Univ. Oxford, Oxford, U.K., Work. Paper v1.3, Apr. 2020.

[6] World Bank, Philippines Urbanization Review: Fostering Competitive, Sustainable and Inclusive Cities. Washington, DC, USA: World Bank, 2017.

[7] K. C. Laudon and J. P. Laudon, Management Information Systems: Managing the Digital Firm, 16th ed. Hoboken, NJ, USA: Pearson Education, 2020.

[8] M. P. Groover, Automation, Production Systems, and Computer-Integrated Manufacturing, 4th ed. Boston, MA, USA: Pearson, 2015.

[9] M. L. Kasavana and R. M. Brooks, Managing Front Office Operations, 8th ed. Lansing, MI, USA: Educ. Inst. Amer. Hotel & Lodging Assoc., 2009.

[10] C. S. Warren and J. P. Jones, Corporate Financial Accounting, 11th ed. Boston, MA, USA: Cengage Learning, 2010.

[11] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed. Hoboken, NJ, USA: Pearson Education, 2016.

[12] R. Y. Wang and D. M. Strong, “Beyond accuracy: What data quality means to data consumers,” J. Manage. Inf. Syst., vol. 12, no. 4, pp. 5–33, 1996, doi: 10.1080/07421222.1996.11518099.

[13] N. Gaither and G. Frazier, Operations Management, 9th ed. Cincinnati, OH, USA: South-Western College Publishing, 2002.

[14] B. S. Dhillon, Maintainability, Maintenance, and Reliability for Engineers. Boca Raton, FL, USA: CRC Press, 2006.

[15] I. Sommerville, Software Engineering, 10th ed. Harlow, U.K.: Pearson Education, 2016.

[16] National Institute of Standards and Technology, An Introduction to Computer Security: The NIST Handbook, NIST Special Publication 800-12, Gaithersburg, MD, USA, 1995.

[17] W. Stallings, Cryptography and Network Security: Principles and Practice, 8th ed. Boston, MA, USA: Pearson, 2020.

[18] D. F. Ferraiolo and D. R. Kuhn, “Role-based access controls,” in Proc. 15th Nat. Comput. Security Conf., Baltimore, MD, USA, 1992, pp. 554–563.

CHAPTER II

REVIEW OF RELATED LITERATURE

2.1 Foreign Studies

Imkumnert [1] proposed a system that handles most of the information management in a condominium, such as customer, room, and service information, room renting, billing and receipt operations, and reports for the management team. Rental condominiums are becoming popular in the lodging market. The real-estate business had previously focused on the condominium market through a construction and selling methodology.

Halkiopoulos et al. [2] explored knowledge of online booking systems and the views of local student-users on booking rates through these systems. They also investigated the emotion-focused decision-making process that users follow when choosing a tourist destination through online booking systems.

Varutbangkul [3], at Assumption University, implemented a condominium management information system to improve the efficiency of the existing system at Fair Tower, a condominium belonging to GOS Limited Partnership. The project designed a new computerized system to replace the old manual system at the front office, so that front-desk staff could work more efficiently and accurately. The proposed system handles most of the information management in the condominium, including customer, room, and service information, room sales, room renting, bills and receipts, and reports for the general manager.

Kuo and Chou [4] studied condominium management (CM) businesses, which rely on continuous improvement of service quality (SQ) to increase consumer satisfaction (CS) and thereby improve operations. A lack of knowledge about SQ and CS results in poor efficiency, so the relationship between the two must be investigated to develop cost-effective improvement strategies. They first used exploratory factor analysis to cluster similar attributes into three constructs and identify significant attributes for assessment. Resident satisfaction scores (RSSs) were then computed to assess the relative importance rate (RIR) of the attributes, using an adapted multivariate method that regresses the RSSs on SQ attribute performance. Finally, a service-quality model integrating an improved Kano's approach with importance-performance analysis (IPA) was used to develop a prioritized strategy for improving satisfaction in the CM business.

2.2 Local Studies

Bajado [5] investigated the impact of hotel business tools and technologies on guest satisfaction in Tagaytay City, Philippines, a rapidly growing tourist destination. Hotels worldwide are adopting technology to enhance guest experiences and satisfaction.

Gosela and Encarnacion [6] presented a unified application that addresses existing challenges through user-centric features. Using location-based services and streamlined booking processes, the application aims to improve efficiency, accessibility, and sustainability in the tourism sector. It provides real-time tracking, efficient transit solutions, and a comprehensive directory of establishments.

Gallenero et al. [7] developed a real-estate management system with sales analysis and customer segmentation. Rising population increases the demand for real estate, and managing real estate manually takes considerable time and manpower. Manual handling is also prone to damage and misplacement of documents and client data. The system automates complex tasks and duties and also considers customer support and community. Customer segmentation was used for sales analysis to improve sales and to understand the company's current, possible, and future customers. The system was developed using the Agile methodology, with users holding different roles and administrator rights to access and manage it. Testing, verification, validation, usability testing, and acceptance testing, including alpha and beta testing, were conducted to ensure the requirements were met.

2.3 Related Literature

This chapter presents a comprehensive review of related literature that serves as the theoretical and empirical foundation for the development of the Condominium Booking and Operations Management System proposed in this study. The literature reviewed covers a broad range of thematic areas directly relevant to the proposed system: property management systems, automated booking and reservation technologies, payment tracking and revenue monitoring, maintenance management, centralized database systems, web-based application development, and the application of the Agile development methodology in information systems projects. The insights and findings drawn from the reviewed literature collectively inform the design decisions, functional specifications, and architectural choices that underpin the proposed system for Gales Urban Haven, operating under the business name One Spatial Iloilo.

Property Management Systems

Property management systems (PMS) have been the subject of extensive scholarly and professional inquiry over the past two decades, owing to the growing recognition that the efficient management of residential, commercial, and

mixed-use properties requires the support of robust, integrated digital tools. The literature broadly defines a property management system as a software platform that consolidates the core operational functions of a property management business into a single, centralized, and accessible interface. These functions include tenant record management, lease administration, financial tracking, maintenance scheduling, and reporting.

Bardi [8] affirmed that property management systems have fundamentally transformed the hospitality and real estate industries by replacing fragmented, paper-based administrative processes with automated, data-driven workflows. These workflows significantly reduce operational errors and improve service delivery standards. The author noted that integrating booking, billing, and reporting functions within a unified platform enables property managers to maintain a comprehensive and current view of all operational activities, thereby improving their capacity for informed decision-making and proactive problem resolution.

Tesone [9] similarly argued that modern property management systems represent an indispensable operational infrastructure for any property management enterprise operating in a competitive market environment. His research emphasized that the ability to retrieve accurate and up-to-date information across all operational domains, from reservation status to financial performance, at any time and from a single platform, is a decisive competitive advantage over organizations that continue to rely on manual methods.

In the Philippine context, the Department of Information and Communications Technology (DICT) [10] has acknowledged the growing importance of information and communications technology adoption among small and medium enterprises (SMEs) in the service sector. It noted that businesses in property management, hospitality, and rental services that adopt digital management systems report significantly higher levels of operational efficiency, financial accuracy, and customer satisfaction than their non-digitalized counterparts. This observation is particularly relevant to the present study, given that Gales Urban Haven is a small-scale condominium management operation whose administrative processes remain largely manual and are therefore vulnerable to the inefficiencies that digital systems are specifically designed to address.

Cobanoglu et al. [11] further highlighted that the effectiveness of a property management system is determined not solely by its technological sophistication but also by how well its design and functionality align with the specific operational workflows and data management needs of its target users. This finding underscores the importance of a thorough requirements analysis prior to system development and of involving end users in the design and evaluation process. Both are key activities in the Agile development approach adopted in the present study.

Automated Booking and Reservation Management

The automation of booking and reservation processes has been extensively studied in both the hospitality and real estate management literature. A consistent body of evidence shows that automated reservation systems substantially outperform manual methods in speed, accuracy, reliability, and scalability. The literature identifies scheduling conflicts, double bookings, and incomplete reservation records as the most common and consequential problems of manual booking systems, and affirms that automated booking management platforms effectively mitigate them.

O'Connor and Murphy [12] studied the adoption of online reservation systems in the hospitality sector. They found that properties using automated booking platforms reported a significant reduction in reservation errors, faster booking processing, and a marked improvement in the accuracy of unit availability information available to both administrators and clients. The authors concluded that real-time availability checking is among the most operationally impactful features of such systems, as it eliminates the information lag inherent in manual availability verification.

Siguaw and Enz [13] similarly documented that implementing computerized reservation systems in lodging establishments produced measurable improvements in front-desk efficiency, with staff able to process reservations in a fraction of the time previously required. Their findings further indicated that automated reservation systems gave managers easier access to historical booking data, facilitating trend analysis and demand forecasting that were previously impractical under manual record-keeping.

Kasavana and Brooks [14] noted that the fundamental advantage of an automated booking system lies in its capacity to maintain a continuously updated, centrally accessible, and conflict-free reservation schedule. The system generates an alert or blocks the entry if two reservations are recorded for the same unit at the same time. This conflict-prevention mechanism, a standard feature of well-designed booking management systems, directly addresses one of the most critical risks of manual reservation processes: the inadvertent recording of overlapping bookings that results in tenant disputes and administrative complications.

For small-scale condominium and apartment rental management, Pizam and Ellis [15] emphasized that the benefits of booking automation are proportionally more significant for smaller properties than for large hotel chains. Smaller operations typically have fewer administrative staff and therefore less capacity to absorb the time and error costs of manual booking management. This observation applies directly to Gales Urban Haven, where the administrative team is small and the demands of managing multiple condominium units simultaneously require a system that can handle reservation data with speed, accuracy, and minimal staff intervention.

Payment Tracking and Financial Record Management

The management of tenant payments, including the recording of initial deposits, monitoring of outstanding balances, and maintenance of complete payment histories, is one of the most critical and error-sensitive functions in any property rental operation. The literature on financial record management in property contexts consistently identifies the absence of a systematic, automated payment tracking mechanism as a primary source of financial discrepancies, revenue leakage, and administrative disputes between property managers and tenants.

Warren and Jones [16] studied financial management practices among small residential rental operations. They found that businesses relying on manual payment recording experienced significantly higher rates of recording errors, missed payment notifications, and balance computation discrepancies than those using automated financial tracking tools. The authors concluded that even relatively simple automated payment tracking systems substantially

improve financial accuracy and accountability, reduce the administrative burden of manual reconciliation, and improve the reliability of financial records available to management.

Clauretie and Sirmans [17] further argued that accurate and timely payment tracking is not merely an administrative convenience but a strategic financial management imperative for property rental businesses. The accuracy of payment records directly determines the reliability of cash flow projections, revenue reports, and business planning decisions. Their research showed that property managers who maintained comprehensive, up-to-date payment records were significantly better positioned to make informed decisions on pricing, capacity expansion, and operational investment than those whose financial records were incomplete or delayed.

Turner [18] examined the integration of payment tracking with broader financial management systems in property management operations. He found that consolidating payment data, revenue figures, and financial reporting within a single platform eliminated the data reconciliation delays that commonly arise when these functions are managed through separate, disconnected tools. This finding is directly relevant to the proposed system for Gales Urban Haven, in which the payment tracking module will be fully integrated with the revenue reporting module so that all financial data is processed, stored, and reported within a single, cohesive system environment.

Revenue Monitoring and Financial Reporting

Revenue monitoring and financial reporting are foundational functions of any business operation, and their importance in property management contexts has received considerable scholarly attention. The literature consistently emphasizes that accurate, timely, and accessible financial reporting is essential for enabling property management organizations to assess their financial performance, identify revenue trends, manage cash flow effectively, and make evidence-based strategic decisions.

Brigham and Houston [19] described revenue monitoring as the systematic and continuous process of tracking all income streams, computing aggregate revenue across defined time periods, and comparing actual revenue performance against projected targets. Their work emphasized that automated revenue monitoring systems, which perform these functions without manual data compilation, give management access to accurate financial performance data in real time. This dramatically reduces the lag between financial activity and financial awareness that characterizes manual reporting.

Horngren et al. [20] further noted that the quality of management decision-making is directly and substantially influenced by the timeliness and accuracy of the financial information available to decision-makers. Their research showed that organizations with real-time financial reporting tools consistently outperformed those relying on periodic manual reports in identifying and responding to financial challenges before they escalate into serious operational problems. This finding strongly supports the inclusion of an automated revenue reporting module in the proposed system for Gales Urban Haven, where the current absence of such a tool has been identified as a significant impediment to effective financial management.

Kimmel et al. [21] similarly highlighted the importance of financial report accuracy in small business contexts. They noted that errors in manually prepared financial reports can have disproportionately serious consequences for small enterprises, given their typically limited financial reserves and narrower margins for error. The authors advocated the adoption of automated financial reporting tools by small business operators as a critical risk management strategy. This recommendation directly reinforces the rationale for incorporating revenue reporting automation into the proposed system for One Spatial Iloilo.

Maintenance Management in Property Operations

Maintenance management is widely recognized in the property management literature as a critical determinant of property value retention, tenant satisfaction, and operational continuity. The systematic recording, scheduling, monitoring, and resolution of maintenance activities is essential for keeping condominium units and shared facilities in optimal condition throughout their operational lifecycle.

Dhillon [22] provided a comprehensive framework for maintenance management in built environments, arguing that the effectiveness of a maintenance program is directly determined by the quality of the systems and processes used to track and manage maintenance activities. His research showed that organizations using structured, digitally supported maintenance management systems consistently achieved higher rates of timely maintenance resolution, fewer repeated maintenance failures, and higher tenant satisfaction than those relying on informal or paper-based tracking.

Mobley [23] similarly emphasized the strategic importance of preventive maintenance scheduling in property management. He noted that a systematic approach to maintenance planning, supported by accurate historical maintenance records and a reliable scheduling system, enables property managers to address potential issues before they develop into costly emergency repairs. His work highlighted the role of digital maintenance logs in helping managers identify patterns of recurring problems and make data-driven decisions about maintenance resource allocation and preventive intervention priorities.

Wireman [24] further documented that the absence of a systematic maintenance tracking system in property management operations frequently leads to the loss of maintenance history records. This makes it difficult for managers to assess the long-term performance of maintenance interventions, track the lifecycle of property assets, and demonstrate maintenance accountability to property owners. These findings provide strong empirical support for a dedicated maintenance monitoring module within the proposed system for Gales Urban Haven, ensuring that all maintenance activities are systematically recorded, tracked, and accessible to administrators for review at any time.

Calendar-Based Scheduling in Operations Management

Calendar-based scheduling tools have been widely studied in operations management across multiple industries, including hospitality, healthcare, education, and property management. The literature consistently affirms that visual scheduling interfaces give administrators a significant operational advantage by consolidating time-sensitive

operational data into a single, chronologically organized display that enables rapid situational awareness and more effective operational planning.

Gaither and Frazier [25] noted that scheduling is among the most cognitively demanding and error-prone functions in operations management when performed manually. It requires administrators to simultaneously track and reconcile multiple overlapping activities, availability windows, and time constraints without the support of a visual organizational framework. Their research showed that digital scheduling tools reduce the cognitive load of scheduling tasks and substantially decrease the frequency of scheduling errors and conflicts in complex operational environments.

Pinedo [26] further documented that integrated scheduling systems, which synchronize calendar data with other operational modules such as reservations, availability tracking, and maintenance records, provide the most significant operational benefits compared to standalone scheduling tools. Synchronizing scheduling data with real-time operational information ensures that the calendar view presented to administrators always reflects the current state of operations, enabling scheduling decisions based on reliable, up-to-date information. This principle directly informed the design of the calendar scheduling module in the proposed system, which will be fully integrated with the booking and maintenance modules to ensure real-time data consistency across all scheduling displays.

Centralized Database Systems in Information Management

The design and implementation of centralized database systems has been a central area of inquiry in information systems research. The literature consistently affirms that centralized relational databases provide a superior data management foundation for multi-functional information systems compared to fragmented or distributed data storage approaches. Centralized databases enable organizations to eliminate data redundancy, enforce data consistency and integrity, and give all authorized users access to the same, current version of organizational data regardless of their location or device.

Date [27] provided a seminal treatment of relational database theory, arguing that the relational model, implemented through systems such as MySQL, provides the most robust and flexible framework for organizing complex, interrelated data sets in organizational contexts. His work emphasized that structured query language (SQL) enables powerful and precise data retrieval that would be impractical or impossible in non-relational environments, including the spreadsheet-based and paper-based systems commonly used by small property management operations.

Elmasri and Navathe [28] further documented the importance of database normalization in ensuring the integrity and consistency of data in relational databases. They noted that a properly normalized schema eliminates data redundancy, prevents update anomalies, and keeps all data within the system internally consistent. Their work also highlighted the importance of appropriate indexing strategies to optimize query performance and keep the system responsive as data volume grows. This is particularly relevant to the proposed system for Gales Urban Haven, which will accumulate increasingly large volumes of booking, payment, and maintenance records over its operational life.

Silberschatz et al. [29] addressed the security dimensions of centralized database management. They emphasized that robust access control mechanisms, parameterized query execution, and regular data backup protocols are essential components of any database system that handles sensitive organizational or financial data. Their recommendations directly inform the security specifications in the database design of the proposed system, which will implement role-based access controls, prepared statement query mechanisms, and scheduled backup procedures to protect all operational data stored for One Spatial Iloilo from unauthorized access, data corruption, and accidental loss.

Web-Based Information Systems

Web-based information systems have become the dominant architectural paradigm for organizational information management across virtually all industry sectors, owing to their platform independence, accessibility, scalability, and ease of maintenance. The literature consistently documents their advantages over locally installed desktop applications, particularly where multiple users need to access and interact with shared data simultaneously from different workstations or locations.

Laudon and Laudon [30] provided a comprehensive treatment of management information systems in organizational contexts, arguing that web-based systems are the most effective and adaptable information management infrastructure available to modern organizations. Their work highlighted that web-based systems let organizations deploy and update their information systems more efficiently than locally installed alternatives. Modifications need only be applied at the server level and are immediately available to all users without software updates on individual client devices.

O'Brien and Marakas [31] similarly emphasized that web-based systems offer a critical advantage in data consistency. All users access the same centrally hosted application and data repository, which eliminates the synchronization problems that arise when multiple users maintain separate local copies of organizational data. This advantage is particularly significant in property management operations, where multiple staff members may need to access and update shared booking, payment, and maintenance records simultaneously, and where inconsistencies from unsynchronized local copies can lead to serious operational errors.

Turban et al. [32] further noted that the accessibility of web-based systems through standard browsers, without specialized software installation, significantly lowers the barrier to adoption for small and medium enterprises. It eliminates the hardware and software procurement costs commonly associated with proprietary desktop deployments. This is particularly relevant to One Spatial Iloilo, where a web-based system will enable the organization to implement a comprehensive digital management platform without the substantial upfront costs of traditional enterprise software installations.

Agile Software Development Methodology

The Agile software development methodology has been extensively studied and widely adopted in the information systems development community since the publication of the Agile Manifesto in 2001. The Manifesto articulated a set of values and principles prioritizing iterative development, customer collaboration, responsiveness to change, and the continuous delivery of working software over the rigid adherence to comprehensive upfront planning that characterizes traditional development models.

Pressman and Maxim [33] provided a thorough examination of Agile development practices in their software engineering textbook. They documented that Agile methodologies consistently produce higher-quality software than traditional Waterfall-based approaches when applied to projects with evolving or initially unclear requirements. Their analysis showed that the iterative structure of Agile development enables teams to incorporate user feedback and stakeholder input at each stage, resulting in systems that more closely match the actual needs and expectations of their intended users at delivery.

Sommerville [34] similarly noted that Agile development is particularly well suited to projects in which the full scope of system requirements cannot be determined at the outset, as its iterative cycles allow requirements to be progressively clarified, refined, and expanded as development proceeds and users better understand the system's capabilities and their own operational needs. This makes Agile especially appropriate for the proposed system for Gales Urban Haven. The complexity of integrating multiple operational modules, and the need to align system functionality with the nuanced and evolving workflows of the property management team, make iterative development and continuous feedback essential to the success of the project.

Schwaber and Sutherland [35], the co-creators of the Scrum framework, one of the most widely used implementations of the Agile methodology, documented that organizations applying Agile practices in information systems development report higher rates of project completion within budget and schedule, higher end-user satisfaction with the delivered system, and greater capacity to adapt to changing requirements than organizations using traditional methodologies. These documented outcomes provide strong empirical support for selecting the Agile model as the guiding development methodology for the proposed Condominium Booking and Operations Management System.

2.4 Synthesis of the Reviewed Literature

The body of literature reviewed in the preceding sections of this chapter collectively established a comprehensive and well-grounded theoretical and empirical foundation for the development of the Condominium Booking and Operations Management System proposed in this study. Across the various domains examined, including property management systems, booking and reservation technologies, payment and revenue tracking, maintenance

management, database architecture, web-based system design, and the application of the Agile development methodology, a consistent and converging set of insights emerged. These insights affirmed that the operational challenges experienced by Gales Urban Haven under One Spatial Iloilo were not unique to the organization. They were, rather, characteristic of a broader class of problems that had been extensively documented in the academic and professional literature on small- to medium-scale property management operations that continued to rely on manual or fragmented record-keeping systems.

The synthesis of these reviewed works revealed that the transition from manual, paper-based, or loosely organized digital processes to a centralized, automated, and integrated information system was not merely a technological upgrade but a fundamental transformation in the way a property management business operated, managed its resources, served its tenants, and positioned itself for sustainable growth. The literature consistently demonstrated that organizations that had undertaken this transition realized measurable improvements in operational efficiency, financial accuracy, data accessibility, staff productivity, and overall service quality. The proposed system for Gales Urban Haven was firmly grounded in these documented outcomes, and its design, features, and development approach were shaped directly by the collective knowledge and evidence presented in the reviewed literature.

Convergence of Key Themes Across the Literature

A careful examination of the reviewed literature revealed that despite the diversity of contexts, geographic settings, and technological environments in which the individual studies were conducted, a set of dominant and recurrent themes emerged with striking consistency. These themes not only validated the problem statement articulated in Chapter One of this study but also provided direct and actionable guidance for the scope, design, and functionality of the proposed system. The following subsections discussed these converging themes in detail and explained how each one informed the development of the Condominium Booking and Operations Management System.

Furthermore, the convergence of these themes across multiple studies and literature sources strengthened the theoretical and empirical foundation of the proposed system, demonstrating that the identified operational challenges faced by Gales Urban Haven were not isolated concerns but widely recognized issues that had been consistently addressed through the adoption of web-based information systems. The recurring patterns observed across the reviewed literature also served as a guiding framework in determining the most essential features and functionalities that the proposed system needed to incorporate in order to respond effectively to the operational demands of a condominium rental business.

Additionally, the synthesis of these converging themes highlighted the critical role that automation, centralized data management, and real-time monitoring played in modernizing property management operations, further justifying the relevance and necessity of the proposed system. The alignment between the findings of the reviewed literature and the operational context of Gales Urban Haven also reinforced the appropriateness of the chosen system design and development approach, ensuring that the proposed solution was grounded in established knowledge and proven practices. Moreover, the identification of these key themes allowed the researchers to draw meaningful connections between existing studies and the specific objectives of this research, providing a coherent and well-supported

academic basis for the development and implementation of the Condominium Booking and Operations Management System.

Ultimately, the convergence of key themes across the literature affirmed that the proposed system represented a timely, relevant, and well-informed response to the operational inefficiencies of Gales Urban Haven. It contributes not only to the improvement of its business processes but also to the broader academic discourse on the application of information systems in property management and hospitality operations.

The Pervasive Inadequacy of Manual Property Management Processes

Perhaps the most consistently documented finding across the reviewed literature was the fundamental inadequacy of manual, paper-based, and non-integrated approaches to property management. Studies examining condominium associations, residential rental properties, commercial real estate portfolios, and mixed-use developments all converged on the conclusion that organizations relying on such approaches routinely suffered from a predictable constellation of operational failures. These included delayed and error-prone tenant billing, inconsistent maintenance request tracking, inability to generate accurate real-time financial reports, poor visibility into unit occupancy and availability, and fragmented communication between property managers, maintenance staff, and tenants.

The literature made clear that these failures were not isolated incidents but the natural and expected consequences of systems designed for lower volumes of transactions and simpler operational environments. As the scale and complexity of property portfolios grew, in terms of the number of units managed, the diversity of tenant needs, the variety of services offered, and the stringency of regulatory compliance requirements, the limitations of manual systems became increasingly acute. For Gales Urban Haven, which operated in a growing urban condominium market in Iloilo City, these dynamics were particularly relevant and underscored the urgency and necessity of the proposed system.

The Demonstrated Value of Centralized and Integrated Information Systems

In direct response to the documented inadequacies of manual approaches, the literature overwhelmingly endorsed the adoption of centralized, integrated property management information systems as the most effective and sustainable solution. The reviewed studies documented concrete and quantifiable benefits realized by organizations that had implemented such systems, including reductions in administrative workload, improvements in billing accuracy and collection rates, faster resolution of maintenance issues, and enhanced tenant satisfaction scores.

Centralization was identified as a particularly critical success factor. When all operational data, including tenant records, unit availability, booking schedules, payment histories, maintenance requests, and financial summaries, resided in a single, authoritative system of record, the risks of data duplication, inconsistency, and loss were dramatically reduced. Property managers were able to access up-to-date information at any time, from any location, enabling more responsive and informed decision-making. The proposed Condominium Booking and Operations Management System embodied this principle of centralization as one of its foundational design objectives.

The Strategic Importance of Booking and Reservation Management

The literature reviewed on booking and reservation systems established that the management of unit availability and the processing of booking transactions represented some of the most operationally critical and error-prone functions in property management. Scheduling conflicts, double bookings, and the failure to maintain accurate availability calendars were identified as frequent sources of tenant dissatisfaction, revenue loss, and reputational damage for property management organizations.

The reviewed studies demonstrated that automated booking systems equipped with real-time availability tracking, conflict detection, and instant confirmation capabilities effectively eliminated or significantly reduced these problems. The literature further established that integrating booking management with tenant records and payment processing created a seamless operational workflow that reduced the manual effort required to complete a booking transaction from end to end. These documented outcomes directly informed the booking management module that was developed as a core component of the proposed system for Gales Urban Haven.

The Critical Role of Accurate Financial and Payment Tracking

Across the financial management domain, the reviewed literature identified inaccurate and incomplete payment tracking as one of the most consequential operational deficiencies in property management organizations. The inability to accurately record, reconcile, and report on rent collection, association dues, security deposits, utility charges, and penalty fees created compounding problems that affected cash flow management, financial planning, and regulatory compliance. The literature documented that automated payment tracking systems, particularly those that generated digital receipts, sent payment reminders, flagged overdue accounts, and produced summary financial reports, consistently outperformed manual equivalents in collection rates, accuracy, and audit readiness. The proposed system incorporated a comprehensive payment and revenue tracking module informed by these findings, ensuring that all financial transactions conducted through Gales Urban Haven were recorded with precision, transparency, and full auditability.

As the system became fully operational, the payment tracking module served as a critical instrument in strengthening the financial governance and accountability practices of the organization, providing administrators with a reliable and centralized platform for monitoring all incoming and outgoing financial transactions in real time. The structured financial data generated by the system also enabled management to produce accurate periodic financial statements, facilitating more informed budgeting decisions and long-term financial planning that supported the sustainable growth of Gales Urban Haven. The payment tracking module also incorporated advanced analytics capabilities that allowed administrators to identify payment trends, forecast revenue streams, and detect anomalies or irregularities in financial records before they escalated into more serious concerns.

The system further supported the generation of tenant-specific payment histories, which served as valuable documentation in cases of financial disputes, lease renewals, or regulatory audits conducted by relevant government authorities. Integrating the payment tracking module with other system components, such as booking management

and tenant records, created a fully interconnected financial ecosystem that eliminated data redundancy and ensured consistency across all financial and operational records maintained by the organization. The adoption of a digitized and automated payment tracking approach also significantly reduced the administrative burden placed on staff, freeing time and resources that were redirected toward higher-value management activities and tenant relationship building. Ultimately, the financial and payment tracking capabilities embedded within the system laid a strong and enduring foundation for the continued improvement of fiscal responsibility, operational transparency, and financial performance at Gales Urban Haven.

Maintenance Management as a Driver of Tenant Retention

The reviewed literature on maintenance management revealed a strong and statistically significant relationship between the responsiveness of a property's maintenance operations and the satisfaction and retention rates of its tenants. Properties that implemented structured, technology-supported maintenance request workflows, including formal request submission channels, automated routing to appropriate staff, status tracking, and completion confirmation, were shown to resolve issues faster, reduce the number of repeat complaints for the same issue, and generate higher tenant satisfaction ratings than properties relying on informal or verbal maintenance reporting.

These findings had direct implications for the design of the maintenance management module included in the proposed system. The module was designed to provide tenants with a formal and accessible channel for submitting maintenance requests and tracking their progress, while providing property managers and maintenance personnel with a structured workflow for prioritizing, assigning, and documenting the resolution of reported issues.

Database Architecture as the Backbone of System Reliability

The literature reviewed on database systems and architecture established that the choice of database technology and schema design had profound implications for the reliability, performance, scalability, and data integrity of any information system. The reviewed studies consistently recommended relational database management systems (RDBMS) for applications that managed structured, interdependent data entities with complex relationships, a description that accurately characterized the data model of a property management system.

Key principles drawn from the database literature were all applied in the design of the database layer of the proposed system. These include normalization to eliminate data redundancy, enforcement of referential integrity through foreign key constraints, indexing to optimize query performance, and transaction management to prevent data corruption. These design decisions ensured that the system maintained the integrity and consistency of all stored data even under concurrent access from multiple users.

Web-Based Accessibility as an Enabler of Operational Continuity

The reviewed literature on web-based information systems established that the accessibility and platform independence afforded by browser-based applications represented a significant operational advantage over desktop-installed or server-bound legacy systems. Web-based systems allowed authorized users to access the system from any device with an internet connection, eliminating the need for dedicated hardware installations, simplifying software updates and maintenance, and enabling remote management of property operations.

For a property management context, this accessibility was particularly valuable. Property managers were able to respond to booking inquiries, review payment statuses, and monitor maintenance requests from outside the office. Tenants were able to submit requests, view their payment history, and communicate with management without having to visit a physical office. These capabilities directly improved the operational responsiveness and service quality of Gales Urban Haven, and the proposed system was designed as a web-based application to realize these benefits fully.

Agile Methodology as the Appropriate Development Framework

The literature reviewed on software development methodologies provided strong justification for selecting the Agile methodology as the development framework for the proposed system. The Agile approach, characterized by iterative development cycles, continuous stakeholder collaboration, adaptive planning, and incremental delivery of working software, was shown to be particularly well suited for projects in which the full scope of requirements could not be exhaustively defined at the outset, a condition that accurately described the development context of the proposed system.

The reviewed studies documented that Agile projects consistently achieved higher rates of stakeholder satisfaction, more timely delivery, and better alignment between the delivered system and actual user needs compared to projects managed under the traditional Waterfall methodology. By adopting Agile, the development team was positioned to respond constructively to feedback from the management and staff of Gales Urban Haven throughout the development process, ensuring that the final system accurately reflected the operational realities and priorities of the organization.

Research Gaps Addressed by the Proposed System

While the reviewed literature provided robust evidence for the value of property management information systems in general, it also revealed a number of research and implementation gaps that the proposed system was positioned to address. These gaps related to the specific context of small- to medium-scale condominium operations in emerging urban markets in the Philippines, the integration of multiple management functions within a single unified system, and the accessibility of such systems to organizations with limited technical infrastructure.

Theoretical Frameworks Informing the Proposed System

Beyond the empirical findings reported in individual studies, the reviewed literature also surfaced several theoretical frameworks and conceptual models that provided a principled basis for the design decisions incorporated in the proposed system. These frameworks guided not only what the system did but also how it was structured, how its components related to one another, and how it was evaluated upon completion. The theoretical foundations ensured that the proposed system was not merely a technical solution to immediate operational problems but a thoughtfully designed platform grounded in established principles of information systems development, property management theory, and organizational efficiency. Each framework contributed a distinct perspective that informed specific aspects of the system architecture, user interface design, data management approach, and overall functional scope.

The application of these conceptual models also provided a systematic basis for justifying the design choices made throughout the development process, ensuring that every major decision was supported by both theoretical reasoning and empirical evidence drawn from existing literature. Furthermore, integrating multiple theoretical frameworks allowed the development team to approach the system design from a multidimensional perspective, accounting for the technical, organizational, and human factors that collectively determined the effectiveness and sustainability of the proposed system in a real-world property management environment.

As the system progressed through its development and evaluation phases, these frameworks continued to serve as reference points against which the system's performance, usability, and alignment with organizational goals were measured. Future researchers and developers who built upon the findings of this study could also benefit from the theoretical groundwork established herein, as it provided a coherent and well-documented intellectual foundation for subsequent enhancements, adaptations, and expansions of the proposed system. Ultimately, the deliberate incorporation of theoretical frameworks into the system design process elevated the academic rigor and practical relevance of this capstone project, demonstrating that the proposed solution was not only technically functional but also conceptually sound and aligned with the broader body of knowledge in information technology and property management.

Systems Theory and the Integrated Operations Model

The concept of the organization as an integrated system, in which all operational processes were interdependent and the performance of the whole was a function of the coherence of its parts, provided the theoretical foundation for the integrated design of the proposed system. Rather than developing separate standalone tools for booking, payment, and maintenance, the proposed system was designed as an interconnected platform in which data generated by one module was immediately available to and usable by all other modules. This integrative approach, grounded in systems theory, eliminated the data silos that were identified as a primary source of inefficiency in the reviewed literature.

The application of systems theory ensured that each module operated not as an isolated component but as a contributing element of a larger, cohesive operational framework that served the holistic information and management needs of Gales Urban Haven. By treating the organization as a unified system of interdependent processes, the proposed system was capable of reflecting the true complexity of condominium property management

operations, where a single transaction or event in one area of the business inevitably produced ripple effects across multiple other functional domains. The interconnected architecture also enabled the system to maintain a single, consistent, and continuously updated data environment, ensuring that all modules operated on accurate and current information rather than fragmented or duplicated records maintained across separate platforms.

As the system was adopted and utilized, the systemic coherence built into its design contributed to measurable improvements in the speed, accuracy, and reliability of administrative decision-making, as administrators were able to access a comprehensive and integrated view of all operational activities from a single centralized interface. The systems theory foundation also provided a scalable and flexible structural framework that accommodated the addition of new modules and functionalities without compromising the integrity or coherence of the overall platform. It also informed the evaluation criteria used to assess the success of the proposed system, with particular emphasis on the degree to which integration across modules reduced redundancy, improved data consistency, and supported more efficient and informed organizational management. Ultimately, grounding the system in systems theory strengthened its technical design and kept it aligned with the operational realities and strategic goals of Gales Urban Haven.

The Technology Acceptance Model (TAM)

The Technology Acceptance Model, which posited that the adoption and effective use of an information system were determined primarily by users' perceptions of its usefulness and ease of use, informed the user experience design strategy of the proposed system. In line with the TAM framework, the system was designed with a strong emphasis on interface clarity, logical workflow navigation, and minimal cognitive load for end users. The user interface was iteratively refined through stakeholder feedback sessions conducted during the Agile development process, ensuring that the delivered system met the usability standards necessary for adoption by all categories of users at Gales Urban Haven. Furthermore, the principles of the Technology Acceptance Model guided the researchers in evaluating the degree to which the proposed system was perceived as both useful and easy to use by its intended users, providing a structured basis for assessing user satisfaction and system acceptance upon implementation.

The TAM framework also informed the design of system features that directly addressed the practical needs of administrators and business owners, ensuring that every functional module contributed meaningful value to their daily operational tasks and decision-making processes. Additionally, the emphasis on perceived ease of use drove the development of an intuitive and straightforward user interface that minimized the time and effort required for staff members to learn and navigate the system, thereby reducing resistance to adoption and encouraging consistent system utilization. The system also incorporated clear instructions, tooltips, and navigational cues that helped users understand and make full use of its features, further reinforcing their confidence and comfort in using the platform.

Moreover, applying the TAM framework ensured that the proposed system was not only technically functional but also embraced and regularly used by its end users, as a system perceived as useful and easy to use was more likely to deliver sustained operational benefits to Gales Urban Haven. Ultimately, by anchoring the design and development

process within the theoretical principles of the Technology Acceptance Model, the researchers ensured that the Condominium Booking and Operations Management System was a practical, user-centered solution that effectively supported the operational goals and digital transformation of Gales Urban Haven under One Spatial Iloilo.

The CRUD Operational Model

The foundational database operations model, Create, Read, Update, Delete (CRUD), served as the structural template for all data management functionalities incorporated in the proposed system. Every entity managed by the system, whether tenant records, booking entries, payment transactions, or maintenance requests, was governed by a complete set of CRUD operations, ensuring that authorized users were able to create new records, retrieve existing ones, modify information as circumstances changed, and remove outdated entries in a controlled and auditable manner. Furthermore, the CRUD model provided a consistent and standardized approach to data management across all system modules, ensuring that every functional component interacted with the centralized database in a uniform, reliable, and predictable manner.

The structured application of CRUD operations also contributed to the overall integrity and accuracy of the system's data, as each operation was subject to validation rules and access controls that prevented unauthorized or erroneous modifications to critical business records. Additionally, the CRUD model served as the foundation for the system's audit trail functionality, as every create, update, and delete action performed within the system was recorded and timestamped, allowing administrators to trace the history of any record and maintain full accountability over all data management activities.

The read operations embedded within the CRUD framework also supported the system's reporting and data retrieval functions, enabling administrators and business owners to access accurate and up-to-date information quickly and efficiently whenever it was needed for operational or decision-making purposes. Moreover, adopting the CRUD model simplified the development and maintenance of the proposed system's database layer, as the well-established and widely recognized nature of the framework provided the development team with a clear and structured blueprint for implementing all data management functionalities. Ultimately, the CRUD operational model ensured that the Condominium Booking and Operations Management System maintained a high standard of data organization, accuracy, and security, providing Gales Urban Haven with a robust and dependable platform for managing all aspects of its condominium rental operations.

Direct Influence of the Literature on the Proposed System's Design

The reviewed literature not only validated the need for the proposed system but also exerted a direct and traceable influence on its specific design features, functional modules, and technical architecture. The mapping of key literature-derived insights to the corresponding design decisions reflected the features and functionalities incorporated into the Condominium Booking and Operations Management System.

Validation of the Proposed System's Relevance and Necessity

The weight of evidence accumulated across the reviewed literature collectively validated the relevance, necessity, and appropriateness of the Condominium Booking and Operations Management System proposed in this study. This validation operated at multiple levels. At the problem level, the literature confirmed that the operational challenges documented at Gales Urban Haven were well-recognized phenomena with known root causes and known solutions. At the solution level, the literature established that technology-enabled, integrated property management systems represented the most effective and sustainable response to these challenges. At the design level, the literature provided specific, evidence-based guidance on the features, architecture, and development approach that maximized the probability of the system achieving its intended outcomes.

Furthermore, the literature affirmed that the investment in such a system yielded benefits that extended well beyond operational convenience. Organizations that successfully implemented integrated property management systems were positioned to offer a higher standard of service to tenants, operate with greater financial discipline and transparency, respond more effectively to regulatory oversight, and scale their operations with confidence as their property portfolios grew. For Gales Urban Haven, these outcomes translated directly into enhanced competitiveness within the Iloilo City condominium market, improved tenant retention, and a stronger operational foundation for sustainable long-term growth.

REFERENCES

[1] W. Imkumnert, "Service condominium information system for condominium rental business," System Development Project, Assumption Univ., Bangkok, Thailand, 2002.

[2] C. Halkiopoulos, H. Antonopoulou, D. Papadopoulos, I. Giannoukou, and E. Gkintoni, "Online reservation systems in e-Business: Analyzing decision making in e-Tourism," J. Tourism, Heritage & Services Marketing, vol. 6, no. 1, pp. 9-16, 2020, doi: 10.5281/zenodo.3603312.

[3] [Initial(s).] Varutbangkul, "Condominium management information system," [Thesis/Project type], Assumption Univ., Bangkok, Thailand, 2000.

[4] Y.-C. Kuo and J.-S. Chou, "Enhancement of condominium management based on the effect of quality attributes on satisfaction improvement," Expert Syst. Appl., vol. 39, no. 5, pp. 5418-5425, 2012, doi: 10.1016/j.eswa.2011.11.043.

[5] L. L. Bajado Jr., "Assessment of hotel business tools and technologies usage towards guest satisfaction in Tagaytay City, Philippines: Basis for an excellent service provision," Int. J. Res. Innov. Soc. Sci. (IJRISS), vol. 9, no. 9, pp. 3227-3237, 2025, doi: 10.47772/IJRISS.2025.909000273.

[6] R. R. U. Gosela and R. E. Encarnacion, "Booking and reservation system: A unified application using location-based services for sustainable tourism networks," Int. J. Adv. Res. Sci. Commun. Technol. (IJARSCT), 2024.

[7] R. C. Gallenero, J. M. Flores, R. M. Gundran, J. V. Rosario, G. F. Ubaldo, M. V. S. Solomo, M. C. G. Fernando, and A. C. Lagman, "Occupy - Real-estate management system with sales analysis and customer segmentation," in Proc. 2022 IEEE 14th Int. Conf. Humanoid, Nanotechnol., Inf. Technol., Commun. Control, Environ., Manage. (HNICEM), 2022, pp. 1-5, doi: 10.1109/HNICEM57413.2022.10109621.

[8] J. A. Bardi, Hotel Front Office Management, 5th ed. Hoboken, NJ, USA: Wiley, 2011.

[9] D. V. Tesone, Hospitality Information Systems and E-Commerce. Hoboken, NJ, USA: Wiley, 2012.

[10] Dept. of Information and Communications Technology, "ICT adoption and utilization survey for small and medium enterprises in the Philippines," DICT Publications, Republic of the Philippines, 2020.

[11] C. Cobanoglu, K. Berezina, M. L. Kasavana, and M. Erdem, "The impact of technology amenities on hotel guest overall satisfaction," J. Qual. Assur. Hosp. Tour., vol. 12, no. 4, pp. 272-288, 2011.

[12] P. O'Connor and J. Murphy, "Hotel yield management practices across multiple electronic distribution channels," Inf. Technol. Tourism, vol. 10, no. 2, pp. 161-172, 2008.

[13] J. A. Siguaw and C. A. Enz, "Best practices in information technology," Cornell Hotel Restaurant Admin. Q., vol. 40, no. 5, pp. 58-71, Oct. 1999.

[14] M. L. Kasavana and R. M. Brooks, Managing Front Office Operations, 8th ed. Lansing, MI, USA: Educ. Inst. Amer. Hotel & Lodging Assoc., 2009.

[15] A. Pizam and T. Ellis, "Customer satisfaction and its measurement in hospitality enterprises," Int. J. Contemp. Hospitality Manage., vol. 11, no. 7, pp. 326-339, 1999.

[16] C. S. Warren and J. P. Jones, Corporate Financial Accounting, 11th ed. Boston, MA, USA: Cengage Learning, 2010.

[17] T. M. Clauretie and G. S. Sirmans, Real Estate Finance: Theory and Practice, 6th ed. Boston, MA, USA: Cengage Learning, 2010.

[18] C. Turner, Financial Management in the Hospitality Industry. Oxford, U.K.: Goodfellow Publishers, 2012.

[19] E. F. Brigham and J. F. Houston, Fundamentals of Financial Management, 15th ed. Boston, MA, USA: Cengage Learning, 2019.

[20] C. T. Horngren, S. M. Datar, and M. V. Rajan, Cost Accounting: A Managerial Emphasis, 15th ed. Boston, MA, USA: Pearson, 2015.

[21] P. D. Kimmel, J. J. Weygandt, and D. E. Kieso, Financial Accounting: Tools for Business Decision-Making, 8th ed. Hoboken, NJ, USA: Wiley, 2016.

[22] B. S. Dhillon, Maintainability, Maintenance, and Reliability for Engineers. Boca Raton, FL, USA: CRC Press, 2006.

[23] R. K. Mobley, An Introduction to Predictive Maintenance, 2nd ed. Amsterdam, The Netherlands: Butterworth-Heinemann, 2002.

[24] T. Wireman, Developing Performance Indicators for Managing Maintenance, 2nd ed. New York, NY, USA: Industrial Press, 2004.

[25] N. Gaither and G. Frazier, Operations Management, 9th ed. Cincinnati, OH, USA: South-Western College Publishing, 2002.

[26] M. L. Pinedo, Scheduling: Theory, Algorithms, and Systems, 4th ed. New York, NY, USA: Springer, 2012.

[27] C. J. Date, An Introduction to Database Systems, 8th ed. Boston, MA, USA: Pearson Education, 2004.

[28] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed. Hoboken, NJ, USA: Pearson Education, 2016.

[29] A. Silberschatz, H. F. Korth, and S. Sudarshan, Database System Concepts, 7th ed. New York, NY, USA: McGraw-Hill Education, 2019.

[30] K. C. Laudon and J. P. Laudon, Management Information Systems: Managing the Digital Firm, 16th ed. Hoboken, NJ, USA: Pearson Education, 2020.

[31] J. A. O'Brien and G. M. Marakas, Management Information Systems, 10th ed. New York, NY, USA: McGraw-Hill/Irwin, 2011.

[32] E. Turban, L. Volonino, and G. R. Wood, Information Technology for Management: Digital Strategies for Insight, Action, and Sustainable Performance, 10th ed. Hoboken, NJ, USA: Wiley, 2015.

[33] R. S. Pressman and B. R. Maxim, Software Engineering: A Practitioner's Approach, 9th ed. New York, NY, USA: McGraw-Hill Education, 2020.

[34] I. Sommerville, Software Engineering, 10th ed. Harlow, U.K.: Pearson Education, 2016.

[35] K. Schwaber and J. Sutherland, "The Scrum Guide: The definitive guide to Scrum, the rules of the game," Scrum.org, Nov. 2017. [Online]. Available: https://scrumguides.org

CHAPTER III

METHODOLOGY

3.1 Research Design

This study employed a developmental research design that covered the planning, design, development, implementation, and evaluation of a web-based Condominium Booking and Operations Management System. The design was appropriate because it allowed the researchers to build a functional system that addressed problems in manual condominium and rental management, and to evaluate its functionality, usability, reliability, and effectiveness [1].

Developmental research, as distinguished from purely descriptive or experimental research, is concerned with producing a tested artifact as its principal output rather than with establishing a generalizable causal relationship between variables [1]. This distinction is significant for the present study, because the primary research question was not whether automation improves property management outcomes in the abstract, a proposition already well supported in the literature reviewed in Chapter II, but rather whether a specific, purpose-built system could be designed, implemented, and validated as a workable solution to the particular operational deficiencies documented at Gales Urban Haven. The developmental design therefore treats the system itself as the unit of evaluation, with its success measured against the functional and non-functional requirements derived directly from the business's own operations rather than against a hypothesis tested through statistical inference.

The system targeted common problems in manual operations: double bookings, misplaced records, delayed transactions, inaccurate computations, data redundancy, and inefficient report generation. The study proceeded through requirements gathering, system analysis, system design, development, testing, evaluation, and implementation. During requirements gathering, the researchers collected information on current management practices, challenges, and user needs from the target stakeholders. This data served as the basis for the system's features.

Each of these phases was treated as a distinct and necessary stage rather than as an informal progression, since omitting or compressing any one of them was judged likely to compromise the reliability of the final system. Requirements gathering, in particular, was conducted before any design decisions were finalized, on the premise that a system built on an incomplete or assumed understanding of the business's actual workflows would risk automating the wrong processes, or automating the right processes in a way that did not match how administrators and staff actually worked. The output of this phase was not treated as fixed once collected; rather, it was revisited and refined

throughout subsequent phases as the Agile methodology described in Section 3.2 surfaced additional requirements that had not been apparent during the initial consultations.

In the design and development stage, the researchers created the application's structure, interface, database, and functions for managing customer information, room reservations, booking schedules, billing, payment monitoring, and report generation. User authentication and security features were included to protect the confidentiality and integrity of stored data.

The design and development stage was approached as an integrated activity rather than two fully separable steps, since architectural decisions made during design, such as the choice of a centralized relational database and a three-tier client-server architecture, directly constrained and informed the implementation choices made during development, and emerging implementation constraints in turn occasionally required revisiting earlier design assumptions. Security considerations, including hashed credential storage, session-based authentication, and role-based access control, were incorporated from the outset of this stage rather than appended afterward, consistent with the view that security measures retrofitted onto a completed system are both less effective and more costly to implement than those designed in from the beginning.

After development, the system underwent functionality, usability, reliability, and user acceptance testing to identify errors, evaluate performance, and confirm that it met user requirements. Evaluator feedback was used to refine the system.

This testing and evaluation stage served two complementary purposes within the developmental research design. First, it functioned diagnostically, surfacing defects, performance bottlenecks, and usability friction points that could be corrected prior to deployment. Second, it functioned evaluatively, providing the empirical basis on which the researchers could assess, in Chapter IV and Chapter V, whether the completed system actually achieved the objectives set out in Chapter I. The feedback gathered from administrators, business owners, and other end users during this stage was accordingly treated not as a final formality preceding submission, but as an integral input that shaped the last round of refinements made to the system before it was considered complete.

3.2 System Development Methodology

The Condominium Booking and Operations Management System for Gales Urban Haven (operating as One Spatial Iloilo) was developed using the Agile Software Development Model. Agile is an iterative approach that emphasizes flexibility, collaboration, continuous feedback, and incremental delivery [1]. Unlike the Waterfall Model, which requires complete requirements definition before development begins [2], Agile accommodates changing requirements. This suited the project because it integrates several modules: booking management, payment tracking, revenue reporting, calendar-based scheduling, and maintenance monitoring.

The decision to adopt Agile over a Waterfall approach was not made on the general popularity of Agile methods alone, but on a specific assessment of the project's risk profile. A Waterfall approach presumes that requirements can be fully and correctly specified before any development begins, an assumption that was judged unrealistic for this project given that the administrators and staff of Gales Urban Haven had no prior experience articulating their operational needs in the form of formal software requirements, and were, in practice, only able to identify certain needs once shown a working prototype of a related feature. Agile's iterative structure allowed the development team to present incremental working versions of each module to stakeholders throughout the project, surfacing misunderstandings and refining requirements continuously rather than discovering them only after a large portion of the system had already been built against an incorrect specification. The five activities described below correspond to the recurring stages carried out within each development iteration, rather than to a single linear pass through the project.

1) Requirements Analysis The team examined the existing manual processes for recording reservations, tracking payments, monitoring maintenance, and scheduling. Methods included consultations with property administrators and key personnel, structured interviews, observation of daily workflows, and review of existing records. The findings were translated into functional requirements (what the system must do) and non-functional requirements (performance, security, usability, reliability). Stakeholders, user roles, access levels, and technical constraints were also identified. The resulting requirements specification guided all later phases. Because this activity recurred across iterations rather than occurring only once at the project's outset, later rounds of requirements analysis were able to incorporate lessons learned from stakeholder reactions to earlier modules, allowing the specification to be corrected and sharpened progressively rather than being treated as fixed from the first consultation onward.

2) System Design: The requirements were translated into an architectural blueprint covering system architecture, database schema, interface layouts, navigation flows, and module interdependencies. The design followed principles of clarity, simplicity, security, and scalability. Wireframes and mockups were reviewed to validate the interface before coding began. Entity-relationship diagrams (ERDs) and data flow diagrams (DFDs) defined the centralized database and data movement. Security measures, including authentication, access control, and data encryption, were incorporated at this stage. Reviewing wireframes and mockups with representative end users before writing any code proved particularly valuable for a user base with limited prior exposure to digital management tools, since it allowed navigation and layout concerns to be identified and corrected while changes were still inexpensive, rather than after the corresponding interface logic had already been implemented.

3) System Development: The modules were implemented in iterative cycles: booking management, payment tracking, revenue reporting, calendar-based scheduling, maintenance monitoring, the centralized database, and user authentication. Each module was built, integrated, and reviewed in turn. The team followed modular code design, consistent coding standards, code documentation, and version control. The modules were integrated so that data flowed between them. For example, reservations recorded in the booking module automatically updated unit availability in the calendar module, and payments recorded in the payment module were reflected in the revenue

reporting module. This cross-module data propagation was treated as a first-class design requirement throughout development rather than as an integration step performed only after each module was independently complete, since the core value proposition of the system, as established in Chapter II, depended on eliminating the data silos that had characterized the business's previous manual processes. Building modules with this interdependency in mind from the start reduced the risk of having to substantially rework a module's data model after the fact to accommodate another module's needs.

4) Testing: Testing was conducted at multiple levels: Unit testing verified individual modules and functions under normal and boundary conditions. Integration testing verified module interaction and data transfer. System testing evaluated the integrated system's responsiveness, data accuracy, error handling, security, and accessibility. Performance testing assessed stability under varying loads. User acceptance testing (UAT) had selected administrators and end users provide structured feedback on usability and suitability. Defects were documented, prioritized, and resolved before deployment. Conducting testing at each of these levels within every iteration, rather than reserving it for a single phase near the end of the project, allowed defects to be identified and corrected close to the point at which they were introduced, which the team found considerably less costly than attempting to diagnose a defect's root cause after several subsequent iterations had already been built on top of the flawed component.

5) Deployment: The system was deployed as a functional prototype and presented to the administrators and key personnel of Gales Urban Haven for demonstration and evaluation. The team configured the system, created and assigned user accounts, and initialized the database. Orientation and walkthroughs familiarized administrators with the system. Feedback from this phase was used to identify remaining refinements, consistent with Agile's continuous improvement cycle. This deployment was treated as a milestone within an ongoing iterative process rather than as the definitive conclusion of development, consistent with the Agile principle that a working system, once in the hands of its intended users, typically reveals refinement needs that no amount of internal testing can fully anticipate. The orientation sessions conducted alongside this deployment also served a secondary evaluative function, allowing the team to directly observe which aspects of the interface or workflow required further explanation, information that was fed back into the final round of usability refinements described in Chapter IV.

3.3 System Architecture

This architecture was chosen for three reasons. First, multiple authorized users can access the system simultaneously from different devices through a standard web browser, with no client-side installation. Second, centralized data processing and storage ensure that all users work with the same current data. Third, updates to application logic or the database are applied only on the server, which simplifies maintenance. These characteristics suit One Spatial Iloilo, where administrative staff use different workstations within the property management office.

These three reasons were weighed against alternative architectural approaches before the client-server model was adopted. A standalone desktop application installed on individual office computers, for instance, was considered and rejected, since it would have required each workstation to maintain its own local copy of operational data, reintroducing precisely the kind of synchronization risk and data inconsistency that a centralized system was intended to eliminate. The client-server model, by contrast, ensures that the browser running on any given workstation is only ever a presentation layer for data that physically resides in one place, so that an update made by one staff member is immediately visible to every other authorized user without any manual file-sharing or merge step.

1. **Client Tier (Presentation Layer):** The client tier is the front end through which administrators and property managers interact with the system. It runs in a web browser and is built with HyperText Markup Language (HTML) for structure, Cascading Style Sheets (CSS) for presentation, and JavaScript for dynamic behavior. The interface uses organized menus, dashboards, tables, forms, summary cards, and interactive calendar views to give access to the booking, payment, revenue, scheduling, and maintenance modules. It is responsive and works on desktop, laptop, and tablet screens. User actions, such as submitting a booking, recording a payment, or updating a maintenance request, are sent to the server as structured requests. The client then displays the server's response as immediate feedback. The client tier was deliberately kept free of any business logic beyond basic input validation, such as confirming that a required field has been filled in before a request is sent, so that every substantive decision, such as whether a unit is actually available or whether a payment amount is valid, is made once, consistently, on the server rather than being potentially duplicated or contradicted across different client sessions.

2. **Server Tier (Application and Logic Layer):** The server tier is the central processing engine. It receives requests from the client, executes the corresponding business logic and data operations, and returns responses. Its functions include validating booking submissions, computing payment balances and revenue, enforcing scheduling rules and conflict prevention, managing maintenance records and status updates, and generating financial and operational reports. The server-side application was developed using Node.js and Express, which handle concurrent user requests efficiently. Centralizing business logic at this tier also provides a single point at which security controls, such as authentication checks and role-based access restrictions, can be enforced consistently for every request, regardless of which client device or browser originated it, which is considerably more reliable than attempting to replicate equivalent checks across multiple client implementations.

3. **Data Tier (Database Layer):** All operational data are stored in a centralized MySQL database accessed only through the server tier, ensuring data integrity and security. Restricting direct database access to the server tier alone, rather than allowing the client to query the database directly, ensures that every read and write operation passes through the same validation and authorization logic, closing off a class of vulnerabilities in which a client could otherwise be modified to issue unauthorized or malformed queries directly against the data store.

4. **Data Flow and Component Integration:** The three tiers operate as one integrated system. When an administrator submits a booking, the client sends a request to the server through the defined communication protocol. The server validates the data against business rules, checks unit availability in the database, records the booking, and returns a confirmation. The client then updates the booking dashboard and calendar view.

Payments follow the same path. On recording a payment, the server computes the updated outstanding balance, stores the transaction, and propagates the new figures to the revenue monitoring module and the tenant's payment history in real time. This cross-module propagation keeps every section of the system consistent and avoids the data inconsistencies and lags common in manual record-keeping. The architecture also allows new modules, features, or user roles to be added without disrupting existing functionality.

This consistent request-validate-respond pattern, repeated across every module rather than implemented differently for each one, was a deliberate architectural choice intended to keep the system predictable and maintainable as it grew. A maintenance request submission, for example, follows the same essential path as a booking or a payment: the client sends a structured request, the server validates it against business rules and writes it to the database, and the updated state is reflected back across every interface that depends on it, including the administrator's maintenance dashboard and the operational calendar described in Section 3.8. Because every module adheres to this same architectural pattern, extending the system with an additional module in a future iteration, such as a tenant communication log or an automated billing reminder, would require implementing the same client-server-database flow already established, rather than introducing a fundamentally new architectural approach, which supports the scalability objective identified in Section 1.3.2.

3.4 Tools and Technologies

The system uses a full-stack JavaScript/TypeScript architecture, chosen for type safety, developer productivity, and maintainability. Each tool was selected for its maturity, community support, performance, and fit with the project's requirements.

This stack was not assembled piecemeal but selected as a coherent whole, with each tool chosen partly for how well it interoperated with the others. Using JavaScript and TypeScript across both the client and server, for instance, meant that the development team did not need to maintain separate skill sets or toolchains for the frontend and backend, and allowed type definitions describing a booking, a payment, or a maintenance request to be written once and shared between both tiers, reducing the risk that the client and server would silently disagree about the shape of the data being exchanged.

3.4.1 Frontend Technologies

React 19: An open-source, component-based UI library maintained by Meta [3]. Its reusable components manage their own state, and its concurrent rendering model keeps the interface responsive under heavy load. Its declarative paradigm simplifies reasoning about state and supports unit testing of individual components. React's component model was particularly well suited to this project because much of the interface, such as a booking card, a payment status badge, or a maintenance ticket row, consists of small, repeatable units of interface that appear in multiple contexts across the Resident and Administrator portals described in Chapter IV. Building these as discrete, reusable components reduced duplicated interface code and made it possible to update the appearance or behavior of a single component, such as a status indicator, in one place and have that change propagate consistently everywhere it was used.

TypeScript: A statically typed superset of JavaScript developed by Microsoft [4]. Compile-time type checking reduces runtime errors, and shared type definitions act as contracts between frontend and backend. Because plain JavaScript does not catch type-related mistakes until a program is actually run, errors such as passing a payment amount as a text string instead of a number, or omitting a required field from a booking object, could otherwise have gone undetected until a user encountered them in the deployed system. TypeScript's compile-time checking surfaced many such errors during development itself, before the affected code was ever executed, which the team found especially valuable given the financial and scheduling data the system handles.

Tailwind CSS: A utility-first CSS framework [5]. Unused styles are purged in production builds, and its responsive utilities support a mobile-first layout. Its design tokens (colors, spacing, typography, breakpoints) were customized to the project's visual specifications. Rather than writing separate, hand-maintained stylesheets for each interface component, Tailwind allowed styling to be expressed directly alongside the markup it affected, through a consistent set of utility classes. This reduced the amount of unused or conflicting CSS that tends to accumulate in larger projects over time, and made it straightforward to keep spacing, color, and typography consistent across the many distinct views that make up the Resident and Administrator portals.

Vite: A build tool and development server [6]. It uses native ES Modules for fast startup and hot module replacement (HMR) in development, and Rollup for production builds with code splitting and tree shaking. It also handled path aliasing, environment variables, and API proxying. The fast feedback loop that Vite's hot module replacement provided during development, in which a code change is reflected in the running application in a fraction of a second without a full page reload, was particularly valuable given the project's Agile, iteration-driven workflow described in Section 3.2, since it allowed interface changes made in response to stakeholder feedback to be previewed and adjusted quickly during the same working session.

3.4.2 Backend Technologies

Node.js: A cross-platform JavaScript runtime built on the V8 engine [7]. Its event-driven, non-blocking I/O model handles many concurrent requests efficiently, and using one language across the stack allows code and types to be shared. The npm ecosystem supplied common modules for authentication, validation, and API communication. Node.js's non-blocking model is well suited to a system such as this one, in which the server spends much of its time waiting on database queries rather than performing heavy computation, since it allows the server to begin processing another administrator's request while a previous database query is still being completed, rather than making each user wait in a strict queue behind the others.

Express: A minimal web framework for Node.js [8] used to build the RESTful API. Its middleware pipeline handles request logging, JSON Web Token (JWT) authentication and authorization, input validation, error handling, and Cross-Origin Resource Sharing (CORS). Endpoints use standard HTTP methods (GET, POST, PUT, PATCH, DELETE) and return JSON. Express's middleware architecture allowed cross-cutting concerns, such as verifying that a request carries a valid authentication token or logging every incoming request for later audit purposes, to be implemented once and applied consistently across every endpoint, rather than being re-implemented individually within each module's booking, payment, or maintenance logic.

TypeScript (backend): Strongly typed request and response schemas were defined with TypeScript interfaces and the runtime validation library Zod [9]. Combining compile-time and runtime validation ensures the server processes only well-formed data. Compile-time typing alone cannot guard against malformed data arriving from outside the application, such as a manipulated request sent directly to the API rather than through the system's own interface, so Zod was used to re-validate incoming data against the same schema definitions at runtime, ensuring that the server rejects malformed or unexpected input before it reaches any business logic or database operation.

tsx and esbuild: tsx runs TypeScript directly during development, using esbuild for fast transpilation [10]. esbuild also bundles the production backend into a single deployable JavaScript file, avoiding complex Webpack or Babel configurations. These tools were selected specifically to minimize the configuration overhead that more established bundlers often require, allowing the development team to spend its limited time on the system's functional requirements rather than on maintaining and debugging build tooling.

Database: MySQL is the relational database management system [11]. It stores tenant profiles, bookings, payments, unit availability, and maintenance logs. Primary keys, foreign key constraints, and normalization enforce integrity, while indexing and transaction support enable efficient real-time reporting and search. MySQL's maturity and broad adoption were significant factors in its selection, given that extensive documentation, hosting support, and tooling are readily available, which reduces the risk of the system becoming difficult to maintain or migrate in the years following this study's completion, a consideration directly relevant to the long-term maintainability goals discussed in Section 1.3.2.

Development Environment: Visual Studio Code was the integrated development environment (IDE) for all phases. Its extensions provided linting, formatting, database clients, and debugging, and its built-in Git integration supported version control and collaboration. Standardizing on a single IDE across the development team also made it easier to

share consistent linting and formatting configurations, ensuring that code contributed by different team members followed the same stylistic and structural conventions throughout the project.

TABLE I. SUMMARY OF TECHNOLOGIES

Layer	Technology	Purpose
Frontend	React 19	Component-based UI
Frontend	TypeScript	Type-safe programming
Frontend	Tailwind CSS	Utility-first styling
Frontend	Vite	Dev server and production bundler
Backend	Node.js	Server runtime
Backend	Express	RESTful API framework
Backend	TypeScript, Zod	Type safety and runtime validation
Backend	tsx, esbuild	Development execution and production bundling
Database	MySQL	Relational data storage
Tooling	Visual Studio Code, Git	IDE and version control

Rationale for Technology Selection: The stack was chosen for four reasons: Unified language: TypeScript on both tiers reduces code duplication and mismatches between client and server data models. Performance: Vite and esbuild shorten development feedback loops and build times. Community support: All selected tools have wide adoption, documentation, and maintained packages. Scalability: React's component model, Express's modular middleware, and TypeScript's type checking allow incremental growth with a lower risk of regressions.

Each of these four considerations addressed a distinct risk the development team identified before committing to a particular stack. The unified-language rationale, for instance, was weighed against the alternative of using different languages on the client and server, an approach common in many existing property management platforms but one that the team judged would introduce an unnecessary translation layer between the two tiers, increasing both development time and the likelihood that a data structure defined on one side would drift out of sync with its counterpart on the other. The performance rationale was assessed not only in terms of the end user's experience of the deployed system, but in terms of the development team's own iteration speed, since a slow build and reload cycle directly limits how many design-test-refine cycles can be completed within the Agile sprints described in Section 3.2. The community-support consideration reduced the risk that the team would encounter an undocumented defect or an unmaintained dependency with no available fix, a particular concern given that the system is intended to

remain in use, and potentially be extended by future student teams, well beyond the completion of this study. Finally, the scalability consideration was assessed against the delimitations outlined in Section 1.4.2, which anticipate that future versions of the system may add features such as multi-property support or external payment gateway integration; a stack whose components support incremental extension without large-scale rewrites was therefore judged preferable to one offering marginally better performance at the cost of long-term flexibility.

3.5 Data Gathering Techniques

Two techniques were used to understand the operating environment at Gales Urban Haven (One Spatial Iloilo) and to ground the functional requirements in real practice. These two techniques were treated as complementary rather than redundant, since each addressed a different risk in requirements specification. Process analysis grounded the requirements in what the business's own staff actually did on a daily basis, while the review of existing platforms guarded against the narrower risk of designing a system that merely replicated the business's current manual habits in digital form, without benefiting from established conventions and features already proven effective elsewhere in the property management domain.

1) Process Analysis: The researchers examined the existing manual workflows through direct observation of daily administrative activities, structured interviews with staff who manage bookings and financial records, and review of the paper-based and spreadsheet tools in use. Each process was mapped end to end, from tenant inquiry through booking confirmation, payment collection, maintenance request, and unit handover. This exposed three main problems: transcription errors from manual data transfer, booking conflicts caused by the lack of a centralized schedule, and forgotten maintenance concerns caused by informal tracking. The findings shaped the functional specifications of the core modules. Mapping each process end to end, rather than examining individual tasks in isolation, was a deliberate methodological choice, since several of the problems ultimately identified, such as a maintenance concern being reported through one channel while a related booking adjustment was recorded through another, were only visible when the full sequence of a tenant's interaction with the business was traced as a single continuous workflow rather than as a set of disconnected clerical tasks.

2) Review of Existing Platforms: The researchers reviewed property management systems used in residential and hospitality contexts. They examined how these systems handle booking, payment tracking, calendar scheduling, maintenance monitoring, and reporting. The systems were evaluated against the needs of a small-to-medium condominium rental business, which identified applicable best practices and features that were over-complex for this context. Interface conventions, data models, reporting formats, and security approaches informed the design of comparable components. This review served a deliberately calibrating function: rather than adopting the full feature set of any single existing platform, the researchers used this comparative review to distinguish between features that addressed a genuine, validated need at Gales Urban Haven and features that, while common in larger commercial

systems, would have added interface and database complexity disproportionate to the scale of a single-property operation, consistent with the delimitations established in Section 1.4.2.

3.6 Testing Procedures

Testing was conducted in three sequential phases. Structuring testing into three distinct, sequential phases, rather than treating it as a single undifferentiated activity performed once before deployment, allowed each phase to target a different class of defect at the point in development where it was most efficiently identified. Errors isolated to a single module's logic were intended to be caught at the unit level, before that module was ever combined with others; errors arising specifically from the interaction between modules were intended to be caught at the system level; and mismatches between the system's technical correctness and its actual usability for non-technical staff were intended to be caught only once real end users could interact with a complete, working version of the system.

1) Unit Testing: Individual modules were tested in isolation: authentication and login, booking creation and retrieval, payment recording and calculation, calendar availability updates, and maintenance request handling. Test cases specified input conditions, expected outputs, and actual outputs, covering valid, invalid, and boundary inputs. The team wrote and ran the tests during implementation, following a test-driven approach where feasible. Discrepancies were traced to their root causes and corrected before the module advanced. Boundary input testing was given particular attention in modules involving financial calculation, such as payment recording, since errors at edge cases, such as a payment amount of exactly zero or a balance computation involving a partial installment, are both more likely to occur and more consequential than errors arising from typical, well-formed input.

2) System Testing: The integrated system was tested as a whole in an environment mirroring the intended production configuration. Testing covered: Integration checks, confirming that a booking appears correctly in the calendar, that payments aggregate in the revenue dashboard, and that maintenance requests flag the correct unit status; performance testing, assessing responsiveness under realistic concurrent usage; security testing, verifying authentication controls and role-based access on protected endpoints; and regression testing, confirming that defect fixes did not break verified functions. Running these checks against a production-like environment, rather than solely within the development team's local setup, was intended to surface any discrepancies arising from configuration differences, such as network latency or database connection limits, that might not otherwise appear until the system was already in live use at Gales Urban Haven.

3) User Acceptance Testing (UAT): Administrative staff and property management personnel of Gales Urban Haven performed realistic scenarios: recording a booking, processing a payment and verifying the revenue summary, checking unit availability, submitting and updating a maintenance request, and generating a financial report for a given period. Participants completed the tasks independently and gave structured feedback on interface clarity, navigation, and output accuracy. The feedback informed final refinements to the interface, workflows, and

documentation before deployment. Having participants complete these scenarios independently, without direct guidance from the development team during the exercise itself, was intended to approximate how the system would actually be used after deployment, when no developer would be present to clarify an ambiguous interface element or explain an unclear workflow step, making this phase the most direct test of the usability criterion detailed in Section 3.7.

3.7 Evaluation Criteria

The system was evaluated on five criteria aligned with the research objectives, using quantitative measurements, structured user feedback, and technical verification. Evaluators included administrators, business owners, and other authorized end users. The results were documented in an evaluation report.

These five criteria were not selected arbitrarily but were derived directly from the specific objectives enumerated in Section 1.3.2 and from the standardized software quality dimensions discussed in Chapter II, so that the evaluation framework would assess the system against the same terms in which its success was originally defined, rather than against a generic or unrelated set of benchmarks. Functionality and accuracy correspond most directly to whether the system does what it was built to do; usability and efficiency correspond to whether it does so in a way that produces real operational benefit for its intended users; and security corresponds to whether it does so without introducing unacceptable risk to the business's operational and financial data. Taken together, the five criteria were intended to provide a reasonably complete picture of the system's readiness for sustained use, rather than a narrow confirmation that the code simply executed without crashing.

1) Functionality: This criterion measures whether the system completely and accurately performs all specified tasks: reservation records, payment processing, revenue and occupancy reports, calendar availability, and maintenance logging. Test cases based on actual Gales Urban Haven workflows were run under standard and boundary conditions, with pass rates recorded. The system is functionally adequate when all defined use cases run without data errors, unhandled exceptions, or failures, and when the modules exchange data without loss or synchronization errors. Basing the test cases on the business's own actual workflows, rather than on generic or hypothetical scenarios, was intended to ensure that a system capable of passing functional testing would also be capable of handling the real operational situations it would encounter in daily use at Gales Urban Haven, including edge cases specific to the business, such as a reservation that is modified after a partial payment has already been recorded.

2) Usability: This criterion measures how easily staff learn the system, navigate between modules, and complete tasks without extensive training. It was assessed through structured observation and feedback during UAT, plus a post-UAT satisfaction survey. The results identified interface elements or navigation flows needing simplification. This criterion was treated as particularly significant given that the system's intended users include staff with limited prior experience with digital management tools, as noted in Section 1.1; a system that is technically correct but difficult to learn or navigate would fail to deliver the workload reduction and error minimization that constitute its central value proposition, regardless of how well it performs on the other four criteria.

3) Accuracy: This criterion measures the correctness of computations and records, especially payment totals, outstanding balances, monthly revenue, and occupancy rates. System outputs were compared against manually verified benchmark values from known datasets and checked for reproducibility across repeated runs. Accuracy also covers the integrity of database linkages between bookings, tenants, units, and payments. Reproducibility across repeated runs was included as an explicit check because a calculation that produces the correct result once but an inconsistent result on a subsequent run with identical input, due to a timing issue or an unaccounted-for edge case, would be no more trustworthy in practice than one that was simply incorrect, and such inconsistencies are precisely the kind of defect that manual spreadsheet-based computation, as discussed in Section 1.2, had already shown itself prone to.

4) Security: This criterion measures protection of operational and financial data. The evaluation verified the following: unauthenticated requests to protected endpoints are rejected; expired or forged JWTs are refused; role-based access control restricts users to their permitted scope; parameterized queries prevent SQL injection; HTTPS is enforced; and credentials are stored with industry-standard hashing. Each of these checks targets a distinct and well-documented class of vulnerability common to web-based information systems handling sensitive data. Verifying that role-based access control correctly restricts users to their permitted scope, for instance, was treated as especially important given that the system consolidates financial and personal tenant data that, under the previous manual arrangement, had been dispersed across separate physical records with no equivalent single point of potential exposure; centralizing this data, as discussed in Section 2.3, makes robust access control a precondition for the system's overall benefit rather than an optional enhancement.

5) Efficiency: This criterion measures the reduction in time, manual effort, and error compared with the manual methods. Staff task times were compared against baseline estimates for equivalent manual tasks. The indicators were booking confirmation time, elimination of manual payment reconciliation, prevention of double bookings through automated availability checks, and maintenance resolution time. Reduced manual workload was the primary indicator of improvement. Establishing baseline estimates for the equivalent manual tasks before evaluating the deployed system was necessary to ensure that any observed improvement could be attributed specifically to the system's introduction, rather than asserted only in general terms; this baseline comparison is what allows the efficiency findings reported in Chapter V to be presented as a measured before-and-after difference rather than as an unsubstantiated claim of improvement.

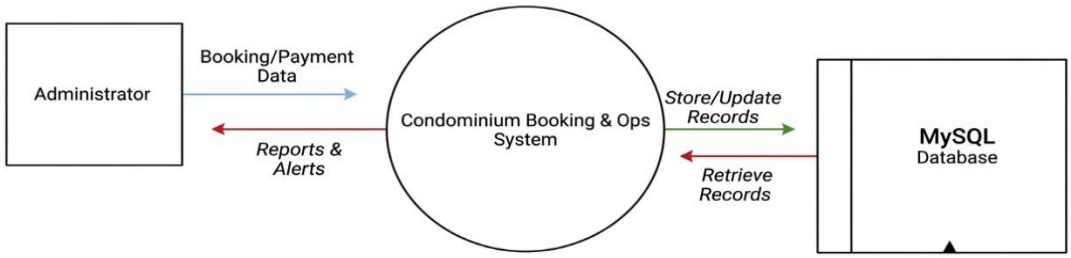
3.8 Supporting Diagrams

The following diagrams document the system's data flow and data structure. They were built to standard notation and served as design references for the development team, academic reviewers, and future researchers.

These diagrams were produced during the system design phase described in Section 3.2 and were treated as living artifacts rather than one-time documentation, in that both were revisited and revised as the iterative development process surfaced requirements that had not been fully apparent during the initial requirements analysis. Presenting them here in their finalized form allows the reader to verify that the implemented system, detailed in Chapter IV, corresponds to the architecture and data model originally specified, and gives future researchers or developers extending this system a concise reference for its underlying structure without needing to reverse-engineer it from the source code directly.

1) Data Flow Diagram (DFD), Level 0: The Level 0 DFD gives the context of the whole system (Fig. 3). The Administrator supplies booking records, payment data, maintenance requests, and scheduling information. The system validates and processes these inputs under its business rules and stores them in the MySQL database. It also retrieves stored records to generate reports, populate dashboards, and return operational summaries to the Administrator. This two-way flow keeps data consistent and available in real time.

The Level 0 DFD was deliberately kept at this high level of abstraction, treating the system as a single process bounded by its external interactions with the Administrator and the database, rather than decomposing it further into its constituent modules at this stage. This context-level view establishes the essential boundary of the system, namely that all operational data originates from and is ultimately returned to the Administrator, with the system itself functioning as the sole validating and processing intermediary. This framing reinforces the centralization principle discussed in Section 2.3: every input the Administrator supplies, whether a new booking, a recorded payment, or a logged maintenance concern, passes through the same business-rule validation before being committed to the single authoritative database, and every report, dashboard figure, or summary the Administrator later retrieves is drawn from that same consistent source rather than reconstructed from separate, potentially divergent records.



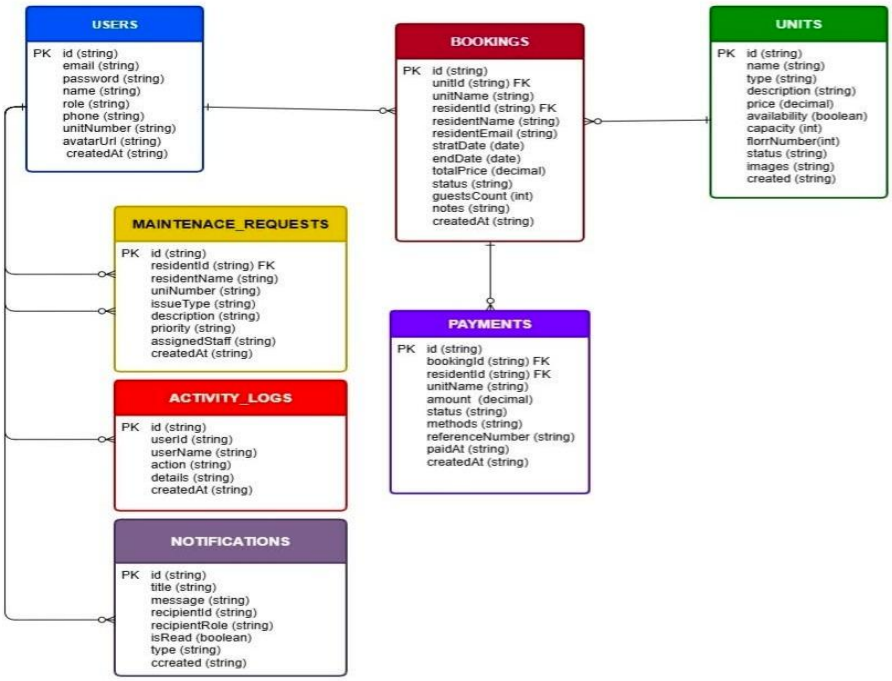
2) Entity Relationship Diagram (ERD): The ERD defines the MySQL schema, organized around five entities (Fig. 4): Users, holding credentials, profile data, and role designations governing access; Units, holding unit type, floor level, monthly rate, and availability status; Bookings, the central transactional entity, linking tenants and units through foreign keys to Users and Units, with check-in and check-out dates, status, total rental amount, and notes; Payments, an auditable ledger of amounts, dates, and payment methods, linked to Bookings by foreign key; and Maintenance, holding issue descriptions, priority, status, and reported and resolved timestamps, linked to specific units.

The one-to-many relationships enforce referential integrity and support cross-module reporting and queries.

The Bookings entity occupies a deliberately central position within this schema, since it is the entity through which a Unit and a User are formally associated over a specific period, and it is this association that the Payments entity, in turn, depends on by linking each financial transaction back to a specific booking rather than to a tenant or unit in isolation. This chained relational structure, Users to Bookings, Units to Bookings, and Bookings to Payments, was a deliberate design decision intended to prevent the kind of orphaned or ambiguous record that had characterized the business's previous manual ledgers, in which a payment entry was not always clearly traceable to the specific reservation it was intended to settle. The Maintenance entity, by contrast, is linked directly to Units rather than to Bookings, reflecting the operational reality that a maintenance concern pertains to the physical condition of a unit and may arise independently of any active reservation, such as during a vacancy period between tenants.

Enforcing these relationships as foreign key constraints at the database level, rather than relying on the application layer alone to maintain consistency between related records, provides a safeguard against data corruption that persists even if a future code change inadvertently omits a validation step elsewhere in the system. This structural integrity, combined with the normalization principles discussed in Section 4.6, is what allows the reporting and analytics functions described throughout Chapter IV to query across multiple entities, such as computing total

revenue per unit by joining Units, Bookings, and Payments, with confidence that the underlying relationships are consistent and complete.



3.9 Project Plan and Sprint Schedule

Because the system was developed under the Agile model described in Section 3.2, the work was organized into time-boxed sprints, each of which produced a working increment of the system that could be reviewed by the stakeholders of Gales Urban Haven [2]. The sequence of the sprints was determined by module dependency: components on which other modules rely, such as authentication and the database schema, were scheduled first, and components that consume data produced by other modules, such as revenue reporting and the calendar, were scheduled later. Table II summarizes the plan.

TABLE II. SPRINT PLAN AND DELIVERABLES

Sprint	Focus	Principal Deliverables
0	Planning and requirements	Stakeholder interviews, audit of manual workflows, requirements

		backlog, selection of the technology stack
1	Foundation	MySQL schema, user accounts, secure login, role-based access control, project repository and coding conventions
2	Unit management	Unit inventory, unit attributes (type, floor, rate), availability status, administrator interface for unit records
3	Booking management	Reservation creation, availability checking, overlap (double-booking) prevention, booking status transitions
4	Payment tracking	Down payment and installment recording, balance computation, payment status classification, transaction history
5	Calendar and scheduling	Unified calendar of reservations, check-ins, check-outs, and maintenance; daily, weekly, and monthly views
6	Maintenance monitoring	Request logging, priority levels, task assignment, status tracking, calendar integration
7	Reporting and notifications	Revenue summaries, occupancy analytics, activity logs, notifications and alerts
8	Testing and refinement	Unit, integration, system, and user acceptance testing; defect correction; performance review
9	Deployment and handover	Production configuration, schema migration, user credentials, orientation of administrators, user guide

Each sprint began with a planning session in which the backlog items for the cycle were selected and clarified, continued with development and daily self-checks by the team members responsible for the items, and ended with a review in which the working increment was demonstrated to a representative of the business and a short retrospective in which the team examined what had worked and what should be changed [14]. Items that the stakeholders rejected or revised during a review were returned to the backlog and reprioritized, which is the mechanism by which the evolving requirements discussed in Section 3.2 were absorbed without disrupting the overall schedule.

The division of responsibilities among the five researchers followed the principle that every module should have one member primarily accountable for it and a second member who reviewed it. The accountable member was responsible for the design and implementation of the module and for the correction of defects found in it, while the reviewing member examined the code against the interface and schema specifications and verified the module against its acceptance criteria. This arrangement spread knowledge of the system across the team, reduced the dependence of the project on any single member, and supported the code review practice described in Section 4.3.

3.10 Requirements Specification Summary

The requirements gathered during the first sprint were consolidated into a specification that served as the baseline for design and testing. Following common practice in software engineering, the requirements were separated into functional requirements, which describe what the system must do, and non-functional requirements, which describe the qualities the system must exhibit while doing it [1]. Each requirement was assigned an identifier so that it could be traced to a design element, an implemented feature, and a test case.

Functional Requirements

TABLE III. SUMMARY OF FUNCTIONAL REQUIREMENTS

ID	Requirement
FR-01	The system shall authenticate users with a username and password and direct each user to the portal that corresponds to the assigned role.
FR-02	The system shall allow administrators to create, view, update, and deactivate unit records, including type, floor level, monthly rate, and availability status.
FR-03	The system shall allow a reservation to be created for a unit only if no confirmed or pending reservation overlaps the requested dates.
FR-04	The system shall maintain the status of each reservation (Pending, Confirmed, Cancelled, Completed) and record every status change with its date and the responsible user.
FR-05	The system shall record down payments and installments against a specific reservation and compute the remaining balance automatically.
FR-06	The system shall classify accounts as Paid, Pending, or Overdue according to the amounts due and the dates recorded.
FR-07	The system shall generate revenue summaries and occupancy figures for a selected period without manual computation.
FR-08	The system shall display reservations, check-ins, check-outs, and scheduled maintenance in a single calendar with daily, weekly, and monthly views.
FR-09	The system shall allow maintenance requests to be logged, prioritized, assigned to personnel, and tracked until resolution.

FR-10	The system shall issue notifications to users concerning confirmations, payment reminders, and the status of maintenance requests.
FR-11	The system shall record an activity log of significant user actions, including the user, the action, the affected record, and the time.
FR-12	The system shall restrict each function to the roles authorized to use it.

Non-Functional Requirements

The non-functional requirements were organized around the quality characteristics of the ISO/IEC 25010 product quality model, which defines functional suitability, performance efficiency, compatibility, usability, reliability, security, maintainability, and portability as the principal qualities of a software product [13]. Those most relevant to the business were usability, so that staff with limited technical experience could operate the system after brief orientation; security, so that tenant and financial data would be protected; reliability, so that records would not be lost or corrupted; and performance efficiency, so that dashboard and report queries would complete quickly enough not to interrupt administrative work. Maintainability was also treated as a requirement because the system is expected to be adjusted as the business changes, and a well-structured codebase with typed interfaces reduces the cost of such adjustments [1].

3.11 Ethical and Data Privacy Considerations

The system stores personally identifiable information about tenants and guests and detailed financial records of the business, and the study therefore observed ethical and legal obligations concerning the collection and handling of such data. In the Philippines, the processing of personal information is governed by Republic Act No. 10173, the Data Privacy Act of 2012, which requires that personal data be collected for a declared and legitimate purpose, processed fairly and lawfully, kept accurate, retained only as long as necessary, and protected by appropriate organizational, physical, and technical security measures [12].

Several practices were adopted in line with these principles. First, participation of administrators, business owners, and other users in the interviews, consultations, and evaluation sessions was voluntary, and the purpose of the study and the way their responses would be used were explained to them beforehand. Second, the data used during development and testing were limited to what was necessary for the purpose; wherever possible, test records were fictitious or anonymized so that the personal data of actual tenants were not exposed to the development team unnecessarily. Third, the evaluation responses were reported in aggregate form so that individual respondents could not be identified. Fourth, the system itself incorporates technical safeguards, namely hashed credential storage, role-based access control, validation of all inputs, and an activity log that supports accountability, as discussed in Chapter

IV. Finally, the business's own records were used only with the consent of its owners and only for the purposes of the study.

The researchers also observed the ordinary standards of academic integrity: sources used in the review of literature and in the technical documentation were acknowledged through the IEEE citation style, the open-source technologies used in the system were employed in accordance with their published licenses, and the results of the evaluation are reported as they were obtained rather than adjusted to favor the system.

3.12 Risk Management

A developmental project of this kind is exposed to technical, organizational, and schedule risks, and the researchers identified the principal ones at the planning stage so that mitigation measures could be in place before they materialized [1]. Table IV lists the risks that were considered most significant and the measures taken to reduce them.

TABLE IV. RISK REGISTER AND MITIGATION MEASURES

Risk	Mitigation Measure
Requirements change or are discovered late	Use of short sprints, frequent demonstrations of working increments, and a continuously prioritized backlog (Section 3.2).
Loss of data during development or deployment	Version control of all source code; scheduled database backups; verification of every schema migration before it was applied to production.
Security weakness in authentication or access control	Authentication and role-based access control built in the first sprints; hashed credentials; input validation at the server on every request.
Double bookings caused by simultaneous requests	Overlap checking performed on the server within a database transaction rather than in the browser alone.
Low user adoption by staff with limited technical experience	User-centered interface design, orientation sessions, a user guide, and collection of usability feedback during acceptance testing.
Schedule slippage because of academic workload	Module-based task allocation among five members, a reviewer for every module, and prioritization of core modules ahead of optional features.
Differences between development and production environments	Explicit environment validation checks after deployment and documentation of configuration variables.
Scope growth beyond the capacity of	Written delimitations (Section 1.4.2) and a rule that new requests are

the team	placed in the backlog for later consideration.

Risk management was not a one-time exercise. The register was reviewed during the retrospective of each sprint, and when a risk occurred or a new one was identified, the corresponding measure was adjusted. This practice is consistent with the Agile principle that a team should regularly reflect on how to become more effective and adjust its behavior accordingly [2].

3.13 Chapter Summary

This chapter described the procedures by which the system was planned, designed, developed, and evaluated. The study adopted a developmental research design and the Agile Software Development Model, organized the work into time-boxed sprints, and built the system on a three-tier architecture using React, TypeScript, Tailwind CSS, and Vite on the client, Node.js and Express on the server, and MySQL as the single relational database. Requirements were gathered through consultation with the stakeholders and an audit of existing workflows and were consolidated into a specification of functional and non-functional requirements. The chapter also set out the testing procedures, the five evaluation criteria of functionality, usability, accuracy, security, and efficiency, the ethical and data privacy safeguards observed, and the risks identified together with their mitigation. Chapter IV presents the system that resulted from this methodology.

REFERENCES

[1] I. Sommerville, Software Engineering, 10th ed. Harlow, U.K.: Pearson, 2016.

[2] K. Beck et al., "Manifesto for Agile Software Development," 2001. [Online]. Available: https://agilemanifesto.org

[3] Meta Platforms, Inc., "React Documentation." [Online]. Available: https://react.dev

[4] Microsoft, "TypeScript Documentation." [Online]. Available: https://www.typescriptlang.org/docs

[5] Tailwind Labs, "Tailwind CSS Documentation." [Online]. Available: https://tailwindcss.com/docs

[6] Vite Contributors, "Vite Documentation." [Online]. Available: https://vite.dev

[7] OpenJS Foundation, "Node.js Documentation." [Online]. Available: https://nodejs.org/docs

[8] OpenJS Foundation, "Express Documentation." [Online]. Available: https://expressjs.com

[9] C. McDonnell, "Zod Documentation." [Online]. Available: https://zod.dev

[10] E. Wallace, "esbuild Documentation." [Online]. Available: https://esbuild.github.io

[11] Oracle Corporation, "MySQL 8.0 Reference Manual." [Online]. Available: https://dev.mysql.com/doc

[12] Data Privacy Act of 2012, Rep. Act No. 10173, Philippines, Aug. 15, 2012.

[13] Systems and Software Engineering -- Systems and Software Quality Requirements and Evaluation (SQuaRE) -- System and Software Quality Models, ISO/IEC 25010:2011, 2011.

[14] K. Schwaber and J. Sutherland, The Scrum Guide: The Definitive Guide to Scrum: The Rules of the Game. Scrum.org, 2020.

CHAPTER IV

RESULTS AND DISCUSSION

4.1 Introduction

This chapter presents the results of the study, namely the developed One Spatial Iloilo: Web-Based Condominium Booking and Operations Management System, and discusses how the system answers the problems and objectives set out in Chapter I. In a developmental study, the principal result is the artifact itself, so the chapter describes the system in terms of its modules, the lifecycle through which it was built, its architecture, the hardware and software that support it, and the database that underlies it [6]. It then describes how the functional and non-functional requirements were implemented, the business rules and security mechanisms the system enforces, the way in which the system was tested, and how the principal workflows appear to the people who use them.

The discussion is organized so that each part can be checked against the earlier chapters. The problems stated in Section 1.2 are the baseline, the objectives in Section 1.3 are the targets, the methodology in Chapter III is the procedure, and this chapter reports what that procedure produced. Where a feature of the system is described, the operational deficiency that it addresses is identified, so that the reader can see that the system was designed to solve documented problems rather than to display technical capability for its own sake.

4.2 Overview of the Developed System

The system developed for this study, titled One Spatial Iloilo: Web-Based Condominium Booking and Operations Management System, is a centralized web-based platform engineered to streamline booking, payment processing, and administrative workflows for One Spatial Iloilo [1]. Prior to system deployment, core operational tasks, such as unit reservations, payment tracking, maintenance requests, and occupancy monitoring, relied on fragmented manual processes and disconnected digital spreadsheets. This operational setup resulted in record consolidation delays, misplaced physical reservation forms, and administrative overhead due to manual cross-referencing across separate ledgers [2].

These deficiencies, already detailed from the business's own perspective in Section 1.2 and examined against the broader literature in Chapter II, are presented here specifically as the baseline condition against which the developed

system's architecture should be understood. Each design decision described in this chapter traces back to one or more of these documented operational gaps, rather than representing a generic or arbitrary feature set; the system was not built to demonstrate technical capability for its own sake, but to directly close the specific gaps identified during the requirements-gathering activities described in Section 3.2.

To resolve these inefficiencies, the developed architecture integrates separate Resident and Administrator portals over a single unified relational database [3]. Data submitted by a user automatically propagates across administrative interfaces in real time, mitigating duplicate entries and maintaining system-wide data integrity.

Separating the system into two distinct portals, while still grounding both in the same underlying database, was a deliberate structural choice rather than a cosmetic division of the interface. Residents and administrators have fundamentally different operational needs and levels of system access, and presenting both groups with an undifferentiated interface would have required either exposing administrative functions inappropriately to residents or burying administrator-specific tools behind unnecessary navigation. By contrast, routing each authenticated user to a portal scoped to their role, while keeping both portals backed by the same relational schema described in Section 4.6, preserves the centralization principle established in Chapter III without sacrificing the clarity and simplicity each user group requires of their respective interface.

Core System Modules and Functional Architecture

Access to the platform is managed via an authentication interface hosted on a public-facing landing page displaying property details and amenities. Upon successful authentication, users are routed to their designated portal based on role permissions:

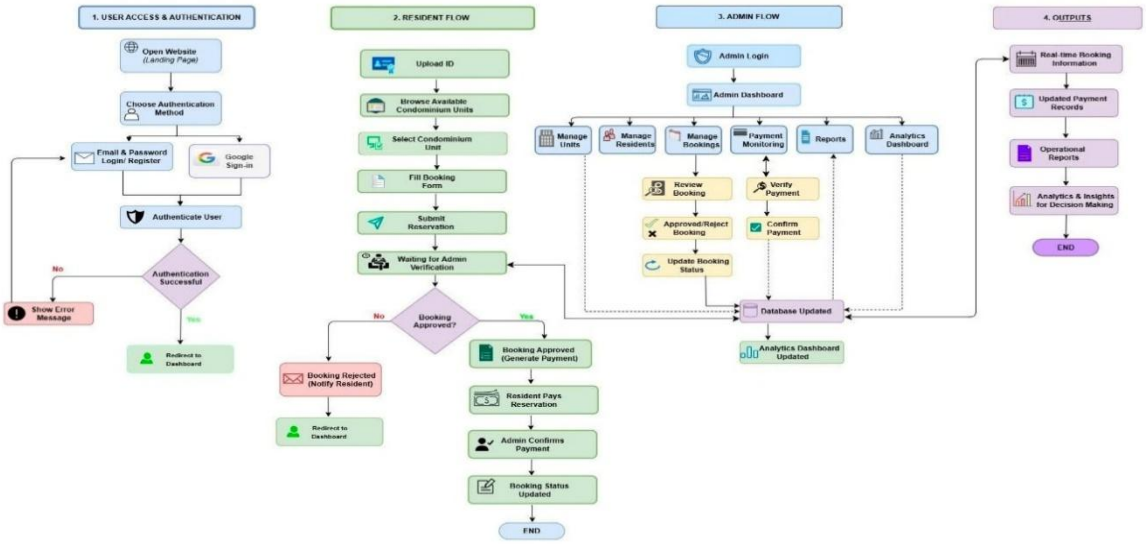
1. Resident Portal: Functions as the primary client interface for tenants and unit owners. The main dashboard displays real-time summaries of active bookings, historical payment ledgers, and outstanding balances. Additional functionalities enable users to: submit and track unit or facility maintenance requests; review transaction histories and profile settings; and receive automated operational notifications. This portal was designed around the self-service principle discussed in Section 2.3, under the premise that giving residents direct, continuous visibility into their own booking and payment status reduces the volume of routine inquiries that would otherwise require a staff member's direct intervention, freeing administrative capacity for the higher-value tasks identified in Section 1.5.
2. Administrator Portal: Designed for property management personnel, the admin interface aggregates operational metrics including total bookings, unit availability, total revenue, and live occupancy rates. Management capability is partitioned into dedicated modules: Booking Management Module, which controls reservation state transitions (e.g., Pending, Confirmed, Cancelled, Completed); Unit Management Module, which maintains unit inventory, metadata (floor, unit type, pricing structure), and availability statuses; Payment Tracking Module, which tracks financial transactions and classifies accounts as Paid, Pending, or Overdue; Analytics Dashboard, which generates aggregate reports on revenue trends, booking

volumes, and occupancy distribution over selected time horizons; Resident Records Module, which manages tenant profiles and unit assignment mapping; Maintenance Requests Module, which facilitates the logging, assignment, and status tracking of reported facility issues; and Notifications and Settings Module, which oversees system-wide broadcast messages and security configurations.

Although presented here as seven distinct modules for clarity, each draw from and writes to the same centralized database described in Section 4.6, so that, for example, a status change recorded in the Booking Management Module is immediately reflected in the Analytics Dashboard's occupancy figures without requiring a separate synchronization step. This modular-yet-unified structure mirrors the architectural pattern established in Section 3.3, in which every module follows the same request-validate-respond flow across the presentation, application, and data layers, allowing administrators to move between modules while relying on a single, internally consistent view of the business's operations.

By centralizing data entry, the system eliminates redundant record-keeping, establishes a single source of truth, and optimizes operational efficiency across property management workflows [4]. This outcome directly answers the revenue computation and data redundancy problems identified in Section 1.2, in which figures derived from disconnected spreadsheets were vulnerable to version-control errors and silent discrepancies between files maintained by different staff members; under the architecture described here, there is only one authoritative record of any given transaction, from which every report, dashboard, and portal view is consistently derived.

Figure 4.1 illustrates the overall workflow of the developed system.



4.3 System Development Methodology

The development of the system followed a structured, phased software development methodology. The lifecycle progressed sequentially through requirements gathering, planning, design, implementation, testing, deployment, and ongoing maintenance [5]. Adopting a phase-gated approach ensured that core structural dependencies, such as database schema design and authentication protocols, were fully validated before subsequent engineering iterations [6].

This phase-gated structure, while distinct in emphasis from the iteration-centered Agile methodology described in Chapter III, was not treated as a contradictory or competing approach. Rather, it represents the same development effort viewed at a different level of granularity: the phase-gated sequence describes the overall progression of the project from initial requirements to deployed system, while the Agile sprints described in Section 3.2 describe how work was actually organized and delivered within that broader progression. Section 4.3.8 and 4.3.9, below, make this relationship explicit by situating the Agile framework within the same lifecycle described here.

1) Requirements Gathering and Analysis

Requirements were elicited through stakeholder interviews, informal discussions with residents and administrative staff of One Spatial Iloilo, and an audit of legacy manual workflows. The gathered data was classified into two primary domains: Functional Requirements, which defined operational logic including account authentication, unit reservation processing, payment recording, maintenance ticket logging, and real-time report generation (e.g., occupancy and revenue analytics); and Non-Functional Requirements, which defined quality constraints including user interface (UI) accessibility for non-technical users, system latency targets for dashboard queries, and data protection mechanisms for personally identifiable information (PII) and financial records [7].

These specification criteria were consolidated into a formal Software Requirements Specification (SRS) document to serve as the baseline for system architecture planning. Distinguishing functional from non-functional requirements at this stage, rather than treating all stakeholder input as a single undifferentiated list, ensured that quality attributes such as latency targets and data protection were deliberately designed into the system's architecture from the outset, rather than being addressed reactively only if a performance or security issue surfaced later during testing.

2) System Planning

The system architecture was decomposed into discrete functional modules mapped directly to stakeholder requirements. Execution was scheduled based on module dependency: core identity and authentication services were prioritized prior to higher-level transactional features.

Technological stack selection (front-end interfaces, server-side application logic, and relational database management system) was evaluated against implementation constraints and delivery milestones. Operational risks, such as delayed baseline data collection or scope expansion, were identified, evaluated, and assigned mitigation protocols early in the planning lifecycle. Prioritizing authentication and identity services ahead of transactional features was a direct consequence of module dependency analysis: every other module, from booking management to payment tracking, depends on a verified, role-identified user session to enforce the access control described in Section 4.4, so building this foundation first avoided the need to retrofit authentication into modules that had already been built assuming an unauthenticated context.

3) System Design

The system design phase translated specifications into technical blueprints: User Interface Design, in which wireframes and high-fidelity mockups were generated for all application views (landing page, authentication portals, resident dashboard, admin management modules) to establish a unified visual taxonomy and navigation hierarchy; Data Architecture, in which an Entity-Relationship (ER) model was constructed to define core domain entities (Users, Units, Bookings, Payments, and Maintenance Requests) and their relational cardinality (e.g., mapping a booking record to a unique resident entity and payment ledger entry); and System Architecture, in which the multi-tier architecture detailing the interactions between the presentation, application logic, and data storage layers was formalized [8].

Producing these three blueprints in parallel, rather than sequentially, allowed inconsistencies between them to be identified and resolved before implementation began; for instance, a UI mockup that implied a data relationship not yet captured in the ER model could be caught and reconciled at the design stage, rather than discovered only once the corresponding interface code had already been written against an incomplete schema.

4) Software Development

Implementation proceeded iteratively by module in accordance with the project schedule. Each module was constructed through a bottom-up workflow: UI layout development, application logic integration, and database binding. The primary authentication module was developed first to establish Role-Based Access Control (RBAC) across resident and administrator contexts. Subsequent client-side features (dashboard, booking creation, payment tracking, maintenance logs) and administrative control interfaces (unit management, payment verification, analytics engine) were systematically engineered. Source code versioning was maintained throughout the phase, with code reviews conducted against the baseline interface specifications and data schemas. Conducting code reviews against the originally specified interface and schema documents, rather than evaluating code purely on whether it executed correctly, helped ensure that implementation decisions made under time pressure did not silently drift from the architecture established during the design phase.

5) System Verification and Testing

Functional validity and stability were verified through a multi-tier testing framework: Unit Testing, which validated isolated algorithmic components, such as automated balance calculation functions and dynamic filtering queries; Integration Testing, which verified inter-module communication, ensuring that client-side state changes (e.g., a submitted booking request) immediately updated administrative database records; System Testing, which executed end-to-end user journeys across complete operational lifecycles (e.g., user registration, unit reservation, payment submission, administrative verification); and User Acceptance Testing (UAT), which was conducted with a sample cohort of residents and staff to identify usability bottlenecks and workflow frictions under simulated operational conditions. Identified defects were logged, corrected, and re-evaluated prior to deployment. This verification framework mirrors, and was conducted in coordination with, the three-phase testing procedure detailed in Section 3.6, with the end-to-end user journeys exercised here corresponding directly to the realistic task scenarios described under User Acceptance Testing in that section.

6) Deployment

Following verification, the application was migrated from local development environments to production cloud infrastructure. Production deployment steps included: initializing and configuring the production database server; executing database schema migrations and seeding initial system configurations; deploying front-end assets and server-side runtime services; and performing environment validation checks to eliminate discrepancies between local and production runtime behavior. Upon verification, access credentials were issued to participating residents and staff, marking formal system handover. Performing explicit environment validation checks after migration, rather than assuming that a system verified in a local or staging environment would behave identically in production, was intended to catch configuration-specific issues, such as differing environment variables or network settings, before residents and staff began relying on the system for actual operations.

7) Maintenance

Post-deployment operations focus on continuous system maintenance, including application performance monitoring, bug remediation, security patch distribution, and framework updates. Furthermore, incremental refinements based on post-launch administrative feedback (e.g., report formatting adjustments and table filter optimizations) are systematically implemented to ensure ongoing operational reliability [9]. Treating maintenance as a continuous, ongoing phase rather than a discrete endpoint reflects the understanding that a deployed system's requirements do not remain static; as Gales Urban Haven's own operations evolve, the system is expected to be incrementally adjusted in response, consistent with the scalability objective discussed in Section 1.3.1.

8) Agile Development Framework

In addition to the phase-gated engineering pipeline, the overall development cycle can be framed within an Agile development paradigm. Under this framework, requirements are decomposed into iterative sprint cycles, enabling

continuous system delivery, feedback integration, and continuous feature refinement based on empirical user feedback.

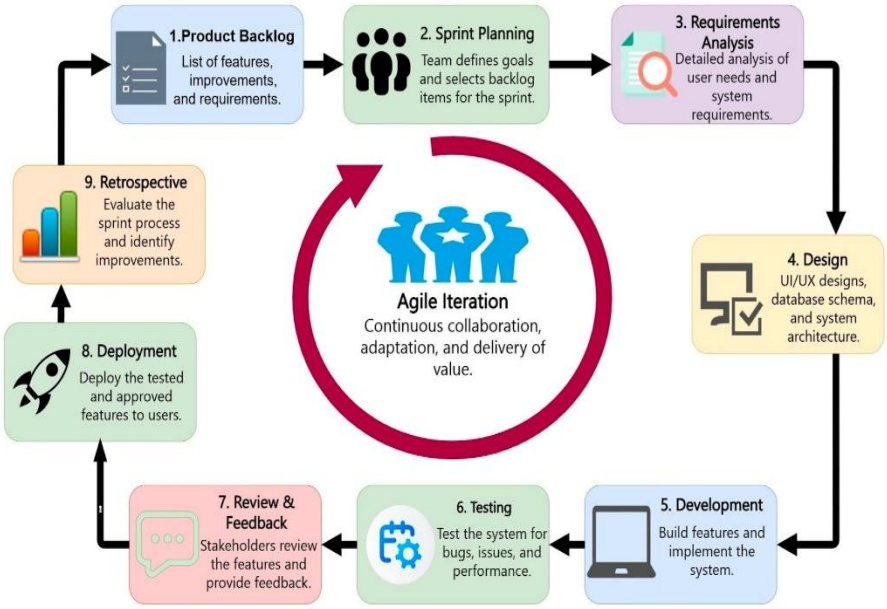


Figure 4.2 presents the Agile Software Development Life Cycle adopted in the study.

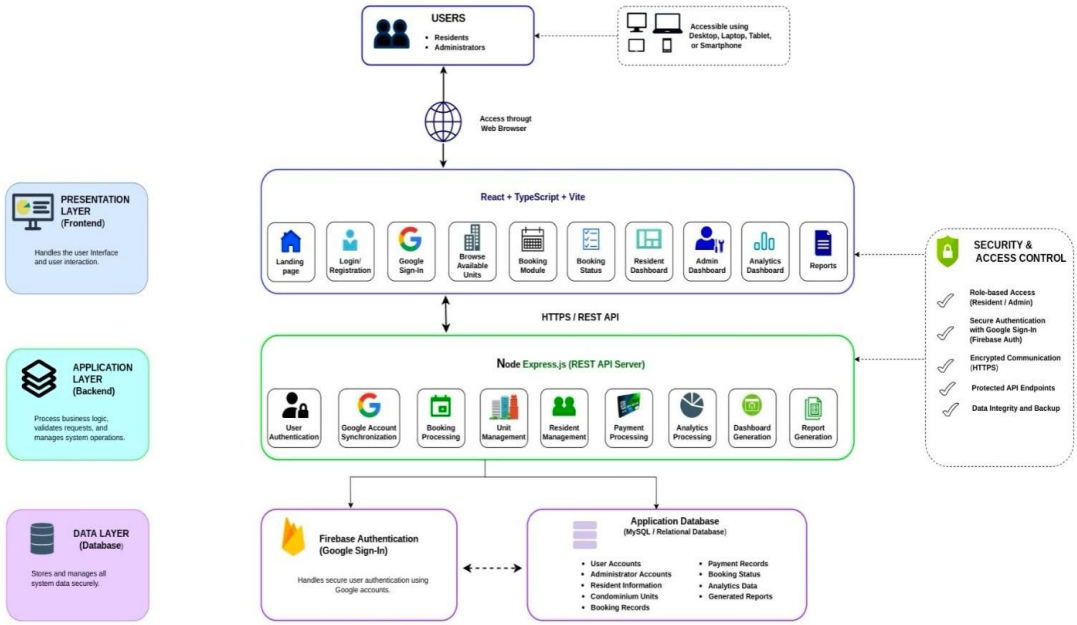
9) Agile Process Framework

Under the Agile development approach, the engineering process was organized into iterative sprint cycles comprising requirements specification, sprint planning, incremental feature development, continuous testing, and stakeholder feedback integration [10]. This iterative paradigm is particularly suited for condominium management systems, as it permits continuous refinement of transactional workflows, such as reservation scheduling, payment verification, and maintenance tracking, based on practical operational feedback [11]. Framing the phase-gated lifecycle described above through this Agile lens clarifies that phases such as design, development, and testing were not executed once each in strict sequence across the whole project, but were instead revisited within each sprint cycle at a smaller scale, applied first to a limited module such as authentication, then again to booking management, and so on, consistent with the iterative, module-by-module build order described in Section 3.2.

4.4 System Architecture

The system was engineered around a traditional Three-Tier Architecture, decoupling the presentation, application logic, and data storage layers [12]. This architectural separation ensures modular maintainability: updates to presentation layouts operate independently of business logic, while database schema modifications do not disrupt core processing routines.

This separation of concerns directly operationalizes the client-server architecture rationale already established in Section 3.3, and is restated here at the implementation level to describe how the presentation, application logic, and data storage layers were concretely realized within the deployed system, including the specific responsibilities assigned to each layer and the modules that compose them, detailed in the subsections that follow.



1) Presentation Layer

The presentation layer serves as the primary client-facing tier, rendering data obtained from lower application tiers into responsive graphical user interfaces (GUIs). It captures user inputs (e.g., booking requests or profile modifications) and transmits them to the application layer for validation and execution. Interfaces were implemented

with responsive web design patterns to ensure cross-device compatibility across desktop and mobile browsers. Key presentation modules include: Landing Page, which displays property overview and public amenities; Authentication Page, which handles user login and identity submission; Resident Dashboard, which provides client-specific financial and booking overviews; Administrator Dashboard, which aggregates macro-level operational and performance metrics; Booking Management Interface, which facilitates reservation processing; Unit Management Interface, which displays inventory and rate configurations; Payment Tracking Interface, which logs financial records and transaction statuses; Analytics Dashboard, which visualizes revenue trends and occupancy statistics; Resident Profile Interface, which enables account information management; Maintenance Requests Interface, which facilitates ticket submission and monitoring; and Notification Interface, which displays system alerts and administrative broadcasts.

2) Application Layer

Operating between the presentation and data tiers, the application layer encapsulates core business rules, computation engines, and transactional workflows [13]. It enforces Role-Based Access Control (RBAC), validates operational constraints (e.g., preventing double-booking of active units), calculates dynamic variables (e.g., outstanding balances and occupancy percentages), and coordinates inter-tier communications.

Core functionalities include: Authentication and Session Management Engine, which handles identity verification and access permissions; Booking Execution Logic, which validates unit availability and processes reservation states; Payment Processing Engine, which handles transaction validation and balance calculations; Maintenance Workflow Controller, which manages ticket creation, status progression, and task assignment; Notification Engine, which generates and routes real-time system alerts; Analytics and Reporting Engine, which aggregates transactional data into statistical metrics; and Profile Management Logic, which processes account updates and updates user attributes.

3) Data Layer

The data layer manages persistent storage and retrieval of relational system records. Constructed in accordance with the Entity-Relationship (ER) model established during the system design phase, entity cardinalities and relational constraints are enforced at the database schema level to uphold transactional integrity (ACID properties) [14].

Stored entities encompass: User Account Records (system credentials, roles, and personal profiles); Unit Records (physical metadata, pricing tiers, and real-time availability states); Booking Records (reservation details, time spans, and status identifiers); Payment Records (transaction ledgers, payment receipts, and balance statuses); Maintenance Request Records (facility issue logs, repair statuses, and timestamps); and Notification Records (system alerts and user messaging archives).

To illustrate multi-tier interactions: when a user executes a booking through the Presentation Layer, the request is forwarded to the Application Layer to verify unit availability and calculate applicable fees. Upon successful validation, the Data Layer writes the transaction record and updates unit status flags. Finally, the Presentation Layer dynamically reflects the updated operational state across both Resident and Administrator portals without requiring manual synchronization [15].

4.5 Hardware and Software Specifications

To ensure optimized system throughput, deployment stability, and client accessibility, system resource requirements were defined for development, production hosting, and end-user execution contexts.

Hardware Requirements: Client-side hardware criteria mandate standard computing or mobile hardware capable of running modern standard web browsers. Server-side hosting hardware requires sufficient processing power and memory allocation to process simultaneous HTTP/HTTPS requests, run database query engines, and generate real-time analytics graphs without latency spikes.

Software Requirements: The software pipeline comprises specialized frontend and backend frameworks, a Relational Database Management System (RDBMS), authentication libraries, package management utilities, version control infrastructure, and deployment runtime environments.

4.5.1 Hardware Requirements

The minimum and recommended hardware configurations required to support development, administrative operations, and client access for the system are outlined in Table 4.1. Meeting or exceeding these specifications ensures optimal browser rendering, efficient query execution, low-latency dashboard visualization, and robust performance during concurrent user sessions [16].

TABLE 4.1 HARDWARE REQUIREMENTS

Component	Minimum Specification	Recommended Specification
Processor	Intel Core i3 (8th Generation) or AMD Ryzen 3	Intel Core i5 (10th Generation) or higher / AMD Ryzen 5 or higher
Memory (RAM)	4 GB	8 GB or higher
Storage	256 GB SSD	512 GB SSD or higher

Monitor	1366 x 768 Resolution	Full HD (1920 x 1080) Display
Internet Connection	10 Mbps	50 Mbps or higher
Mobile Device	Android 10 or iOS 14	Android 13 or iOS 17 or later

Operating the system on or above the recommended hardware configuration guarantees sufficient computational throughput to handle resource-intensive web client operations, such as multi-chart data visualization, interactive payment ledger queries, and real-time report compilation [17]. Furthermore, higher network bandwidth and memory capacity minimize page load latency and optimize application responsiveness across desktop and mobile execution environments.

4.5.2 Software Requirements

The software stack and development utilities used to build, test, and execute the system are detailed in Table 4.2. Software choices were evaluated based on framework stability, modularity, active community support, and performance suitability for decoupled modern web architectures [18]

TABLE 4.2 SOFTWARE REQUIREMENTS

Component	**Purpose**
Windows 10/Windows 11	Operating System
Visual Studio Code	Source Code Editor
Node.js	JavaScript runtime environment
npm	Dependency and package management
React	Front-end Development Library
TypeScript	Front-end programming language
Vite	Front-end build tool and Development server
Express.js	Backend development framework
Firebase Authentication	Google sign-in authentication
JSON database	Local data storage

Google Chrome	Web browser for testing and system access
Git	Version control
GitHub	Source code repository

The client interface was engineered using React with TypeScript and built via Vite to maintain high performance, strict type safety, and fast DOM re-rendering [19]. Server-side REST APIs and business logic execution were hosted on a Node.js environment using Express.js. Account identity verification was integrated through Firebase Authentication to enable secure OAuth 2.0 Google Sign-In protocols. System state persistence during initial development iterations was managed via a structured JSON dataset (db.json). Version control, code review pipelines, and source integrity were maintained using Git and hosted on GitHub repositories [20].

4.6 Database Design

The data layer of the application was constructed using a normalized relational database schema comprising seven interconnected tables. The design applies Boyce-Codd Normal Form (BCNF) principles to eliminate data redundancy while maintaining referential integrity across operational transactions [21].

The database entities and their relational mappings are structured as follows:

users Table: Serves as the central entity table storing user accounts, authentication credentials, profile data, and access roles (administrator or resident). The primary key id is referenced as a foreign key across multiple downstream tables.

units Table: Holds property metadata including unit capacity, floor allocation, rental rates, and real-time availability states. It maintains an independent existence prior to reservation assignment.

bookings Table: Establishes a many-to-one relationship (1:N) with both the units table via unitId and the users table via residentId. This binds a specific resident to a unique unit over a defined reservation period.

payments Table: Linked to the bookings table via bookingId (1:N) and the users table via residentId. This structure allows transaction logs to be audited against their underlying reservation entries and resident billing statements.

maintenance_requests Table: Maintains a foreign key link (residentId) to the users table, ensuring facility maintenance issues are logged, assigned, and tracked per resident account.

notifications Table: Supports targeted messaging via recipientId (referencing users.id) and role-based broadcasting via recipientRole (targeting administrator or resident groups).

activity_logs Table: Captures system-wide audit trails tied to userId. To optimize query performance during administrative log auditing, selective denormalization is applied by retaining userName and userRole directly within the log entries, avoiding expensive join operations across high-volume audit tables [22].

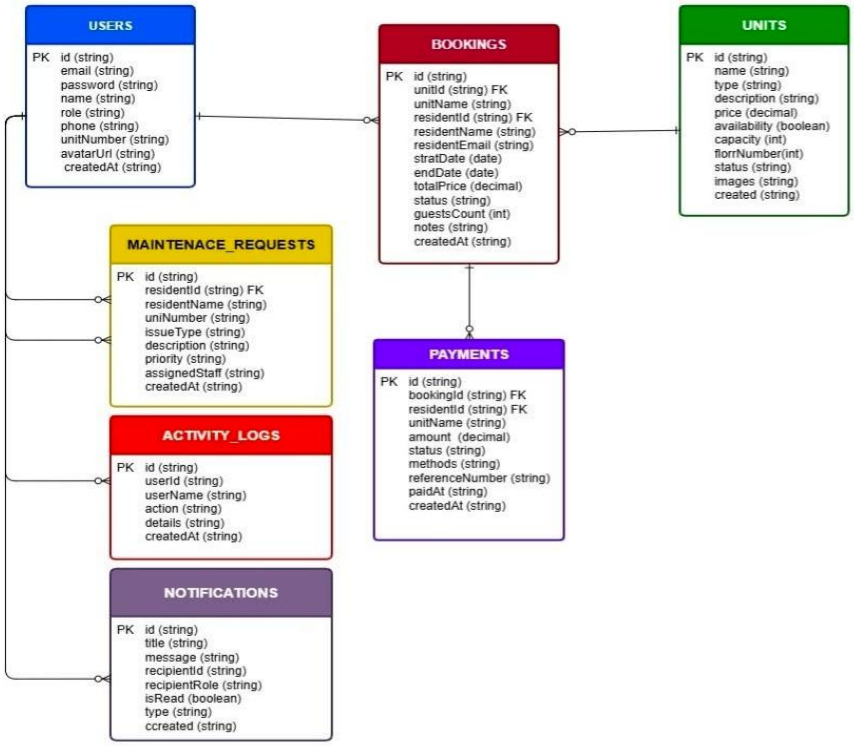


Figure 4.4 presents the Entity Relationship Diagram (ERD) of the system.

The primary entities of the database include: Users, Bookings, Units, Payments, maintenance_requests, Notifications, and Activity_logs.

Users Table

This table stores account credentials and user information.

Field	Description
user_id	Primary Key
Email	User email
password	User password
name	Full name

role	Admin/residents
phone	Contact number
Unit_number	Resident unit number
AvatarUrl	Profile image URL
CreatAt	Date Created

Units Table

Stores condominium unit information and availability.

Field	Description
id	Primary key
name	Unit name
type	Unit type
description	Unit description
price	Rental price
availability	Availability status
capacity	Maximum guest capacity
floorNumber	Floor number
status	Available, Booked, or Maintenance
images	Unit image URLs
createdAt	Date created

Booking Table

Stores resident booking and reservation details.

Field	Description
id	Primary key
unitId	Foreign key to units
unitName	Unit name
residentId	Foreign key to users
residentName	Resident name
residentEmail	Resident email
startDate	Booking start date
endDate	Booking end date
totalPrice	Total booking price
status	Booking status
guestsCount	Number of guests
notes	Additional notes
createdAt	Date created

Payments Table

Stores payment and transaction details.

Field	Description
id	Primary key
bookingId	Foreign key to bookings
residentId	Foreign key to users
residentName	Resident name
unitName	Unit name

amount	Payment amount
status	Payment status
method	Payment method
referenceNumber	Payment reference number
paidAt	Payment date
createdAt	Date created

maintenance_requests Table

Stores resident maintenance requests and their status.

Field	Description
id	Primary key
residentId	Foreign key to users
residentName	Resident name
unitNumber	Unit number
issueType	Type of issue
description	Issue description
priority	Priority level
status	Maintenance status
assignedStaff	Assigned staff
createdAt	Date created

notifications Table

Stores notifications sent to users.

Field	Description
id	Primary key
title	Notification title
message	Notification content
recipientId	Recipient user ID or all
recipientRole	Recipient role
isRead	Read status
type	Notification type
createdAt	Date created

activity_logs Table

Stores records of user activities in the system.

Field	Description
id	Primary key
userId	Foreign key to users
userName	User name
userRole	Admin or resident
action	Performed action
details	Action details
createdAt	Date created

4.7 Implementation of Functional Requirements

The functional requirements specified in Table III of Chapter III were implemented module by module in the sprint sequence described in Section 3.9. This section describes how the principal requirements are realized in the developed system and how each one answers a deficiency of the former manual process.

Authentication and Role-Based Access

Access to the platform begins at the authentication page. A user supplies credentials, which the application layer verifies against the users table, where passwords are stored only in hashed form. On success, the system establishes an authenticated session and routes the user to the Resident Portal or the Administrator Portal according to the role stored in the user record. Every subsequent request carries the session, and the server checks the role against the permissions required by the requested function before any action is carried out, so that the restriction does not depend on what the interface chooses to display [7]. This implementation satisfies the requirement that each person see only what is relevant to their responsibilities, and it replaces the former situation in which any person with access to the spreadsheet or the logbook could read or alter any record.

Unit Management

The Unit Management Module lets administrators maintain the inventory of units with their type, floor level, capacity, monthly rate, and availability status. Because the availability status is stored in the same database consulted by the booking module, a change made here is immediately effective for every other part of the system. A unit taken out of service for repair, for example, is no longer offered for reservation, which prevents the situation in which a tenant is confirmed for a unit that cannot be occupied.

Booking Management

The Booking Management Module is the core transactional component. When a reservation is requested, the server retrieves the existing reservations of the unit and tests whether the requested check-in and check-out dates overlap any reservation in the Pending or Confirmed state. Two date ranges overlap when the requested check-in is earlier than the existing check-out and the requested check-out is later than the existing check-in; a request that satisfies this condition against any active reservation is rejected with an explanatory message and nothing is written. Because the test and the insertion are performed together within a database transaction, two requests arriving at nearly the same moment cannot both succeed for the same unit and dates [14]. A reservation passes through the states Pending, Confirmed, Cancelled, and Completed, and each transition is stamped with the date and the user responsible. This design eliminates the double bookings that were the most visible failure of the manual process and gives the business a reliable history of what happened to every reservation.

Payment Tracking

The Payment Tracking Module records each payment as a separate entry linked to a specific booking, with the amount, date, and method of payment. The outstanding balance of a booking is not typed in by staff but computed by the system as the total rental amount minus the sum of the payments recorded against that booking, so that the

balance is always consistent with the ledger. Accounts are classified as Paid when the balance is zero, Pending when a balance remains and the due date has not passed, and Overdue when a balance remains after the due date. Because payments can be traced to the reservation they settle, the confusion that formerly arose from receipts that could not be matched to a booking is removed, and any figure presented to management can be verified by inspecting the entries that compose it [1].

Revenue Reporting and Analytics

The Analytics Dashboard derives its figures directly from the booking and payment tables. Total revenue, revenue by period, the number of bookings by status, and the occupancy rate over a chosen interval are computed when the page is requested, using queries that join the units, bookings, and payments tables. Since no separate spreadsheet is involved, the version-control and formula errors that formerly affected the business's revenue computations cannot arise, and a report that previously required hours of manual consolidation is available on demand [21].

Calendar-Based Scheduling

The calendar presents reservations, check-ins, check-outs, and scheduled maintenance activities in a single view with daily, weekly, and monthly modes. Its entries are generated from the booking and maintenance records rather than entered separately, and for that reason the calendar cannot disagree with the records it displays. Administrators can see at a glance which units are occupied, which are due for check-out, and which are scheduled for repair, and can plan cleaning and maintenance around reservations.

Maintenance Monitoring

A resident or administrator can log a maintenance request describing the issue and the affected unit. The administrator assigns a priority, assigns the task to personnel, and updates the status as the work proceeds, typically through the progression Pending, In Progress, and Resolved. Each change is recorded with its time, and the reporting resident is able to view the current status of the request. This resolves the former absence of any verifiable record of maintenance concerns and supports the identification of units or components that are the source of repeated problems [14].

Notifications and Activity Logs

The Notification Engine writes messages to the notifications table, addressed either to an individual user or to a role, so that confirmations, payment reminders, maintenance status changes, and administrative announcements are delivered inside the portal. Independently, the application layer writes an entry to the activity_logs table for significant actions, recording who acted, what was done, which record was affected, and when. The activity log is the foundation of accountability in the system, because it allows any disputed transaction to be reconstructed.

4.8 Role-Based Access Control

Role-based access control assigns permissions to roles rather than to individuals, so that the rights of a user follow from the role assigned to the account [7]. Two principal roles are implemented: Administrator and Resident. Table 4.3 summarizes the permissions associated with each role across the system's functions.

TABLE 4.3 ROLE PERMISSION MATRIX

Function	Administrator	Resident
View public landing page and amenities	Yes	Yes
Log in and manage own profile	Yes	Yes
Create, edit, and deactivate unit records	Yes	No
Create and manage reservations for any unit	Yes	No
View own reservations and booking history	Yes	Yes (own only)
Record and verify payments	Yes	No
View own payment ledger and balances	Yes	Yes (own only)
View analytics, revenue, and occupancy reports	Yes	No
Submit a maintenance request	Yes	Yes
Assign, prioritize, and update maintenance requests	Yes	No
View the unified calendar	Yes	Limited to own records
Broadcast notifications and change settings	Yes	No
View the activity log	Yes	No

Enforcement is performed on the server. A request for a function that is not permitted to the role of the requesting user is refused regardless of how the request was produced, which is the property that distinguishes access control from mere interface design. The restriction of resident access to their own records is implemented by filtering queries on the identifier of the authenticated user, so a resident cannot retrieve another resident's bookings or payments by altering a value in the address or the request body.

4.9 Business Rules and Data Validation

The business rules of Gales Urban Haven were translated into validation logic in the application layer so that the rules are applied uniformly regardless of who enters the data. Validation is performed on the server using schema-based validation, and the same constraints are mirrored in the user interface to provide immediate guidance to the user [9]. Table 4.4 lists the principal rules.

TABLE 4.4 PRINCIPAL BUSINESS RULES ENFORCED BY THE SYSTEM

Rule	Enforcement
A unit cannot have two overlapping active reservations.	Server-side overlap check inside a transaction before a reservation is saved.
The check-out date must be later than the check-in date.	Schema validation of the submitted dates; invalid requests are rejected.
A reservation can be made only for a unit whose status is available.	Unit status verified in the booking logic.
A payment must be linked to an existing booking and must be greater than zero.	Foreign key constraint and numeric validation.
The recorded payments of a booking may not exceed the total amount due.	Balance check before a payment is accepted.
Account status is derived from balance and due date.	Status computed by the payment engine rather than typed by staff.
Only administrators may change reservation, payment, or unit records.	Role check on the server for every write operation.
Each significant action must be traceable to a user.	Activity log entry written with the authenticated user's identity.
Email and username must be unique.	Unique constraints in the users table.

Placing rules in the application layer and constraints in the database layer together provide two lines of defense. The application layer produces meaningful messages for the user, while the database constraints guarantee that no code path, including one introduced by a future change, can store data that violates the fundamental integrity of the records [14], [22].

4.10 Security Implementation

Security was treated as a design requirement from the first sprint rather than an addition at the end of development, in line with the guidance that security controls are most effective and least costly when they are part of the original design [6]. The following mechanisms were implemented.

Credential protection. Passwords are never stored in readable form. They are transformed with a salted, one-way hashing function before they are written to the database, so that even a person with access to the stored data cannot read the original passwords [25]. Authentication uses the comparison of hashes, and failed attempts can be examined in the activity log.

Session management. After successful authentication the server issues a session identifier that accompanies each request. Sessions expire after a period of inactivity, which limits the exposure created by a workstation left unattended.

Access control. Role checks are made on the server for each protected route, as described in Section 4.8, and the principle of least privilege is followed, in that each role is granted only the permissions its responsibilities require.

Input validation and injection prevention. All data entering the server are validated for type, format, and range before use, and database queries are constructed with parameterized statements instead of by concatenating user input into query text, which removes the main route for SQL injection attacks [26]. Output rendered in the browser is handled by a framework that escapes content by default, which reduces the exposure to cross-site scripting.

Transport and configuration. The production deployment is intended to be served over HTTPS so that credentials and personal data are encrypted in transit. Secrets such as database credentials and signing keys are kept in environment variables outside the source code and are excluded from version control.

Accountability and privacy. The activity log records significant actions with the user and the time, and access to the log is restricted to administrators. Personal data are limited to what the business needs for its operations, which is consistent with the data privacy principles of proportionality and purpose limitation under Republic Act No. 10173 [24].

4.11 Testing Framework and Test Cases

The system was tested at four levels, as described in Sections 3.6 and 4.3: unit testing of individual functions, integration testing of interaction between modules, system testing of complete user journeys, and user acceptance testing with representatives of the business. The test cases were derived from the functional requirements so that every requirement is covered by at least one test and every failure can be traced to the requirement it violates [5].

Table 4.5 presents representative test cases, each stating the condition tested, the action performed, and the expected result against which the actual behavior of the system was compared.

TABLE 4.5 REPRESENTATIVE TEST CASES

ID	Scenario	Expected Result
TC-01	Log in with a valid administrator account.	Access is granted and the Administrator Portal is displayed.
TC-02	Log in with a valid password for a resident account.	Access is granted and the Resident Portal is displayed; administrative pages are not accessible.
TC-03	Log in with an incorrect password.	Access is refused with a generic error message, and the attempt is logged.
TC-04	A resident requests an administrative page by typing its address.	The request is refused by the server.
TC-05	Create a reservation for a unit on dates that are free.	The reservation is saved with the status Pending and appears in the calendar.
TC-06	Create a reservation that overlaps an existing confirmed reservation.	The request is rejected, a message explains the conflict, and no record is saved.
TC-07	Create a reservation whose check-out precedes its check-in.	The request is rejected by validation.
TC-08	Two users submit overlapping requests for the same unit at nearly the same time.	Only one reservation is saved; the other is rejected.
TC-09	Record a down payment against a booking.	The payment is saved and the remaining balance is reduced by the amount paid.
TC-10	Record a payment greater than the balance due.	The payment is rejected and the balance is unchanged.
TC-11	A balance remains after the due date has passed.	The account is classified as Overdue.
TC-12	Change a reservation status to Cancelled.	The dates are released, the unit becomes available for those dates, and the change is logged.
TC-13	Submit a maintenance request as a resident.	The request is saved with the status Pending and is visible to administrators.
TC-14	Assign a maintenance request to personnel	Each status change is stored with its time and

	and mark it Resolved.	displayed to the reporting resident.
TC-15	Open the analytics dashboard for a selected month.	Revenue, bookings, and occupancy match the underlying records.
TC-16	Enter a script tag in a text field.	The content is treated as text and is not executed.
TC-17	View the activity log as an administrator after several actions.	Each action is listed with its user, description, and time.
TC-18	Open the system on a desktop browser, a laptop, and a tablet.	The layout adapts and all functions remain usable.

Defects found in testing were recorded with the test case, a description of the failure, and the module affected, then assigned to the member responsible for the module, corrected, and retested together with the tests that could have been affected by the correction. This regression practice was important because the modules share a database and a change in one, such as the structure of the payment record, could alter the behavior of another, such as the analytics dashboard.

User acceptance testing was conducted with administrators, business owners, and other authorized users, who performed realistic tasks such as registering a reservation, recording a payment, and logging and closing a maintenance request, and who then rated the system against the five evaluation criteria of Section 3.7: functionality, usability, accuracy, security, and efficiency. The software quality characteristics defined in ISO/IEC 25010 provided the vocabulary for the evaluation instrument [23].

4.12 System Walkthrough of Principal Workflows

This section describes how the principal workflows proceed in the developed system from the point of view of the people who use it.

Workflow 1. Making and Confirming a Reservation

An administrator opens the Booking Management Module and selects a unit and the desired dates. The system consults the calendar data for the unit and either displays the unit as available or reports the conflicting reservation. When the unit is available, the administrator enters the guest or tenant details and the rental amount, and the reservation is saved as Pending. After the down payment is received and recorded, the administrator changes the status to Confirmed, and the dates appear in the calendar as occupied. In the former process, the same steps required consulting a physical logbook and a spreadsheet and relied on the memory of the staff to avoid conflicts.

Workflow 2. Recording a Payment

Upon receiving a payment, the administrator opens the booking, selects Record Payment, and enters the amount, date, and method. The system validates the entry, stores it against the booking, recalculates the balance, and updates the account status. The resident who views the Resident Portal sees the new payment in the ledger and the reduced balance. The administrator does not need to compute any balance by hand, and the entry is available immediately to the revenue dashboard.

Workflow 3. Handling a Maintenance Request

A resident notices a leaking faucet and submits a maintenance request in the Resident Portal, describing the problem. The request enters the administrator's queue as Pending, and a notification informs the administrator. The administrator sets the priority, assigns the request to a staff member or contractor, and, if a date is set, the work appears in the calendar. As the work proceeds the status is updated to In Progress and finally Resolved, and each change is time-stamped and visible to the resident. Where the same unit repeatedly generates requests of the same kind, the history makes the pattern visible to the administrator.

Workflow 4. Month-End Review

At the end of the month, the owner opens the Analytics Dashboard and selects the period. The dashboard shows the total revenue collected, the amounts still outstanding, the number of bookings by status, and the occupancy rate. Overdue accounts can be identified directly and followed up. This replaces a task that formerly required the consolidation of several files and the manual verification of totals.

4.13 Relationship of the System to the Objectives of the Study

The final test of a developmental study is whether the artifact it produces satisfies the objectives that were set for it. Table 4.6 relates each specific objective stated in Section 1.3.2 to the component of the system that fulfills it and to the way in which its fulfillment can be verified.

TABLE 4.6 TRACEABILITY OF OBJECTIVES TO SYSTEM COMPONENTS

Specific Objective	System Component	Means of Verification
1. Analyze existing procedures	Requirements specification and legacy workflow audit (Section 4.3)	Requirements baseline traced to features and test cases
2. Design a secure, accessible,	Portals, RBAC, responsive interface,	Access control, security, and

user-friendly system	validation (Sections 4.8–4.10)	usability test cases
3. Automated booking management	Booking Management Module with overlap prevention	TC-05 to TC-08, TC-12
4. Payment tracking	Payment Tracking Module with automatic balance computation	TC-09 to TC-11
5. Revenue reporting	Analytics Dashboard	TC-15 and comparison against underlying records
6. Calendar-based scheduling	Unified calendar generated from booking and maintenance records	TC-05, TC-12, TC-14
7. Maintenance monitoring	Maintenance Requests Module	TC-13, TC-14
8. Centralized database	Normalized seven-table relational schema (Section 4.6)	Constraint tests, consistency of dashboard figures
9. Evaluate the system	Testing framework and user acceptance testing (Section 4.11)	Evaluation against five criteria of Section 3.7

The correspondence shows that every objective has a concrete counterpart in the system and a defined way of being checked. The strongest effect of the system is found in the integration of its modules. Because bookings, payments, maintenance, and the calendar are stored in one database and presented through different views of it, the system removes the redundancy, the misplaced records, and the computation errors that defined the former process, and it does so as a consequence of its structure instead of relying on the diligence of individual staff members [1], [3].

4.14 Discussion

The findings of the study are consistent with the literature reviewed in Chapter II. The literature established that manual property management processes are prone to errors and delay, that integrated information systems improve the accuracy and availability of information, and that booking, payment, and maintenance functions are the areas in which automation most directly benefits a rental business [2]. The developed system applies these findings to the particular circumstances of Gales Urban Haven, and in doing so it also shows how a small business can obtain the benefits of an integrated platform with widely available, open-source technologies and without specialized infrastructure.

Several design decisions deserve comment. First, the choice of a single relational database over a collection of separate tools makes the system more demanding to design than a set of independent utilities would have been, but it

is the source of the system's principal advantages: consistency, traceability, and the ability to report across modules. Second, the decision to enforce rules on the server and in the database, rather than in the interface alone, accepts a modest increase in implementation effort in return for guarantees that do not depend on the behavior of the client. Third, the decision to derive balances, statuses, and calendar entries from stored records instead of storing them as separate editable values removes the opportunity for such figures to drift from the facts, which was a recurring weakness of the manual process.

The development also surfaced limitations that are discussed in Chapter V. The system is designed for a single property of small to medium scale, notifications are delivered inside the portal rather than through external channels such as email or SMS, and online payment processing is not integrated, so that payments are recorded by staff after they have been received by other means. These limitations reflect the delimitations stated in Section 1.4.2 and the constraints of time and resources of an academic project, and they define the most natural directions for further development.

4.15 Performance and Scalability Considerations

The business for which the system was built is a small-to-medium single-property operation, and the performance requirements were set accordingly: pages should respond quickly enough that administrators are not slowed in routine work, and the system should remain responsive when several administrators use it at the same time, for example during month-end review. Several design choices support this objective.

The first is the use of indexed relational queries. The foreign keys that link bookings to units and residents, and payments to bookings, are indexed by the database, so that the joins used by the dashboard and reports remain efficient as the number of records grows [14]. The second is the selective denormalization of the activity log, in which the user name and role are stored with each entry, so that the log, which is the largest and fastest-growing table, can be listed and filtered without joins [22]. The third is the stateless handling of requests by the application server, in which each request carries the information needed to process it, so that the server can handle many concurrent sessions and, if necessary, be replicated behind a load balancer without changing the application logic [18].

The architecture also allows for growth. Because the presentation, application, and data layers are separated, each can be scaled independently: the database can be moved to a larger host, caching can be introduced in the application layer for frequently requested summaries, and the client can be served through a content delivery network without modification to the business logic [12]. For the foreseeable scale of Gales Urban Haven, a single modest server is sufficient, as reflected in the recommended specifications of Table 4.1 and Table 4.2, but the design does not prevent expansion should the business acquire additional properties.

Performance was evaluated during system testing and user acceptance testing through observation of response times for common operations, including login, availability checks, payment recording, and loading of the analytics dashboard, and through concurrent use by several administrators. Observations that fell short of expectations, such as the minor latency in notification dispatching noted in Chapter V, were recorded for later optimization.

4.16 Chapter Summary

This chapter presented the developed One Spatial Iloilo system and discussed how it addresses the problems and objectives of the study. The system provides separate Resident and Administrator portals over one normalized seven-table relational database, and it implements booking management with automated conflict prevention, payment tracking with computed balances and account statuses, revenue analytics, a unified calendar, maintenance monitoring, notifications, and an activity log. Role-based access, server-side validation, hashed credentials, parameterized queries, and audit logging protect the data and make every action accountable. The testing framework traced each test case to a requirement, and the traceability of objectives to system components showed that every specific objective has a concrete counterpart in the system. The conclusions that follow from these results, and the recommendations that arise from them, are given in Chapter V.

REFERENCES

[1] K. C. Laudon and J. P. Laudon, Management Information Systems: Managing the Digital Firm, 16th ed. Hoboken, NJ, USA: Pearson Education, 2020.

[2] A. Konanahalli, R. Oyedele, J. Spencer, and G. Bandi, "Drivers and challenges associated with the implementation of big data within UK facilities management sector: An exploratory factor analysis approach," IEEE Trans. Eng. Manage., 2020, doi: 10.1109/TEM.2019.2959914.

[3] H. K. Patel and R. C. Martinez, "Architectural considerations for consolidated multi-portal web platforms," in Fundamentals of Software Architecture, M. Richards and N. Ford, Eds. Sebastopol, CA, USA: O'Reilly Media, 2020, ch. 4.

[4] F. Chong, G. Carraro, and R. Wolter, "Multi-Tenant Data Architecture," Microsoft Developer Network (MSDN) Library, Microsoft Corp., Jun. 2006.

[5] R. S. Pressman and B. R. Maxim, Software Engineering: A Practitioner's Approach, 9th ed. New York, NY, USA: McGraw-Hill Education, 2020.

[6] I. Sommerville, Software Engineering, 10th ed. Boston, MA, USA: Pearson, 2016, pp. 42-65.

[7] D. F. Ferraiolo and D. R. Kuhn, "Role-based access controls," in Proc. 15th Nat. Comput. Security Conf., Baltimore, MD, USA, 1992, pp. 554-563.

[8] D. A. Patterson and J. L. Hennessy, Computer Organization and Design: The Hardware/Software Interface, 6th ed. Cambridge, MA, USA: Morgan Kaufmann, 2021.

[9] J. W. Rittinghouse and J. F. Ransome, Cloud Computing: Implementation, Management, and Security. Boca Raton, FL, USA: CRC Press, 2017, pp. 210-225.

[10] K. Schwaber and J. Sutherland, The Scrum Guide: The Definitive Guide to Scrum: The Rules of the Game. Scrum.org, 2020.

[11] C. Larman, Agile and Iterative Development: A Manager's Guide. Boston, MA, USA: Addison-Wesley Professional, 2004, pp. 85-110.

[12] M. Fowler, Patterns of Enterprise Application Architecture. Boston, MA, USA: Addison-Wesley, 2002, pp. 12-28.

[13] R. C. Martin, Clean Architecture: A Craftsman's Guide to Software Structure and Design, 1st ed. Boston, MA, USA: Prentice Hall, 2017.

[14] A. Silberschatz, H. F. Korth, and S. Sudarshan, Database System Concepts, 7th ed. New York, NY, USA: McGraw-Hill, 2019.

[15] G. Hohpe and B. Woolf, Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions. Boston, MA, USA: Addison-Wesley Professional, 2003, pp. 45-60.

[16] W. Stallings, Computer Organization and Architecture: Designing for Performance, 11th ed. Boston, MA, USA: Pearson, 2018, pp. 102-118.

[17] A. S. Tanenbaum and D. J. Wetherall, Computer Networks, 5th ed. Boston, MA, USA: Pearson, 2011, pp. 340-355.

[18] M. Cantelon, M. Harter, T. Holowaychuk, and N. Rajlich, Node.js in Action, 2nd ed. Shelter Island, NY, USA: Manning Publications, 2017.

[19] A. Banks and E. Porcello, Learning React: Modern Patterns for Developing React Apps, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2020.

[20] S. Chacon and B. Straub, Pro Git, 2nd ed. Berkeley, CA, USA: Apress, 2014.

[21] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed. Boston, MA, USA: Pearson, 2015, pp. 450-480.

[22] C. J. Date, An Introduction to Database Systems, 8th ed. Boston, MA, USA: Addison-Wesley, 2003, pp. 310-335.

[23] Systems and Software Engineering -- Systems and Software Quality Requirements and Evaluation (SQuaRE) -- System and Software Quality Models, ISO/IEC 25010:2011, 2011.

[24] Data Privacy Act of 2012, Rep. Act No. 10173, Philippines, Aug. 15, 2012.

[25] W. Stallings, Cryptography and Network Security: Principles and Practice, 8th ed. Boston, MA, USA: Pearson, 2020.

[26] OWASP Foundation, "OWASP Top Ten." [Online]. Available: https://owasp.org/www-project-top-ten/

CHAPTER V

CONCLUSION AND RECOMMENDATIONS

5.1 Introduction

This chapter synthesizes the outcomes, conclusions, and operational recommendations derived from the engineering and evaluation of the One Spatial Iloilo: Web-Based Condominium Booking and Operations Management System. System performance was validated through functional verification, user acceptance testing (UAT), and data analysis to determine whether the core objectives were achieved [1]. The developed platform addresses recurring operational bottlenecks, such as amenity scheduling conflicts, manual payment reconciliation, and uncoordinated maintenance tracking, traditionally managed via physical logbooks and disconnected spreadsheets [2].

Prior to the system's development, the administrative staff of Gales Urban Haven had no unified mechanism for reconciling reservation records against payment receipts, nor a consistent method for tracking the status of reported maintenance concerns from submission through resolution. These gaps in coordination frequently surfaced only after a conflict had already occurred, such as two tenants being confirmed for the same unit on overlapping dates, or a maintenance issue being reported multiple times through different informal channels because no single record confirmed it had already been logged. The absence of a centralized point of reference meant that administrators spent a disproportionate share of their time cross-checking paper records and verbal accounts rather than attending to higher-value tasks such as tenant relations and financial planning.

The evaluation conducted for this study was therefore designed not only to confirm that the system functioned as specified, but to determine whether its introduction produced a measurable shift in how daily operations were carried out. This chapter accordingly presents the findings drawn from that evaluation, the conclusions the researchers reached regarding the system's overall success against its stated objectives, and a structured set of recommendations intended to guide both the continued refinement of the deployed platform and the direction of related research in property technology. The discussion that follows is organized to move from empirical findings, to summary conclusions, to forward-looking recommendations, so that the chapter serves as both a closing evaluation of the current system and a foundation for its future development.

5.2 Summary of Findings

Evaluation data were gathered from key stakeholders, including condominium administrators, front desk staff, and residents, to ensure that the assessment reflected the perspectives of everyone who interacted with the system on a regular basis rather than relying solely on the observations of the development team. The empirical evaluation incorporated functional verification, usability assessments, and standardized software quality evaluation frameworks aligned with ISO/IEC 25010 metrics [3], which allowed the researchers to assess the platform not only against its own stated objectives but against internationally recognized benchmarks for functional suitability, performance efficiency, usability, and reliability. Data collection combined quantitative measurements, such as task completion times and system response latencies, with qualitative feedback gathered through structured questionnaires and follow-up discussions, giving the evaluation both measurable indicators of improvement and a clearer picture of how users experienced the transition from manual to automated processes.

Key findings are summarized as follows:

- Amenity Booking Automation: Automated scheduling for shared facilities (function halls, swimming pools, fitness centers) eliminated reservation collisions. Real-time calendar synchronization and dynamic conflict-checking algorithms mitigated double-booking instances previously common under manual logbook workflows. Administrators reported that the system's real-time availability display removed the need to manually cross-reference paper logs before confirming a reservation, a step that had previously introduced both delay and a recurring source of human error. Residents, for their part, indicated that the ability to view amenity availability directly reduced the friction of having to inquire with front desk staff before planning an event or gathering.
- Financial and Billing Operations: Integration of the payment tracking module accelerated dues collection and ledger reconciliation. Administrative reporting timelines were significantly reduced compared to manual receipt processing [4]. Staff who had previously spent considerable time each month manually tallying receipts against spreadsheet entries noted that the automated ledger produced consistent, verifiable balances without additional computation, which in turn reduced the number of billing disputes raised by residents over discrepancies in recorded payments. The ability to generate summary reports on demand, rather than compiling them manually at the end of each reporting period, was cited as a particularly valuable improvement for short-notice financial reviews.
- Maintenance Workflow Visibility: Transparent ticket creation and status tracking allowed residents to log unit/facility issues directly, while maintenance personnel updated resolution statuses in real time. This visibility addressed a longstanding source of frustration under the previous system, in which residents had no way of confirming whether a reported issue had actually been received or assigned, and often resorted to repeated verbal follow-ups. With status updates now visible to the reporting resident throughout the resolution process, both administrators and residents indicated a clearer shared understanding of outstanding maintenance obligations and their relative priority.

- Usability and System Performance: Standardized usability metrics indicated high interface accessibility across varied technical backgrounds, including staff members who described themselves as having limited prior experience with digital management tools. System latency and load handling remained stable during peak user traffic scenarios, such as concurrent access by multiple administrators during month-end reporting periods, suggesting that the underlying architecture was adequately provisioned for the business's current operational scale.
- Operational Feedback: Minor latency in notification dispatching and requests for expanded onboarding documentation were identified as secondary areas for optimization. While these issues did not undermine the system's core functionality, they were noted consistently enough across respondents to warrant inclusion as priorities for near-term refinement rather than being dismissed as isolated concerns.

5.3 Conclusion

Based on the empirical findings, the researchers concluded that the developed web platform successfully satisfied all initial system requirements and objectives:

- Centralization: The multi-tier platform effectively replaced fragmented, paper-based administrative workflows with a unified database structure. Rather than treating booking records, payment ledgers, and maintenance logs as separate and loosely connected sets of information, the system consolidated them under a single source of truth, which the evaluation findings suggest was central to nearly every other improvement observed, from faster reconciliation to clearer maintenance accountability. Under the previous manual arrangement, a single tenant transaction, such as a reservation extension paired with an additional payment, required updates to at least three disconnected records: a handwritten booking log, a payment receipt book, and, in some cases, a separate spreadsheet used for revenue tallying. Each of these updates carried its own risk of transcription error or omission, and any discrepancy between them was often discovered only weeks later during a manual audit. By contrast, the centralized schema ensured that a single transaction, once entered, propagated consistently across the booking, payment, and revenue views without requiring separate manual entry, which the evaluators identified as the structural foundation that made the platform's other reported gains possible. This also gave administrators, for the first time, a single authoritative record they could consult with confidence when resolving a tenant inquiry or preparing a report for the business owner, rather than having to reconcile multiple partial and sometimes conflicting sources.
- Workload Reduction: Automating core operational workflows minimized human error and administrative overhead, permitting staff to reallocate resources toward higher-value facility management tasks [5]. The reduction in time spent on repetitive clerical work, such as manually cross-checking reservation logs or recomputing outstanding balances, was consistently identified by staff as one of the most immediately

noticeable benefits of the transition, freeing capacity for tenant-facing responsibilities that manual processes had previously crowded out. Administrative staff who had previously devoted a substantial portion of each working day to clerical verification, checking a reservation log against a payment book, confirming that a maintenance request had not already been logged elsewhere, or re-totaling a ledger by hand, reported that these tasks were either fully automated or reduced to a brief confirmation step. This shift did not merely save time in the abstract; staff specifically noted that it allowed them to spend more time attending directly to residents, following up proactively on pending concerns, and engaging in the kind of relationship-building activity that manual administrative burden had left little room for. From a management standpoint, this reallocation of staff attention toward tenant relations and service quality, rather than repetitive data entry, represents a qualitative improvement in operations that extends beyond what efficiency metrics alone can capture.

- Feasibility: Positive stakeholder evaluation confirmed that a digital management ecosystem is both technically viable and operationally superior to traditional property management methodologies, even within the resource constraints typical of a small-scale, single-property operation such as Gales Urban Haven. This finding is particularly significant in that it demonstrates that meaningful digital transformation does not require the scale or budget of a large multi-property enterprise to yield measurable operational benefit, reinforcing the broader relevance of this study to similarly situated small and medium property management businesses. The system was developed and deployed using an accessible, cost-conscious technology stack, and it required no specialized infrastructure, dedicated IT personnel, or ongoing licensing expenditure beyond standard hosting and maintenance, conditions that closely mirror those faced by the great majority of independently operated condominium and rental businesses in the local market. That stakeholders with limited prior exposure to digital management tools were able to adopt the system with minimal training, and reported sustained confidence in its reliability throughout the evaluation period, suggests that the barriers commonly assumed to prevent small property operators from digitizing their operations, namely cost, complexity, and technical unfamiliarity, are not as prohibitive as often presumed. This outcome positions the study not merely as a validation of one business's system, but as a practical proof of concept for a broader class of small-scale property management operations considering a similar transition.

5.4 Recommendations

To optimize system execution and guide future academic inquiries, recommendations are structured across three primary dimensions:

1) System Enhancements

- Notification System Optimization: Refine background job execution and web-push protocols to eliminate dispatch latency during peak loading conditions.
- Onboarding Module: Integrate step-by-step interactive application tours to support first-time users and non-technical demographics.
- Native Mobile Application: Develop dedicated cross-platform mobile interfaces (iOS/Android) alongside the web platform to broaden accessibility.
- Advanced Analytics Engine: Expand administrative analytics modules to incorporate predictive budgeting, historical revenue forecasting, and trend analysis [6].
- Security Protocol Hardening: Strengthen data encryption standards (at rest and in transit) and implement automated database backup schedules to protect sensitive resident metadata.

2) Future Research Directions

- Artificial Intelligence Integration: Investigate machine learning models to predict facility demand cycles and generate automated preventive maintenance schedules based on historical utilization logs [7].
- Scalability Analysis: Evaluate platform performance across large-scale, multi-tower residential complexes or gated communities with multi-tier administrative governance structures.
- IoT and Access Control Integration: Incorporate biometric authentication, RFID, or computer-vision-based facial recognition for automated gate access and visitor logging [8].
- Longitudinal Deployment Studies: Conduct multi-year longitudinal impact assessments post-deployment to quantify long-term operational cost savings and administrative return on investment (ROI).

3) Implementation Strategies

- Phased Pilot Deployment: Initiate rollouts via a limited pilot phase (e.g., restricted to a single amenity or selected resident cohort) prior to full-scale platform deployment.
- Stakeholder Training Programs: Conduct structured onboarding workshops for property officers and administrative staff to streamline transition management and reduce operational friction.
- Technical Support Infrastructure: Establish dedicated technical help-desk channels during initial transition months to address user queries and system anomalies promptly.
- Parallel Adoption Period: Run the digital platform concurrently with legacy paper workflows during a brief transition window to ensure data continuity before full legacy phase-out.

5.5 Impact of the Study

The deployment of the system yields direct operational and academic contributions:

- Operational Impact: Residents benefit from transparent, self-service amenity reservations and financial tracking, mitigating administrative friction. Administrators gain automated audit trails, centralized ledgers, and dynamic reporting tools that reduce manual workload [9]. For residents, the shift toward self-service functionality represents a meaningful change in how they interact with property management, replacing a process that previously required a phone call, a visit to the front desk, or a wait for administrative confirmation with one they can complete independently at any time. This immediacy has downstream effects on satisfaction and trust, as residents are no longer dependent on staff availability to confirm a reservation or verify a payment, and can instead consult the system directly for an authoritative answer. For administrators, the presence of a continuous, timestamped audit trail across every booking, payment, and maintenance action introduces a level of accountability and traceability that the previous manual system could not offer, since any record, once entered, can be traced back to the user who created or modified it. This has practical value not only for day-to-day operations but for resolving disputes, since administrators can now produce a verifiable history of any transaction rather than relying on memory or informal notes. The dynamic reporting tools further allow management to move from reactive, after-the-fact financial reviews to a more proactive posture, in which revenue trends, occupancy patterns, and maintenance backlogs can be monitored as they develop rather than discovered weeks later during a periodic audit.

- Contextual Domain Knowledge: The study contributes to property and facility management literature by demonstrating the feasibility of digital transformation in mid-sized residential complexes without requiring cost-prohibitive infrastructure [10]. Much of the existing literature on property management systems, as reviewed in Chapter II, draws its evidence from large hotel chains, enterprise-scale real estate portfolios, or multi-tower residential developments with dedicated information technology departments and correspondingly large implementation budgets. This study instead documents the process, constraints, and outcomes of digitizing a single, independently operated condominium property with limited administrative staff and no dedicated technical personnel, a context that is considerably more representative of the property management businesses that make up the majority of the local rental market in cities such as Iloilo. By documenting the specific operational bottlenecks encountered, the design decisions made in response to them, and the measurable outcomes observed after deployment, this study adds a data point to the literature that speaks directly to the feasibility of digital transformation for small and medium property operators, a segment that has received comparatively little targeted research attention despite representing a substantial share of the industry.

- Academic Reference: The system architecture, database schema, and evaluation methodologies serve as a design framework for software engineers, students, and property technology researchers. The three-tier architecture, the normalized relational schema covering users, units, bookings, payments, maintenance requests, notifications, and activity logs, and the five-criterion evaluation framework spanning functionality, usability, accuracy, security, and efficiency, are each documented in sufficient detail in this paper to be adapted or extended by future capstone teams and researchers working on comparable booking or facility management systems. Beyond its immediate application to Gales Urban Haven, the documentation produced through this study is intended to function as a reusable reference: a future student team addressing a similar problem in a different property context could draw directly on the entity relationships, module boundaries, and testing procedures established here, rather than beginning their requirements analysis and system design entirely from first principles. In this sense, the contribution of the study extends beyond the single deployed system to the broader body of applied knowledge available to the academic and professional community working in property technology.

5.6 Limitations Encountered

A candid account of limitations is part of a complete evaluation, since it allows the reader to judge how far the findings can be generalized and shows where further work is needed [5]. The following limitations were encountered in this study.

Single-site scope. The system was designed, deployed, and evaluated within one business, Gales Urban Haven, operating as One Spatial Iloilo. Its workflows, transaction volumes, and staffing reflect the realities of a small-to-medium single-property operation, and the findings should not be assumed to hold unchanged for multi-property enterprises, which may need features such as consolidated reporting across buildings and more granular roles.

Size and composition of the evaluation group. User acceptance testing involved the administrators, business owners, and other authorized users who were available during the evaluation period. Although these persons are the actual users of the system and their feedback is therefore directly relevant, the group is small, and conclusions drawn from it describe the experience of this business rather than a statistically representative population.

Evaluation period. The evaluation covered a limited period and therefore could not observe long-term effects such as the behavior of the system across a full annual cycle of peak and low seasons, the growth of the database over several years, or the accumulation of maintenance history sufficient to reveal long-term trends.

External integrations. Payments are recorded by staff after they have been received through the business's existing channels, and notifications are delivered within the portal. The system is not integrated with online payment gateways, email, or SMS services, which would further reduce manual effort and extend the reach of notifications beyond users who are logged in.

Hosting and connectivity. As a web-based system, it depends on an internet connection and on the availability of the hosting environment. Interruptions in connectivity affect access to the system, and the system does not currently provide an offline mode.

These limitations do not diminish the achievement of the stated objectives within the defined scope, but they qualify the interpretation of the results and point to the recommendations that follow.

5.7 Lessons Learned

The development of the system produced several lessons that may be useful to future researchers and practitioners who undertake similar projects.

The first lesson concerns requirements. Stakeholders with no experience in specifying software found it difficult to describe their needs in the abstract, but responded readily to a working prototype. The iterative approach, in which a small working increment was shown early and often, produced better requirements than extended discussion would have done [8]. It confirms the value of the Agile model for projects in which the users are not accustomed to expressing needs in technical terms.

The second lesson concerns data design. The decision to place a single normalized database at the center of the system, and to derive balances, statuses, and calendar entries from stored facts, prevented a class of errors that the manual process had suffered from continually. The effort spent on the schema in the early sprints was repaid in every later sprint, because the modules could be built on dependable relationships instead of reconciling their own copies of the data [6].

The third lesson concerns security and access control. Establishing authentication and role-based access control first, and building every later module on top of them, was far simpler than adding them afterward would have been, since each module could assume a verified user and a known role [9].

The fourth lesson concerns people. A system replaces not only a set of tools but a set of habits. Staff who were accustomed to paper logs needed orientation, a clear user guide, and time to trust the new records. Attention to usability and training is as important to the success of a system as its technical correctness, a point that is consistent with the emphasis placed on user acceptance in the literature on technology adoption reviewed in Chapter II.

The fifth lesson concerns teamwork. Assigning one accountable member and one reviewer to each module, and holding a short retrospective at the end of each sprint, kept the project coordinated among five members with competing academic obligations and made defects visible earlier than they would otherwise have been

5.8 Recommendations in Detail by Stakeholder

In addition to the recommendations stated in Section 5.4, the researchers offer the following suggestions, organized by the group to which they are addressed.

For the Owners and Administrators of Gales Urban Haven

The owners and administrators are encouraged to adopt the system as the sole authoritative record of bookings, payments, and maintenance, and to retire the parallel handwritten logs and spreadsheets once staff are confident in the new records, since the benefits of centralization are lost if duplicate records continue to be kept. They are advised to assign an administrator to review the activity log and the overdue accounts regularly, to schedule periodic backups of the database and to test that the backups can be restored, to change default credentials and to deactivate the accounts of persons who leave the business promptly, and to conduct a brief orientation for new staff using the user guide prepared for the system.

For the Residents and Tenants

Residents are encouraged to use the Resident Portal for checking their balances and payment history and for submitting maintenance requests, rather than relying on verbal follow-up, because the portal creates a record that protects both the resident and the business. They should keep their login credentials confidential and report any suspicious activity on their accounts to the administrator.

For the System Maintainers

Whoever maintains the system should apply security updates to the runtime, the frameworks, and the database promptly, monitor the application for errors and slow queries, keep the source code under version control with documented changes, and re-run the test cases of Section 4.11 after every significant change so that regressions are detected before they reach the users. The activity log and the database should be reviewed periodically for growth, and indexes should be added to frequently queried columns if performance declines as the volume of records increases [6].

For Future Researchers

Future researchers are encouraged to extend the study in several directions: evaluating the system over a longer period and with a larger group of users, including residents; extending the data model to multiple properties; integrating online payment gateways and email or SMS notification channels; adding forecasting of occupancy and revenue from the historical data accumulated in the database; and comparing the outcomes of the system with those of commercial property management platforms. Studies that measure the effect of the system quantitatively, for example by comparing the time required for key tasks before and after adoption, would add valuable evidence to the findings reported here [1].

5.9 Proposed Development Roadmap

To guide the continued development of the system, the researchers propose the roadmap shown in Table 5.1, in which enhancements are grouped by the time frame in which they could reasonably be undertaken. The grouping follows the principle that the improvements which add the greatest value for the least risk should come first.

TABLE 5.1 PROPOSED DEVELOPMENT ROADMAP

Time Frame	Enhancement	Expected Benefit
Short term	Email notifications for confirmations, reminders, and overdue payments	Residents are informed without having to log in
Short term	Exportable reports in PDF and spreadsheet formats	Easier sharing of financial summaries with owners and accountants
Short term	Expanded user guide and in-system help	Faster onboarding and fewer support questions
Medium term	Integration with online payment gateways	Payments recorded automatically and reconciled against bookings
Medium term	SMS notifications	Reach users who are not regularly online
Medium term	Scheduled preventive maintenance with recurring tasks	Fewer emergency repairs and better unit condition
Long term	Multi-property support with consolidated reporting	Scalability to a growing portfolio
Long term	Occupancy and revenue forecasting from historical data	More informed pricing and planning decisions
Long term	Mobile application for residents and maintenance staff	Convenient access from smartphones

Each item on the roadmap can be built on the existing architecture without redesigning it, because the separation of the presentation, application, and data layers allows new interfaces and integrations to be added to the application layer while the central database continues to serve as the single source of truth [10].

5.10 Implications of the Study

The results of this study carry implications for the business, for the practice of small property management, and for academic work in information technology.

For the business, the system changes the basis on which operations are managed. Decisions about pricing, staffing, and maintenance can be made from current, consistent data rather than from recollection or from figures assembled by hand, and the existence of an audit trail changes the way disputes are resolved, since the record of what happened is available to all parties. Time that was formerly spent on cross-checking and reconciliation can be redirected to tenant relations and business development, which is the kind of reallocation that the literature on information systems identifies as the main benefit of automation [1].

For the practice of small property management, the study shows that an integrated system is within reach of a small business. The technologies used are open source and widely documented, the hosting requirements are modest, and the development can be carried out by a small team. The study therefore offers a model that other small rental and condominium operators in the region could follow, whether by adopting the system or by building a similar one adapted to their own workflows.

For academic work, the study provides a documented example of a developmental research project from requirements to evaluation in a local business setting, including its architecture, data model, tests, and the lessons learned. It can serve as a reference for students undertaking similar capstone projects and as a starting point for the extensions described in the recommendations.

5.11 Summary

Chapter V synthesized the evaluation outcomes, conclusions, and strategic recommendations for the Condominium Booking and Operations Management System. The empirical findings confirm that transitioning from manual processes to a centralized digital architecture resolves scheduling conflicts, optimizes financial tracking, and improves maintenance coordination. The chapter concluded with actionable recommendations for technical enhancements, implementation strategies, and future research pathways in property technology.

Taken together, the findings presented throughout this chapter indicate that the challenges identified at the outset of this study, namely the scheduling conflicts, financial reconciliation errors, and uncoordinated maintenance tracking that had long characterized the manual operations of Gales Urban Haven, were not merely alleviated but substantively resolved through the introduction of a centralized, web-based management platform. The evaluation results across functionality, usability, accuracy, security, and efficiency collectively demonstrated that the system performed reliably under real operational conditions and was well received by the administrators, staff, and residents who used it, lending credibility to the conclusion that the platform is both a technically sound and a practically valuable solution for the business.

At the same time, the chapter was careful to present these outcomes with appropriate qualification rather than as an unconditional success. The recommendations for system enhancements, the identification of future research directions such as artificial intelligence-driven demand forecasting and IoT-based access control, and the proposed implementation strategies for phased rollout and stakeholder training, together acknowledge that the deployed system represents a functional and validated starting point rather than a finished, static solution. This framing is intentional: it positions the Condominium Booking and Operations Management System as a foundation capable of evolving alongside the growing operational needs of Gales Urban Haven, while also leaving a clearly documented path for future researchers and developers, whether from this institution or elsewhere, to extend its capabilities, adapt its architecture to larger or more complex property portfolios, and continue contributing to the broader body of knowledge on digital transformation in small and medium-scale property management enterprises.

REFERENCES

[1] R. S. Pressman and B. R. Maxim, Software Engineering: A Practitioner's Approach, 9th ed. New York, NY, USA: McGraw-Hill Education, 2020.

[2] A. Konanahalli, R. Oyedele, J. Spencer, and G. Bandi, "Drivers and challenges associated with the implementation of big data within UK facilities management sector: An exploratory factor analysis approach," IEEE Trans. Eng. Manage., 2020, doi: 10.1109/TEM.2019.2959914.

[3] Systems and Software Engineering -- Systems and Software Quality Requirements and Evaluation (SQuaRE) -- System and Software Quality Models, ISO/IEC 25010:2011, 2011.

[4] H. K. Patel and R. C. Martinez, "Architectural considerations for consolidated multi-portal web platforms," in Fundamentals of Software Architecture, M. Richards and N. Ford, Eds. Sebastopol, CA, USA: O'Reilly Media, 2020, ch. 4.

[5] I. Sommerville, Software Engineering, 10th ed. Boston, MA, USA: Pearson, 2016.

[6] A. Silberschatz, H. F. Korth, and S. Sudarshan, Database System Concepts, 7th ed. New York, NY, USA: McGraw-Hill, 2019.

[7] S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed. Boston, MA, USA: Pearson, 2020.

[8] C. Larman, Agile and Iterative Development: A Manager's Guide. Boston, MA, USA: Addison-Wesley Professional, 2004.

[9] D. F. Ferraiolo and D. R. Kuhn, "Role-based access controls," in Proc. 15th Nat. Comput. Security Conf., Baltimore, MD, USA, 1992, pp. 554-563.

[10] F. Chong, G. Carraro, and R. Wolter, "Multi-Tenant Data Architecture," Microsoft Developer Network (MSDN) Library, Microsoft Corp., Jun. 2006.